# Succinct Structure Representations for Efficient Query Optimization

Anonymous Author(s)

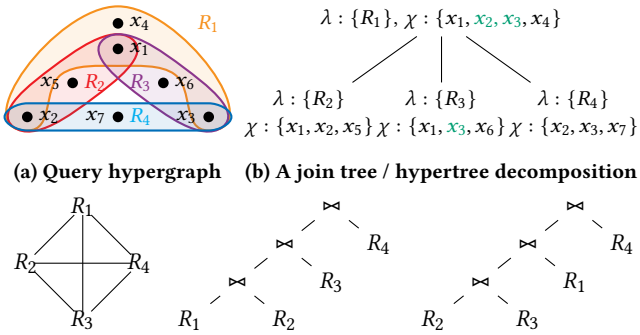

## Abstract

Structural decomposition methods offer powerful theoretical guarantees for join evaluation, yet they are rarely used in real-world query optimizers. A major reason is the difficulty of combining cost-based plan search and structure-based evaluation. In this work, we bridge this gap by introducing meta-decompositions for acyclic queries, a novel representation that succinctly represents all possible join trees and enables their efficient enumeration. Meta-decompositions can be constructed in polynomial time and have sizes linear in the query size. Building on this representation, we design an efficient polynomial-time cost-based optimizer based directly on the meta-decomposition, without the need to explicitly enumerate all possible join trees. We characterize plans found by this approach using a novel notion of width, which effectively shows the theoretical worst-case asymptotic bounds of intermediate result sizes and running time of any query plan. Experimental results demonstrate that, in practice, the plans in our class are consistently comparable to—even in many cases better than—the optimal ones found by the state-of-the-art dynamic programming approach, especially on large and complex queries, while our planning process runs orders of magnitude faster, comparable to the time taken by common heuristic methods.


## 1 Introduction

Joins are the cornerstone of relational database queries. However, query optimizers face extreme difficulty when planning large multi-way join queries, as the search space of join orders grows exponentially as the number of relations increases. In fact, finding the optimal join order of a query is well-known to be NP-hard in general [13, 41]. For this reason, beyond a handful of joins (e.g., more than 12), current query optimizers must abandon exhaustive dynamic programming and resort to heuristic or greedy algorithms. As a result, real-world optimizers often miss the best plans and produce suboptimal ones that can be orders of magnitude slower.

Nevertheless, the database theory community has long been providing tools that exploit query structures to tackle this complexity. These approaches usually *decompose* queries into specific tree structures, such as join trees [52] using the GYO algorithm [25, 53], which are later generalized to hypertree decompositions [21]. The structures of the resulting decompositions effectively capture the hardness of queries and, through specific requirements on connectivity, allow one to evaluate queries with theoretically-guaranteed instance-optimal complexities using the Yannakakis algorithm [7, 52] or its variants [21, 47]. Over the years, these structure-based methods have been generalized to evaluate joins with projections [4, 15, 27], aggregations [2, 29], differences [28, 56], top-$k$ [48], comparisons [49], etc., all with strong theoretical performance guarantees. These results suggest that leveraging a query's structural properties could be of great help for query optimization and could dramatically improve query evaluation, with robust




(a) Query hypergraph (b) A join tree / hypertree decomposition

(c) Query graph (d) A width-1 query plan (e) A width-2 query plan

**Figure 1: The hypergraph, a join tree, the query graph, and query plans of the query in Example 1.1**

worst-case bounds. Yet, in practice, despite limited attempts by, e.g., Scarcello et al. [40], RelationalAI [3], and Gottlob et al. [20], real-world systems rarely utilize such structure-based approaches. Therefore, they continue to struggle with large, complex queries.

Why are structure-based methods rarely adopted in mainstream systems? A core obstacle is the dichotomy between cost-based plan search and structure-based evaluation [31]. Current cost-based optimizers usually search for query plans with optimal or near-optimal cost under some cost model using (1) **exact algorithms**, mostly based on dynamic programming, such as [36, 42, 43, 45, 46], which have to run in exponential time, and thus do not scale to very large numbers of relations; (2) **heuristic strategies** such as greedy operator ordering (GOO) [17], iterative dynamic programming [30], UnionDP [34], and adaptive approaches with multiple options [9, 37], which have guarantees of neither optimality nor a reasonable approximation factor unless under some assumption such as independence of predicate selectivities; (3) **transformation-based techniques**, such as the Volcano/Cascades framework [23, 24] and genetic algorithms [6]; and/or (4) more recently, **machine-learning-based tuning and rewriting** [12, 33, 35, 51, 58, 59]. Although certain approaches leverage, e.g., *query graphs* [9, 34, 36, 37], they only show the existence of join attributes between relations but do not have a clear connection with the requirements of the aforementioned structural decompositions that help reason about asymptotic complexities.

*Example 1.1.* Consider the query

$$Q = \pi_{\emptyset}(R_1[x_1, x_2, x_3, x_4] \bowtie R_2[x_1, x_2, x_5] \bowtie R_3[x_1, x_3, x_6] \bowtie R_4[x_2, x_3, x_7]),$$

which is a Boolean acyclic query. Its hypergraph is shown in Fig. 1a, and a join tree, which is also a hypertree decomposition, in Fig. 1b. Its query graph, as shown in Fig. 1c, is a clique with the four relations as vertices, as each pair shares some join attribute. Fig. 1d is a query plan that follows the structure of the join tree (which

we formally define later). Fig. 1e does not, but it is still within the search space of traditional algorithms according to the query graph.

On the other hand, theoretical research typically focuses on the worst-case asymptotic data complexity, assuming the cost difference among decompositions is negligible. In practice, however, different decompositions may lead to significantly different concrete cost values. Therefore, using solely the structural properties such as hypertree widths is not sufficient [20]. However, selecting the best decomposition is challenging, as there can be an exponentially large number of candidates. Without an efficient cost-based exploration of alternatives, explicitly scanning through all of them would be computationally prohibitive. Furthermore, evaluation plans for structure-based methods are not natively supported by the current systems using standard binary query plans [20, 31]. As a result, deploying them requires a dedicated optimization and execution pipeline, which limits the practical adoption of these approaches.

P11 R1.O7

## 1.1 Our Contributions

In this work, we take a major step towards bridging structural decomposition techniques with cost-based query optimization. We start by introducing **a novel *width* measurement for query plans** and show that, for query plans of width 1, we can effectively embed the query's tree structure within a traditional query plan to guide query planning even in the absence of cost information. In particular, we show that acyclic Boolean queries and relation-dominated queries can be evaluated in *instance-optimal time* using width-1 plans. We also present a method which, for any given join tree of a query, constructs the optimal width-1 query plan that adheres to the join tree in $O(|Q|)$, *linear in the query size* $|Q|$.

By searching for plans of minimal width, we restrict the search space to only those that follow the query's optimal tree decompositions. To make this practical, we introduce *meta-decompositions* for acyclic conjunctive queries that **universally encode, in one succinct structure, the entire family of join trees for the query**. Despite the possibly exponential number of join trees of a given query, this representation **can be constructed in polynomial time and has size linear in the query size**. We give an efficient procedure that enumerates all join trees in polynomial amortized delay from a meta-decomposition, as a potential tool for other structure-based methods requiring explicit join tree enumeration.

Nevertheless, such an enumeration still incurs an exponential overhead. Therefore, we develop an efficient ***structure-guided cost-based optimization* framework to find the optimal width-1 query plan among all join trees** without naïvely scanning all candidates. For querys whose meta-decompositions have bounded fan-out, *our algorithm finds the optimal width-1 plan in linear time.* For general queries, we also provide efficient heuristics that have the same linear complexity with a low constant factor.

Although we currently restrict our focus to acyclic queries only, such queries in fact account for the overwhelming majority of real workloads [10]: among 8,125 queries across popular benchmarks, 7,929 (97.59%) are acyclic (See Appendix B.1 for detailed breakdowns). Structure-based methods for optimizing acyclic queries have therefore been of sustained interest [5, 8, 28, 47–50, 52, 57].

We then empirically demonstrate that our approach can indeed be game-changing for query optimization for large joins. We **implement an optimizer metaDecomp** and evaluate it on queries from 4 benchmarks, which include many large, complex queries. We compare the results against several state-of-the-art approaches. The results show that width-1 plans consistently match or even outperform the plans found by these baselines, while taking very small amount of time for optimization. We also demonstrate that join tree enumeration can be orders-of-magnitude faster than existing approaches based on naïve GYO reductions, and that selecting join trees using our cost-based optimization framework significantly improves the effectiveness of existing structure-based methods.

*Paper organization.* The rest of the paper is organized as follows: After laying out the necessary background in Section 2, we first define the *width* notion of query plans and discuss the connection between query structures and the widths of query plans in Section 3. In Section 4, we define *meta-decompositions*, the algorithm to construct them, as well as the way to enumerate all join trees based on them. In Section 5, we demonstrate how to perform cost-based optimization using meta-decompositions. In Section 6, we present experimental results to show the efficiency of query plans generated by this approach. We conclude and discuss possible future work in Section 7. Due to the space constraints, all missing proofs can be found in the appendix.

# 2 Preliminaries

We write $2^S$ for the set of all subsets of a set $S$. If $f : A \to B$ and $S \subseteq A$, we let $f(S) = \{f(a) : a \in S\}$. For a set of sets $S$, we write $\cup S = \bigcup_{x \in S} x$. If $f : A \to T$, where $T$ is a set, and $S \subseteq A$, we let $f(S) = \bigcup_{x \in S} f(x)$.

## 2.1 Conjunctive Queries and Hypergraphs

Conjunctive queries can be represented by hypergraphs, and our terminology follows the seminal references on this topic [18, 19, 21, 22, 47]. Let $\mathcal{D} = \{R_1[\bar{x}_1], \cdots, R_n[\bar{x}_n]\}$ be a database consisting of $n$ relations, where $\bar{x}_i$ is the set of attributes of relation $R_i$. A conjunctive query (with self-predicates) is defined as

$$Q \leftarrow \pi_{\bar{y}} \left(\boldsymbol{\sigma_1}(R_1[\bar{x}_1]) \bowtie \cdots \bowtie \boldsymbol{\sigma_n}(R_n[\bar{x}_n])\right),$$

where $\boldsymbol{\sigma_i}$ is the set of selection predicates involving only attributes in $\bar{x}_i$, $\bar{y}$ is the set of output attributes, and $\bowtie$ denotes natural join (with attribute renaming to avoid ambiguity). If $\bar{y} = \bigcup_{i \in [n]} \bar{x}_i$, the query is called a full join query and we omit $\bar{y}$ for simplicity; if $\bar{y} = \emptyset$, the query is a boolean query that returns true if there exists a tuple satisfying all specified joins and predicates, or false otherwise. We also require the query to be *self-join-free* at the logical level: If multiple relations correspond to the same physical relation, they are treated as distinct logical relations with unique renamings.

We define the associated *hypergraph* for the conjunctive query $Q$ as $H(Q) = (V(H), E(H))$, where the vertices set $V(H)$ consists of all attributes appearing in the query, i.e., $V(H) = \bigcup_{i \in [n]} \bar{x}_i$, and the *hyperedges* set $E(H) \subseteq 2^{V(H)} \setminus \{\emptyset\}$ contains one hyperedge for each relation, i.e., $E(H) = \{e_1, \cdots, e_n\}$, where each $e_i = \bar{x}_i$.

## 2.2 Acyclicity and Join Trees

A conjunctive query is acyclic if its associated hypergraph is acyclic, and we define acyclicity in the standard way in database theory, e.g., [1, Section 6.4]. Namely, a hypergraph $H$ is acyclic if and only if

the output of the GYO algorithm [25, 53] on $H$ is empty [1, Theorem 6.4.5]. Equivalently, $H$ is acyclic if and only if there exists a valid *join tree* [1, Theorem 6.4.5], or, a width-1 hypertree decomposition [21].

Slightly different from the usual definition of join trees [1], where each vertex is simply a relation, in this paper, we define them using notations for hypertree decompositions [21], to make it easier to generalize to the meta-decompositions that we will define later.

P5 R3.Q2 P3 R3.R4

Let $Q$ be an acyclic conjunctive query with associated hypergraph $H$. A *join tree* for $Q$ is a labelled tree $T = (V(T), r(T), E(T), \chi, \lambda)$ with vertices $V(T)$, root $r(T)$, and edges $E(T)$. $\chi : V(T) \to 2^{V(H)}$ is a function s.t. $\bigcup_{p \in V(T)} \chi(p) = V(H)$, and $\lambda : V(T) \to E(H)$[1]. And $T$ satisfies the following conditions:

($H_1$) **Coverage condition.** For each $e \in E(H)$, there exists exactly one $p \in V(T)$ with $\lambda(p) = \{e\}$.

($H_2$) **Connectedness condition.** For each $v \in V(H)$, the set of vertices $\{p \in V(T) \mid v \in \chi(p)\}$ induces a connected subtree of $T$. Or, equivalently, for all pairs of vertices $p, q \in V(T)$ and all $s$ on the (unique) path between $p$ and $q$ on $T$, we have $\chi(p) \cap \chi(q) \subseteq \chi(s)$.

($H_3$) **Width-1.** For all $p \in V(T)$, $|\lambda(p)| = 1$ and $\chi(p) = V(\lambda(p))$.

*Example 2.1.* We revisit the query discussed in Example 1.1. Its hypergraph is illustrated in Fig. 1a. It is easy to verify that the join tree in Fig. 1b satisfies conditions ($H_1$) and ($H_3$). Regarding condition ($H_2$), consider, e.g., attribute $x_1$. The vertices in the join tree that include $x_1$ are the ones with $R_1$, $R_2$, and $R_3$, which induces a connected subtree. Similar arguments apply for all attributes.

P1 R3.O2 R3.R2

We assume all trees are rooted and consider rerootings as distinct trees. We write $T_p$ as the subtree rooted at $p$, i.e., with $p$ and all its descendants. For every neighbor (child or parent) $q$ of vertex $p$ in $T$, we write $T_{p \to q}$ for the subtree with

$V(T_{p \to q}) = \{s \in V(T) \setminus \{p\} : \text{the path from } p \text{ to } s \text{ in } T \text{ contains } q\}$.

Intuitively, if $q$ is a child of $p$, it is the same as $T_q$. If $q$ is a parent of $p$, it denotes the subtree of $T$ with all vertices except those in $T_p$.

We define the *fan-out* $f(p)$ of any $p \in V(T)$ as the number of children of $p$ on $T$. The *fan-out* of the entire tree $T$, denoted by $f(T)$, is the maximum fan-out among all nodes on $T$.

Given a join tree $T$, for every $p \in V(T)$, we define the *induced query* $Q(p)$ as the join of all relations in the subtree rooted at $p$.

One important observation is that *an acyclic conjunctive query may have an exponential number of distinct join trees.*

Theorem 2.2. *Consider* star queries[2] *of the form*

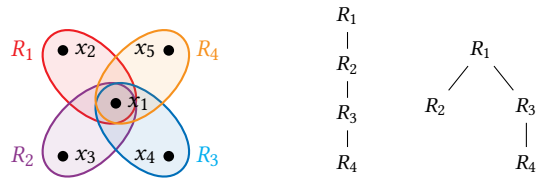
$$Q_{\text{star}} \leftarrow R_1[\bar{x}_1] \bowtie \cdots \bowtie R_n[\bar{x}_n],$$

*where for any $i, j \in [n]$, $\bar{x}_i \cap \bar{x}_j = \{x\}$, for some attribute $x$. There are $n^{n-1}$ possible join trees for such a star query with $n$ relations.*

*Example 2.3.* The query $Q = R_1[x_1, x_2] \bowtie R_2[x_1, x_3] \bowtie R_3[x_1, x_4] \bowtie R_4[x_1, x_5]$ is such a star query. Fig. 2a shows its associated hypergraph, and Fig. 2b shows two of its valid join trees.

In fact, the possible join trees of such queries in Theorem 2.2 and Example 2.3 are simply *all possible trees with $n$ vertices* (or, equivalently, all spanning trees of a complete graph $K_n$), each labeled by one relation. Any such tree satisfies the connectedness condition ($H_2$). There are a total of $n^{n-1}$ such trees ($n^{n-2}$ distinct non-rooted trees [38], each of which has $n$ possible root nodes).

P1 R3.O2 R3.R2




**(a) Hypergraph of the query** **(b) Two valid join trees**

**Figure 2: Hypergraph and two valid join trees of the star query $Q$ in Example 2.3**

## 2.3 Query Plans and Join Ordering

A *query plan* $\mathcal{P} = (V(\mathcal{P}), E(\mathcal{P}))$ for a query $Q$ is a rooted tree, where each non-leaf node is a relational operator (e.g, join, projection, selection), and each leaf node corresponds to a base relation. A plan specifies an order of operations to evaluate the query, where the outputs of child nodes serve as inputs to the parent. In this work, we focus on query optimization of binary joins, where, for every join operation on the query plan, there are exactly two child nodes. For the purpose of join ordering, *we use only join operators in internal nodes and assume selections and projections are inherently integrated in the join or scan operation at the lowest possible level.* Namely, in each step of join or scan, we apply a filter condition if all relevant attributes already appear in the subtree, and we project out attributes that neither appear in the output nor act as a join key with some remaining relation.

P3 R3.O4

Given a query plan $\mathcal{P}$, for every $p \in V(\mathcal{P})$, we define the *induced query* $Q(p)$ as the join of all relations in the subtree rooted at $p$ (with all possible projection and filters inherently applied).

P1 R3.O2 R3.R2

We exclude Cartesian products: For every join node, the output of its children must have some shared attribute as the join key.

A query plan is called *left-deep* (or *right-deep*) if all join operations have their right (left) child node corresponding to a base relation; otherwise, the query plan is referred to as *bushy*.

We apply cost functions $C$ on query plans, in the following form, to measure the optimality of query plans.

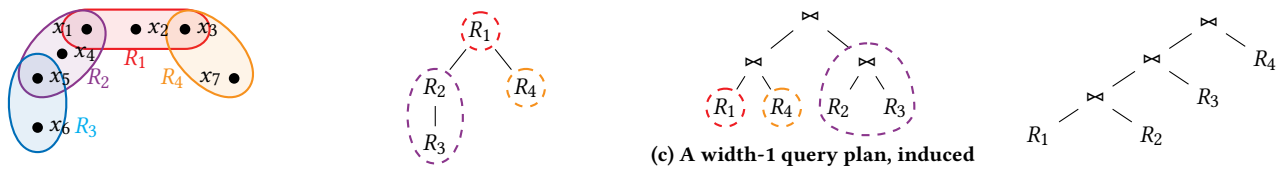
$$C(\mathcal{P}) = \begin{cases} c(r), & \text{if } r \text{ is single relation} \\ c(r) \oplus C(\mathcal{P}_1) \oplus C(\mathcal{P}_2), & \text{if } r \text{ is join operation} \end{cases}$$

where $c(r)$ is the cardinality of the output of $r$, $\mathcal{P}_1$ and $\mathcal{P}_2$ (if applicable) are the left and right subtrees of $r$, respectively, and $\oplus$ is some operator to aggregate $c(r)$, $C(\mathcal{P}_1)$, and $C(\mathcal{P}_2)$. In this work, we adopt the cost function $C_{\text{out}}$ to minimize the total intermediate result size by substituting the $\oplus$ operator simply with arithmetic $+$, as is standard in, e.g., [13, 17, 37, 43].

The typical approach to find the optimal plan is to recursively find the optimal split of a set of relations into two disjoint subsets [36, 42, 43, 45, 46]. With a given cost function $C$, the problem can be solved by the dynamic programming (DP) algorithm DPccp [36] in $O(3^n)$. The recently proposed algorithm DPconv [43] reduces the complexity for the cost model $C_{\text{out}}$ to $O(2^n n^3 W \log(Wn))$, where $W$ is the largest join cardinality.

[1] Due to space limitations, we may simply use hyperedges (e.g., $R_1$) to label the vertices on join trees instead of writing out the full $\lambda$ and $\chi$ labels when the context is clear.

[2] We note that, in some literature, e.g., [9], such queries are called "*clique*" queries, because their *query graphs* are cliques.




(a) Hypergraph of the query (b) A valid join tree (c) A width-1 query plan, induced by the join tree shown in Fig. 3b (d) A width-2 query plan

**Figure 3: The hypergraph, a join tree, a width-1 query plan, and a width-2 query plan of the query in Example 3.1**

## 3 Widths of Query Plans & Width-1 Plans

P1 R3.R1

In this section, we begin to introduce our notion of width for query plans, which connects a plan's structure to the potential size of its intermediate results. We consider the "interface" of each intermediate step in the query plan, i.e., the set of attributes that serve as join keys with the remaining relations. Intuitively, the width of a query plan shows the number of relations needed to cover the interface of each step of the query plan. This measure helps quantify the sizes of intermediate results and the complexity of a query plan before looking at the specific database instance.

*Example 3.1.* Consider the query

$Q = \pi_{\emptyset}(R_1[x_1, x_2, x_3] \bowtie R_2[x_1, x_4, x_5] \bowtie R_3[x_5, x_6] \bowtie R_4[x_3, x_7])$.

Its hypergraph is shown in Fig. 3a and a join tree in Fig. 3b. Given this join tree, Fig. 3c is an example of a valid width-1 query plan. After joining $R_1$ and $R_4$, $x_3$ is the only attribute in the intersection with the remaining relations $R_2$ and $R_3$, and it can be covered by only one relation $R_1$. Similarly, at the step that joins $R_2$ and $R_3$, $x_1$ is the only attribute in the interface. A width of 1 implies that, if the size of any one single relation is bounded by $N$, then the size of each intermediate step can be bounded by $N$, as it is at most the size of the relation that covers the interface.

On the contrary, Fig. 3d shows a query plan that is width-2: When we join $R_1$ and $R_2$, the interface contains $x_5$ and $x_3$, which are needed to join $R_3$ and $R_4$, respectively. But there is no single relation that can cover both $x_5$ and $x_3$, so the intermediate sizes can be as high as $N^2$, instead of $N$ as in the width-1 plan in Fig. 3c.

More formally, given a query plan $\mathcal{P}(V(\mathcal{P}), E(\mathcal{P}))$, for any node $p \in V(\mathcal{P})$, let $H_p = (V(H_p), E(H_p))$ be the hypergraph induced by the relations being joined in the subtree rooted at $p$, and let the induced query $Q(p) = \bowtie_{e \in E(H_p)} e$. Let $\overline{H}_p = (V(\overline{H}_p), E(\overline{H}_p))$ be the *residual hypergraph* with relations that are not yet joined in the subtree rooted at $p$, i.e., $E(\overline{H}_p) = E(H) \setminus E(H_p)$, and $V(\overline{H}_p) = \cup(E(\overline{H}_p))$. We define the *interface* $I(p) = V(H_p) \cap V(\overline{H}_p)$, i.e., the set of attributes that will serve as join keys with the remaining relations. The *width of node p* is the smallest number of relations needed to cover this interface, i.e.,

P1 R3.O2 R3.R2 R3.R3

$$w(p) = \min\left\{|S| : S \subseteq E(H_p) \text{ and } I(p) \subseteq \cup S\right\}.$$

The width of the query plan $P$ is the maximum width over all nodes.

The width of a query plan indicates an upper bound on the minimal intermediate results over all nodes $p$ during evaluation.

Theorem 3.2. *Given a Boolean conjunctive query $Q$ and a query plan $\mathcal{P}$ for the query, the maximum intermediate result size, i.e., $\max_{p \in V(H)} |\pi_{I(p)} Q(p)|$, is upper bounded by $O(N^{w(\mathcal{P})})$ in the worst case, where $N$ is the maximum cardinality of any base relation.*

This notation of width has a direct connection with join trees:

*Example 3.3.* We revisit the query in Example 3.1. Take $p$ as the node on the join tree with $R_2$, as an example. The induced query $Q(p) = R_2 \bowtie R_3$. In the width-1 query plan in Fig. 3c, the right child of the root node, which we denote by $q$, induces exactly $Q(p) = R_2 \bowtie R_3$. On the other direction, for the node $q$ on the query plan, we consider its children $u$ and $v$ (which correspond to the leaf nodes with $R_2$ and $R_3$ respectively). $u$ has interface $I(u) = \{x_1, x_5\}$, which can be covered by a single relation $R_2$. And $v$ has interface $I(v) = \{x_5\}$, which can be covered by $R_3$.

However, for the width-2 query plan in Fig. 3d, there is no such $q$ on the query plan such that $Q(p) = Q(q) = R_2 \bowtie R_3$.

P1 R3.R1

We formally define this correspondence as follows:

*Definition 3.4.* We say a query plan $\mathcal{P} = (V(\mathcal{P}), E(\mathcal{P}))$ is *induced by* a join tree $T = (V(T), E(T))$ if

(1) for each node $p \in V(T)$, there exists $q \in V(\mathcal{P})$ with $p = Q(q)$, or $Q(p) = Q(q)$, i.e., the induced query of node $p$ in the join tree is exactly the induced query of node $q$ in the query plan, unless $Q(q)$ involves only a single relation; and
(2) for each non-leaf node $q \in V(\mathcal{P})$ with two children $u$ and $v$, there exists some $s \in E(H_u)$ and $t \in E(H_v)$ such that $I(u) \subseteq s$, $I(v) \subseteq t$, and there exists an edge between $s$ and $t$ in $E(T)$.

Under this definition, we have that

Theorem 3.5. *Given a query plan $\mathcal{P}$ for a query $Q$, the width $w(\mathcal{P}) = 1$ if and only if there exists a join tree $T$ of $Q$ that induces $\mathcal{P}$.*

Theorem 3.5 directly lead to the relationship between width-1 query plans and acyclic queries:

Theorem 3.6. *A conjunctive query $Q$ has width-1 query plans if and only if $Q$ is acyclic.*

Width-1 query plans are closely related to structure-based query evaluation methods. Such methods follow a similar pattern: given a join tree, they apply a series of reductions from the leaves upwards until only a single node remains. Width-1 query plans exploit the join tree structure in a similar way, evaluating the query bottom-up: each node $p \in V(T)$ completes all its internal joins with its child subtrees before joining with its parent. Such a strategy provides several benefits.

P2 R1.O1–2

*Limiting intermediate result sizes.* We have seen that, by projecting out attributes that are neither part of the output nor in the interface at each step, we limit the sizes of intermediate results, as shown by Theorem 3.2. Although this approach excludes some alternative plans with higher widths, these excluded ones do not preserve such a theoretical guarantee. Taking again Example 3.1, if we choose the width-2 query plan in Fig. 3d, $x_3$ has to be kept in all intermediate results before $R_4$ is joined at the very end, whereas,

in the width-1 query plan in Fig. 3c, it can already be projected out after completing $R_1 \bowtie R_4$.

*Local optimization.* Instead of searching in a vast space of arbitrary join orders for all relations at once, we can locally optimize the join order at each node of the tree. For each node $p \in V(T)$ with children $c_1, \cdots, c_{f(p)}$ in the join tree, we only need to determine the optimal join order between $p$ and $Q(c_1), \ldots, Q(c_{f(p)})$. The order of doing these joins can be optimized independently for each node, without considering other parts of the query. Such an approach, therefore, significantly prunes the search space for optimization, making plan selection much more tractable, and can yield near-optimal performance for practical queries. In the experiments (Section 6), we show in Table 1 that the fan-out $f(T)$ tends to be very low in real benchmarks.

**P1 R3.R1**

*Example 3.7.* Consider the join tree again in Fig. 3b for the query in Example 3.1. Given this join tree, a width-1 query plan requires that a subtree with $R_2 \bowtie R_3$ exist in the query plan. It therefore remains only to optimize the order to join $R_1$, $(R_2 \bowtie R_3)$, and $R_4$, which is the local optimization problem at the root node $R_1$ of the join tree. Suppose that the cardinalities $|R_1 \bowtie R_4| < |(R_2 \bowtie R_3) \bowtie R_1|$. Then, the optimal plan in our cost model is the one in Fig. 3c.

With only local search needed, assuming a low fan-out, finding the optimal width-1 query plans induced by a join tree can be done more efficiently than a global search:

Theorem 3.8. *Given a join tree $T$, the optimal width-1 query plan induced by $T$ can be found in time $O(f(T)2^{f(T)}|Q|)$.*

*Structure-guided query evaluation.* An additional advantage of width-1 query plans is that they naturally accommodate those newly developed structure-guided query evaluation methods. For example, we can simulate the Yannakakis algorithm for evaluating *relation-dominated* conjunctive queries [47] by following a width-1 query plan. A conjunctive query is relation-dominated if it is acyclic and there exists a relation $R_i(\bar{x}_i)$ that contains all the output attributes, i.e., $\bar{y} \subseteq \bar{x}_i$. It shall be noted that an acyclic Boolean query is also a relation-dominated query, since $\bar{y} = \emptyset$. For this type of query, we have the following guarantee of evaluation time:

Theorem 3.9. *A relation-dominated conjunctive query can be evaluated in $O(|db|)$ time using a width-1 query plan, where $|db|$ represents the size of the database.*

To create a linear-time width-1 query plan for evaluating a relation-dominated query, let $R_i$ denote the relation containing all output attributes. One can construct the query plan using any join tree with $R_i$ as the root. The width-1 query plan will then evaluate the query in a bottom-up fashion. Because all output attributes are located at the root of the join tree and at the end of the query plan, when the query plan is evaluating node $p$, all attributes in $T_p$ but not in $p$ are removed from the query, guaranteeing a linear running time.

**P9a R2.O1**

To conclude this section, we highlight some important distinctions between our definition of width-1 query plans and Yannakakis-style query plans [47, 52]. Although they both evaluate queries by traversing join trees, width-1 query plans do not incorporate full reductions using semi-joins, as is done in the Yannakakis-style algorithms. We have shown, nevertheless, that these plans, by appropriate projections of irrelevant attributes, still preserve favourable theoretical guarantees of the asymptotic complexity for Boolean conjunctive queries (Theorem 3.2) and relation-dominated queries (Theorem 3.9). In Section 6, we will experimentally show that these plans, in practice, are indeed highly efficient. Furthermore, without the need for full reductions, width-1 query plans are much easier to integrate into the query execution frameworks of current database systems, enabling more efficient cost-based optimization, as we will discuss in the remainder of the paper.

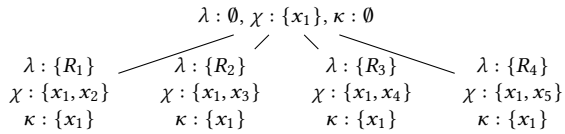



**Figure 4: Meta-decomposition of the query in Example 2.3**

## 4 Meta-Decompositions

Although we have shown a connection between join trees and width-1 query plans in Theorem 3.2 and Theorem 3.5, it is impractical to find the optimal width-1 query plan by enumerating every possible join tree—As noted earlier, a query can have an exponential number of possible join trees. Therefore, in this section, we introduce and explore a generalized representation that efficiently captures all join trees of an acyclic query. This enables us to optimize queries directly on this representation, eliminating the need to enumerate all possible join trees.

**P1 R3.O2 R3.R1 R3.R2 R3.R3**

We highlight an observation that *the exponential number of possible join trees often arises when multiple relations share common join attributes*, as is the case for star queries discussed in Theorem 2.2 and illustrated by Example 2.3. In such a query, any pair of relations has valid join predicates, so in join trees of such queries, these relations can be connected in any arbitrary tree structure. On the contrary, in the case of a query like in Example 3.1, where the join attributes are unique for all possible pairs of relations to be joined, the query demonstrates a more "tree-like" structure. There is a unique tree structure for such a query, with $n$ distinct, structurally isomorphic join trees, differing only by the choice of the root relation.

In light of this, we introduce *meta-decompositions*. In the case of acyclic conjunctive queries, it can capture all possible join trees but requires only linear space. One of the major differences from standard join trees is the addition of "minor nodes" to handle cases when more than two relations share the same join key. These nodes have empty $\lambda$-labels, meaning they are not associated with a single relation, but their $\chi$-labels contain the shared join keys. Additionally, we add a $\kappa$-label to each node to denote the "interface" with the remaining relations on the join tree that are not in the subtree rooted at the node.

*Example 4.1.* The meta-decomposition of the query in Example 2.3 is shown in Fig. 4. The root of this tree is a minor node, showing that the four children share precisely the attribute $x_1$. Note also that the $\kappa$-label of all 4 non-root nodes is exactly $\{x_1\}$.

*Example 4.2.* The meta-decomposition of the query in Example 3.1 is shown in Fig. 5. It has the exact same structure as the

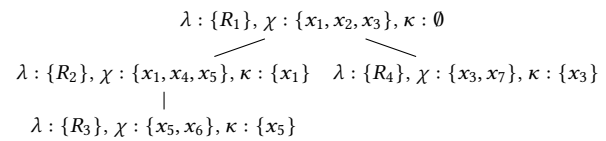



**Figure 5: Meta-decomposition of the query in Example 3.1**

join tree shown in Fig. 3b and has no minor node, as all pairs of relations have different intersections of attributes.

The formal definition is given as follows.

*Definition 4.3.* Given a conjunctive query $Q$ and the associated hyergraph $H(Q)$, its *meta-decomposition* $M = (V(M), r(M), E(M), \lambda, \chi, \kappa)$ is a labelled tree, where

- $V(M)$ is the set of vertices, $r(M) \in V(M)$ is the root, and $E(M)$ is the set of edges,
- $\lambda : V(M) \to 2^{E(H)}$ maps each tree vertex in $M$ to a set of hyperedges in $H$ (possibly empty), and
- $\chi : V(M) \to 2^{V(H)}$ and $\kappa : V(M) \to 2^{V(H)}$ each maps each tree vertex in $M$ to a set of vertices in $H$,

such that it satisfies ($H_1$), ($H_2$), and

($H_3'$) For all $p \in V(M)$, $\chi(p) \subseteq V(\lambda(p))$ if $\lambda(p) \neq \emptyset$.

($H_4$) **Interface condition.** For every non-root node $p \in V(M)$, let $q$ be $p$'s parent node. $\kappa(p)$ satisfies
  (a) $\kappa(p) = \chi(p) \cap \chi(V(M) \setminus V(M_p)) \subseteq \chi(q)$, where $M_p$ is the subtree of $M$ rooted at $p$; and
  (b) $\kappa(p) \nsubseteq \chi(s)$ for all $s \in V(M) \setminus V(M_q)$ if $\kappa(p) \neq \chi(q)$.
  For root node $p = r(M)$, we set $\kappa(p) = \emptyset$.

($H_5$) **Minor nodes and uniqueness condition.** For every pair of $p, q \in V(M)$ such that $\kappa(p) = \kappa(q)$, there exists one unique node $o \in V(M)$ with $\lambda(o) = \emptyset$ and $\chi(o) = \kappa(p) = \kappa(q)$.

For acyclic queries, the meta-decompositions additionally satisfy

($H_3''$) For all $p \in V(M)$ such that $\lambda(p) \neq \emptyset$, we have $|\lambda(p)| = 1$ and $\chi(p) = V(\lambda(p))$.

Besides introducing minor nodes in condition ($H_5$) and defining $\kappa$ as the interface in condition ($H_4$)(a), we also include condition ($H_4$)(b) that limits the shape of the meta-decomposition $M$: there can be multiple nodes that can satisfy $\kappa(p) \subseteq \chi(q)$, while condition ($H_4$)(b) limits $q$ to be the highest node with $\kappa(p) \subseteq \chi(q)$. We illustrate with the following example:

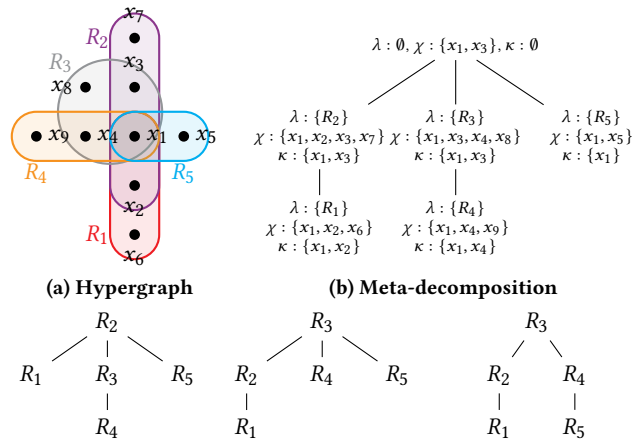
*Example 4.4.* Consider the query

$$Q = R_1[x_1, x_2, x_6] \bowtie R_2[x_1, x_2, x_3, x_7] \bowtie R_3[x_1, x_3, x_4, x_8] \bowtie R_4[x_1, x_4, x_9] \bowtie R_5[x_1, x_5]).$$

Its hypergraph and meta-decomposition are shown in Fig. 6a and Fig. 6b, respectively. Fig. 6c and Fig. 6d show two valid join trees of this hypergraph. We highlight that $R_5$ could in fact be the child of any other node on the join tree, because its only interface with the rest of the hypergraph is $\{x_1\}$. For example, Fig. 6e is also a valid join tree. But according to condition Item ($H_4$)(b), we make it the child of the highest possible node that contains $x_1$, which in this case is the root node, i.e., the minor node with $\chi$-label $\{x_1, x_3\}$.

## 4.1 Constructing Meta-Decompositions

Similar to the original GYO algorithm [25, 53], a meta-decomposition can be constructed by a reduction-based algorithm,




**(a) Hypergraph** **(b) Meta-decomposition**

**(c) A join tree** **(d) A second join tree** **(e) A third join tree**

**Figure 6: Hypergraph, meta-decomposition, and three join trees of the query in Example 4.4**

as shown in Algorithm 1. In this algorithm, $o(e, H) = e \cap (\cup(E(H) \setminus \{e\}))$ denotes the interface, as a set of vertices, of hyperedge $e$ with the rest of the remaining hyperedges $E(H) \setminus \{e\}$. As in the original GYO algorithm [25, 53], a hyperedge $e$ is *reducible*, or, is an *ear*, if there exists another hyperedge $e'$, such that $o(e, H) \subseteq e'$. The subsuming hyperedge $e'$ is called the *witness*. We note that the GYO algorithm removes exactly one ear (among possibly many) in each round, thereby establishing a total order of all hyperedge removal. In contrast, Algorithm 1 *removes all ears in every round* until only a single hyperedge remains in the hypergraph. This process introduces only a partial order for all hyperedges, which *maintains the flexibility to represent all possible ordering of ear removal, and hence all possible join trees*, through the resulting meta-decomposition.

*Example 4.5.* For the star query in Example 2.3, the four hyperedges could be removed in any order in the standard GYO algorithm. For example:

(1) With the order $R_4$ (witness $R_3$) – $R_3$ (witness $R_2$) – $R_2$ (witness $R_1$) – $R_1$, we obtain the first join tree in Fig. 2b.
(2) With the order $R_4$ (witness $R_3$) – $R_2$ (witness $R_1$) – $R_3$ (witness $R_1$) – $R_1$, we obtain the second join tree in Fig. 2b.

Furthermore, we note that we could also obtain the second join tree by keeping the order in (1) but only changing the witness, namely $R_4$ (witness $R_3$) – $R_3$ (witness $R_1$) – $R_2$ (witness $R_1$) – $R_1$.

To address this source of variation, in every round, our algorithm first handles such cases where some edges $e_1, e_2$ are *mutually reducible*, i.e., $o(e_1, H) = o(e_2, H)$ (Lines 4-12). (For example, in the case of the star query in Example 2.3, all pairs among $R_1$, $R_2$, $R_3$, and $R_4$ are mutually reducible.) The algorithm creates a minor node $p$ with $\chi(p) = o(e_1, H)$ if such a node does not yet exist, and then remove $e_1$ and $e_2$ from the hypergraph. We then add a hyperedge $e_m = o(e_1, H)$, which conceptually corresponds to the minor node, back to the hypergraph, so that the acyclicity of $H$ is not broken. The rationale for adding such an $e_m$ can be illustrated by the following example:

*Example 4.6.* Consider the query

$$Q = \pi_{\emptyset}(R_1[x_1, x_2, x_3, x_4] \bowtie R_2[x_1, x_2, x_5] \bowtie R_3[x_1, x_3, x_6] \bowtie R_4[x_2, x_3, x_6] \bowtie R_5[x_1, x_2, x_3, x_7]),$$

which is similar to Example 1.1 but contains an additional relation $R_5$. In this case, at the beginning, $R_1$ and $R_5$ are both ears and mutually reducible, as $o(R_1, H) = o(R_5, H) = \{x_1, x_2, x_3\}$. However, if we simply remove both $R_1$ and $R_5$ at the same time, *the remaining hyperedges, $R_2$, $R_3$, and $R_4$, form a cyclic hypergraph.* But, if we add $e_m = o(R_1, H) = o(R_5, H) = \{x_1, x_2, x_3\}$, then $e_v$, $R_2$, $R_3$, and $R_4$ still form an acyclic query, and the algorithm can proceed to pick $R_2$, $R_3$, and $R_4$ as ears, before $e_m$ is picked and removed at the very end.

We formalize this idea as follows:

Lemma 4.7. *Given an acyclic hypergraph $H$, if there exist two distinct ears $e_1, e_2 \in E(H)$ such that $o(e_1, H) = o(e_2, H)$, then, the hypergraph $H'$, with $E(H') = E(H) \setminus \{e_1, e_2\} \cup \{o(e_1, H)\}$ and $V(H') = \cup E(H')$, is acyclic.*

In each round, the algorithm then finds and removes the set $\mathcal{E}$ of all ears of $H'$. For each $e \in \mathcal{E}$, we create a physical node $p$ and set $\kappa(p) = o(e, H')$. During the procedure, the algorithm creates minor nodes if necessary (Lines 9, 14, 17–18, and 20).

This algorithm has the following guarantee of running time:

Theorem 4.8. *Algorithm 1 terminates in time $O(|E(H)|^3)$.*

Proof sketch. The key observations are (1) that the size $|E(H')|$ decreases by at least one in each iteration of the while-loop (Line 3), and (2) that, in each iteration of the while loop, all sets $S$ (Line 4) can be enumerated in $O(|E(H')|) = O(|E(H)|)$ time, if we maintain a hash map that maps each $o \subseteq V(H')$ to all hyperedges $e \in E(H')$ such that $o(e, H') = o$, which is updated whenever a hyperedge $e$ is added or removed from $E(H')$. □

We show the correctness of the algorithm:

Theorem 4.9. *Given an acyclic query with an acyclic hypergraph $H$, Algorithm 1 returns a valid meta-decomposition of $H$.*

Finally, we note that meta-decompositions are space-efficient. They only require space linear to the size of the hypergraph.

Theorem 4.10. *In the meta-decomposition $M$ of an acyclic hypergraph $H$ as constructed by Algorithm 1, the number of vertices $|V(M)|$ and the number of edges $|E(M)|$ are $O(|E(H)|)$.*

Proof sketch. Vertices in $M$ are either physical or minor. We create one physical node per hyperedge in $E(H)$. For minor nodes, we note that each minor node has fan-out at least 2. □

## 4.2 Join Tree Enumeration

The meta-decomposition allows one to efficiently enumerate all possible join trees of a query, as we will show by an algorithm in this section. This not only serves as proof of the universality of this representation but also provides a powerful approach to enumerating join trees, which could be of great help for other structure-based methods that require doing so.

P9a R2.O1

As an intuition, we can observe that the distinct join trees of the same hypergraph can be different in the following ways:

**Algorithm 1:** Constructing the meta-decomposition of an acyclic hypergraph

**input** : An acyclic hypergraph $H$
**output** : The meta-decomposition $M$ of $H$

1 $H' \leftarrow$ a hypergraph with $V(H') = V(H)$ and $E(H') = E(H)$
2 $V(M) \leftarrow \emptyset$
3 **while** $|E(H')| > 1$ **do**
4  **foreach** *maximal* $S \subseteq E(H')$ *such that* $|S| > 1$, *and, for any* $e_1, e_2 \in S$, $o(e_1, H') = o(e_2, H')$ **do**
5  $o \leftarrow$ the value of $o(e_i, H')$ that all $e_i \in S$ share
6  **if** *there exists* $v \in V(M)$ *such that* $\lambda(v) = \emptyset$ *and* $\chi(v) = o$ **then** // There is already a minor node. We remove it and create a new one later
7  Remove $v$ from $V(M)$
8  **foreach** $e \in S$ **do**
9  Add a node $p$ to $V(M)$ with $\lambda(p) = \{e\}$ if $e \in E(H)$, or $\lambda(p) = \emptyset$ otherwise, $\chi(p) = e$, and $\kappa(p) = o(e, H')$
10  Remove $e$ from $E(H')$
11  Add a special hyperedge $e_m = o$ to $E(H')$
12  $\mathcal{E} \leftarrow$ the set of all ears $e \in E(H')$, i.e., for every $e \in \mathcal{E}$, there exists a distinct $e' \in E(H')$ with $o(e, H') \subseteq e'$
13  **foreach** $e \in \mathcal{E}$ **do**
14  Add a new node $p$ to $V(M)$ with $\lambda(p) = \{e\}$ if $e \in E(H)$, or $\lambda(p) = \emptyset$ otherwise, $\chi(p) = e$, and $\kappa(p) = o(e, H')$
15  Remove $e$ from $E(H')$
16  $S \leftarrow \{p' \in V(M) \cup \mathcal{E} \mid \kappa(p') = \kappa(p)\}$
17  **if** $|S| > 1$ *and, for all* $p' \in S$, $\lambda(p') \neq \emptyset$ **then**
18  Add a new node $v$ to $V(M)$ with $\lambda(v) = \emptyset$, $\chi(v) = \kappa(p)$, and $\kappa(v) = \kappa(p)$
19 $e \leftarrow$ the remaining edge in $E(H')$
20 Add a new node $r$ to $V(M)$ with with $\lambda(r) = \{e\}$ if $e \in E(H)$, or $\lambda(r) = \emptyset$ otherwise, $\chi(r) = e$, and $\kappa(r) = \emptyset$
21 Remove $e$ from $E(H')$
22 $E(M) \leftarrow \emptyset$
23 **foreach** $p \in V(M)$ *such that* $\kappa(p) \neq \emptyset$ **do**
24  **if** *there exists* $q \in V(M)$ *such that* $\lambda(q) = \emptyset$ *and* $\chi(q) = \kappa(p)$ **then**
25  Add an edge $(q, p)$ in $E(M)$
26  **else**
27  Find $q \in V(M)$ such that $\kappa(p) \subseteq \chi(q)$ and $\kappa(p) \not\subseteq \kappa(q)$
28  Add an edge $(q, p)$ in $E(M)$
29 **return** $M = (V(M), r, E(M), \lambda, \chi, \kappa)$

(1) **"Re-branching".** If a vertex $p$ in a join tree has children $c_1$ and $c_2$ such that $\kappa(c_1) = \chi(c_1) \cap \chi(p) \subseteq \chi(c_2)$, then $c_1$ could have been a child of $c_2$ without violating any condition. In the meta-decomposition, condition ($H_4$) ensures that $c_1$ is positioned at the highest possible node. Therefore, when enumerating a join tree, we can make $c_1$ a child of $c_2$ if $\kappa(c_1) \subseteq \chi(c_2)$. The join tree Fig. 6e based on meta-decomposition Fig. 6b from Example 4.4 demonstrates such a case.

(2) **"Rerooting".** A rerooting of a join tree is still a valid join tree.
(3) **Minor nodes.** If multiple relations share the same set of attributes pairwise, they can be joined in arbitrary order. This situation is represented by the minor nodes in the meta-decomposition. An example is the root node with $\chi$-label $\{x_1\}$ in Fig. 4 for Example 2.3, based upon which we should be able to enumerate, among others, the join trees in Fig. 2b.

These alternatives should be considered to enumerate all join trees, and the meta-decomposition retains all the necessary information to facilitate this. Algorithm 2 is an algorithm to enumerate all join trees. For each node in the meta-decomposition starting from the root, it recursively enumerates all possible join trees given by the subtree rooted at each of its children, and then enumerates all possible ways to combine these join trees into one. For minor nodes $r$, we find its neighbours that share the common set of attributes given by $\chi(r)$ and generate all possible distinct structures of join trees without repetition, where each vertex will be replaced by the join tree for a subtree given by a neighbour, and at this point we consider all rerootings of some tree as identical. At the very end of this algorithm, we return all the combined join trees we obtain, and it remains only to collect all the rerootings. Due to the space constraints, the algorithms for enumerating all trees and rerootings are given in Appendix D.1.

*Example 4.11.* Consider again the query in Example 4.4. Its meta-decomposition is shown in Fig. 6b. Let the root node be $r$, and we use $v_1, \dots v_5$ to denote the nodes such that $\lambda(v_i) = \{R_i\}$. $r$ is a minor node, with a $\chi(r) = \{x_1, x_3\}$. We start by enumerating all possible join trees induced by the subtrees of the meta-decomposition, rooted at each child of $r$. For the subtree with $v_2$ and $v_1$, there is only one valid join tree, rooted at $v_2$, which has one child $v_1$. It is not possible to reroot to $v_1$ because $\kappa(v_2) = \{x_1, x_3\} \not\subseteq \chi(v_1)$. The case is similar for the subtree with $v_3$ and $v_4$. And the subtree with $v_5$ induces only one join tree with one node $v_5$. We then enumerate all possible trees with vertices $v_2$ and $v_3$, having $\kappa(v_2) = \kappa(v_3) = \chi(r)$. There are only two such trees, each of which contains one edge connecting $v_2$ and $v_3$, and they are rerootings of each other. For each tree, we replace $v_2$ and $v_3$ by their induced join tree. In the end, we have one child $v_5$ with $\kappa(v_5) \neq \chi(r)$. It can be attached as a child of any node $v$ such that $\kappa(v_5) \subseteq \chi(v)$. In this example, it can, in fact, be a child of any other node. Fig. 6c is a join tree where the spanning tree is rooted at $R_2$, and $R_5$ is attached as a child of $R_2$. Fig. 6d is a similar example where the spanning tree is rooted at $R_3$, and $R_5$ is attached as a child of $R_3$. And Fig. 6e is based on the same spanning tree as for Fig. 6d, but $R_5$ is attached as a child of $R_4$ instead of $R_3$.

We now show that this algorithm is both correct (producing valid join trees) and complete (enumerating all possible join trees).

Theorem 4.12 (Correctness). *Given a meta-decomposition $M$ of an acyclic hypergraph $H$, all trees $T$ output by Algorithm 2 are valid join trees of $H$.*

Theorem 4.13 (Completeness). *Given a meta-decomposition $M$ of an acyclic hypergraph $H$, Algorithm 2 enumerates all possible join trees of $H$.*

As is detailed in the proof of Theorem 4.13, for each join tree, we can pinpoint exactly one way in which is enumerated by the al-

**Algorithm 2:** Enumerating all join trees based on a meta-decomposition

**input** : A meta-decomposition $M = (V(M), r, E(M), \lambda, \chi, \kappa)$
**output** : All possible non-homomorphic join trees up to rerooting

1 **Function** enumRec($v$: *a node on* $M$):
2 $C \leftarrow$ set of children $c$ of $v$ on $M$, sorted by partial order $\supseteq$ on $\kappa(c)$
3 **if** $\lambda(v) = \emptyset$ **then** // minor node
4 $S \leftarrow \{c \in C \mid \kappa(c) = \chi(v)\}$ // they can be found at the head of the sorted $C$
5 **foreach** $c \in S$ **do**
6 $\mathcal{T}_c \leftarrow$ enumRec($c$) // All non-homomorphic join trees for subtree $M_c$ rooted at $c$, up to rerooting
7 **if** $\kappa(v) = \chi(v)$ **then**
8 $\mathcal{T}_P \leftarrow$ enumerateTrees($S \cup \{v\}$), all rerooted to $v$ // All possible trees with vertices $S \cup \{v\}$, by Prüfer sequences // The minor node $v$ will eventually be replaced on Line 33
9 **else** // $\kappa(v) \neq \chi(v)$, the parent should not participate
10 $\mathcal{T}_P \leftarrow$ enumerateTrees($S$)
11 **foreach** $T_P \in \mathcal{T}_P$ **do** // A skeleton of a way to connect subtrees, given by a Prüfer sequence
12 **foreach** $c \in V(T_P)$ **do**
13 **if** *$c$ is a leaf node* **then**
14 $\mathcal{T}'_c \leftarrow \mathcal{T}_c$
15 **else**
16 $\mathcal{T}'_c \leftarrow \emptyset$
17 **foreach** *non-leaf $c \in V(T_P)$, in bottom-up order, $T_c \in \mathcal{T}_c$, child $d$ of $c$ on $T_P$, $T_d \in \bigcup_{T_d \in \mathcal{T}_d}$* rerootings$(T_d, \emptyset, \kappa(d))$, *$u \in V(T_c)$ such that $\kappa(d) \subseteq \chi(u)$* **do**
18 $T'_c \leftarrow T_c$, with $T_c$ added as a child of $u$
19 Add $T'_c$ to $\mathcal{T}'_c$
20 $\mathcal{T} \leftarrow \mathcal{T} \cup \mathcal{T}'_{\text{root}(T_P)}$
21 $X \leftarrow C \setminus S$, preserving the order
22 **else**
23 $\mathcal{T} \leftarrow$ a set of one tree with one vertex $v$ only
24 $X \leftarrow C$, preserving the order
25 **foreach** *child $c \in X$ in order* **do**
26 $\mathcal{T}_c \leftarrow \bigcup_{T \in \text{enumerate}(c)}$ rerootings$(T, \emptyset, \kappa(c))$
27 $\mathcal{T}' \leftarrow \emptyset$
28 **foreach** $T_c \in \mathcal{T}_c, T \in \mathcal{T}$ **do**
// Enumerate all possible $v$ in some $T$ which $T_c$ can be attached to
29 **if** *$\lambda(c) = \emptyset$ and $\kappa(c) = \chi(c)$* **then**
// The root of $T_c$ would be a minor node from Line 8. We find a parent of each of its children.
30 **foreach** *child $d$ of the root of $T_c$* **do**
31 $\mathcal{T}'' \leftarrow \emptyset$
32 **foreach** *$T' \in \mathcal{T}', u \in T'$ such that $\kappa(d) \subseteq \chi(u)$* **do**
33 $T'' \leftarrow T'$ with $T_d$ attached as a child of $u$
34 Add $T''$ to $\mathcal{T}''$
35 $\mathcal{T}' \leftarrow \mathcal{T}''$
36 **else**
37 **foreach** *$u \in T$ such that $\kappa(c) \subseteq \chi(u)$* **do**
38 $T' \leftarrow T$ with $T_c$ added as a child of $u$
39 Add $T'$ to $\mathcal{T}'$
40 $\mathcal{T} \leftarrow \mathcal{T}'$
41 **return** $\mathcal{T}$
42 **return** $\bigcup_{T \in \text{enumRec}(r)}$ rerootings$(T, \emptyset, \emptyset)$

**Algorithm 3:** Find the optimal width-1 query plan to evaluate the query induced by a meta-decomposition

**input** : A meta-decomposition $M = (V, r, E)$
**output** : The optimal query plan

```
// Bottom-up traversal
foreach non-root node q ∈ V(M), in bottom-up order do
    p ← q's parent node on M
    plan(T_{p→q}) ← optimizeLocal(q, {plan(T_{q→c}) | c ∈ C(q)})   // C(q) is the set of all children of node q
plan(M) ← optimizeLocal(r, {plan(T_{r→c}) | c ∈ C(r)})
// Top-down traversal
foreach c ∈ C(r) do
    plan(T_{c→r}) ← optimizeLocal(r, {plan(T_{r→c'}) | c' ∈ C(r) \ {c}})
foreach non-root node q ∈ V(M), in top-down order do
    p ← q's parent node on M
    foreach child c ∈ C(q) do
        plan(T_{c→q}) ← optimizeLocal(q, {plan(T_{q→p})} ∪ {plan(T_{q→c'}) | c' ∈ C(r) \ {c}})
return the plan plan(T_{p→q}) ⋈ plan(T_{q→p}) with the minimum cost among all (p, q) ∈ E(M)
```

gorithm based on the meta-decomposition. Noting that the number of children of each node on a meta-decomposition is bounded by the fan-out, we get the following guarantee of enumeration delay.

Theorem 4.14 (Running time). *Given a meta-decomposition $M$ of an acyclic hypergraph $H$, Algorithm 2 enumerates all possible join trees of $H$ in an amortized delay of $O(f(M) \cdot |V(H)|)$.*

# 5 Cost-Based Optimization from Meta-Decompositions

Although we have presented an algorithm, enumerating all possible join trees is infeasible, as their number grows super-exponentially with the number of relations, as shown by Theorem 2.2. Instead, we propose a cost-based optimization algorithm that *operates directly on the meta-decomposition* to find an optimal width-1 query plan. Similar to the case for join tree enumeration discussed in Section 4.2, the search space for such plans is defined by four key factors: (1) selecting a root of the join tree to induce the width-1 query plan; (2) unnesting minor nodes; (3) handling possible re-branching, and (4) determining the local join order for each node and its neighbors. In this section, we detail our procedure to address them.

## 5.1 Re-Branching

As previously discussed, re-branching can happen when a child node $c$ of a node $p$ in the meta-decomposition could also be a child of another descendant node $d$ of $p$ in a join tree, if $\kappa(c) \subsetneq \kappa(d)$. This scenario, however, is in fact very rare in practice: we found no queries in the JOB benchmark that exhibited this scenario, and only one query in the TPC-H benchmark can involve re-branching. Therefore, for simplicity, we do not consider re-branching when describing the algorithm in this section. As we will show in the appendix, accommodating a possible re-branching adds only a constant factor to the overall complexity.

## 5.2 The Overall Algorithm

Given a meta-decomposition $M$, our algorithm to produce a width-1 query plan is shown in Algorithm 3. This algorithm first makes two traversals of $M$, one bottom-up and one top-down.

In the bottom-up traversal, for each non-root node $q$, we find the optimal plan to join the relation in $q$ and all the plans given by the children of $q$, which gives the optimal plan to evaluate $Q(T_q)$, the induced query on the subtree of $M$ rooted at $q$.

P1 R3.R1

*Example 5.1.* For the query in Example 3.1, with meta-decomposition shown in Fig. 5, this bottom-up traversal visits the nodes in the following order. (For simplicity, we simply use $R_i$ to refer to the node with $\lambda$-label $\{R_i\}$.)

(1) $R_3$. It generates the plan for the subtree $T_{R_3}$, or $T_{R_2 \to R_3}$, containing $R_3$ only. The plan is simply a scan of $R_3$.
(2) $R_2$. It generates the plan for $T_{R_2}$, or $T_{R_1 \to R_2}$, containing $R_2$ and $R_3$. It simply needs to join $R_2$ with the plan for $T_{R_3}$ in the previous step, i.e., $R_2 \bowtie R_3$.
(3) $R_4$. It generates the plan for the subtree $T_{R_4}$, or $T_{R_1 \to R_4}$, containing $R_4$ only. The plan is simply a scan of $R_4$.
(4) $R_1$. This is the root node. The traversal terminates here.

Then, in the top-down traversal, for each node $q$ and each child $c$ of $q$ on the meta-decomposition, we find the optimal plan to join $q$ and all plans given by the neighbors (parent and children) of $q$ except $c$, which gives the optimal plan to join all relations in $T_{c \to q}$.

*Example 5.2.* Again, for the query in Example 3.1, the top-down traversal visits the nodes in the following order.

- $R_1 \to$ child $R_2$. It generates the plan for the subtree $T_{R_2 \to R_1}$, containing $R_1$ and $R_4$. The plan is simply $R_1 \bowtie R_4$.
- $R_2 \to$ child $R_3$. It generates the plan for the subtree $T_{R_3 \to R_2}$, containing $R_1$, $R_2$, and $R_4$. Similarly, we join $R_2$ with the result of $R_1 \bowtie R_4$, which is the plan for the subtree $T_{R_2 \to R_1}$.
- $R_1 \to$ child $R_4$. It generates the plan for the subtree $T_{R_4 \to R_1}$, containing $R_1$, $R_2$, and $R_3$. Here, $R_2$ and $R_3$ are in the same subtree $T_{R_1 \to R_2}$ of $R_1$. So, we first take the plan generated for this subtree, which is $R_1 \bowtie R_2$, and then join this result with $R_1$.

At the end, we choose the root of the query plan, which is the minimum-cost plan of the form $\mathsf{plan}(T_{p \to q}) \bowtie \mathsf{plan}(T_{q \to p})$ among all $(p, q) \in E(M)$. For example, the width-1 plan shown in Fig. 3c corresponds to $\mathsf{plan}(T_{R_2 \to R_1}) \bowtie \mathsf{plan}(T_{R_1 \to R_2})$.

In the example we just considered, the fan-out is 2. For higher fan-outs, at each node in the meta-decomposition, we need to decide on an ordering to join the results of multiple subtrees. To do so, the algorithm calls the function optimizeLocal, which optimizes the local join for each node and its neighbors. We will elaborate on this in Section 5.3.

## 5.3 Optimizing the Local Join with Neighbors

One essential component of this algorithm is to, for a vertex $q$ in the meta-decomposition, find the optimal plan to join the relation in this vertex and the optimal query plans given by (a subset of) its neighbors (optimizeLocal). This can be viewed as a classic star join optimization problem, where $R_q$ is the hub relation, and the result of the induced query $Q(T_{c_i})$ from each of its neighbors $c_i$ is a satellite relation, denoted by $R_i$ for $i = 1, \ldots, |C_q|$, where $\chi(c_i) \cap \chi(q) \neq \emptyset$

and $\chi(c_i) \cap \chi(c_j) \subseteq \chi(q)$ for all $i \neq j$. Unfortunately, even in this restricted setting, this problem of optimizing the join of $R_q$ and all $R_i$'s is still NP-hard, because this can be reduced from the problem of optimizing the ordering a Cartesian product $S_1 \times S_2 \times \cdots \times S_n$—which is already shown to be NP-hard [41]—by adding a shared attribute to each $S_i$ such that all tuples have the same value for this attribute. By using dynamic programming algorithms [36, 43], the problem can be solved in $\widetilde{O}(2^{|C_q|})$, where $\widetilde{O}$ suppresses polylogarithmic factors and $|C_q|$ equals the fan-out of $q$ on the meta-decomposition. However, as mentioned in Section 3, *the fan-out of real-world queries is typically very small.* This makes the exponential-time exact algorithm practical for the vast majority of cases.

In the rare case that a node has a high fan-out, making the exact DP algorithm too slow, we can substitute it with a heuristic. An effective choice is Greedy Operator Ordering (GOO) [17], which iteratively joins the neighbor that results in the smallest intermediate result and results in a left-deep plan. This method is outlined in Algorithm 8 in Appendix E.2.

## 5.4 Case Study: Optimizing Relation-Dominated Queries with Meta-Decompositions

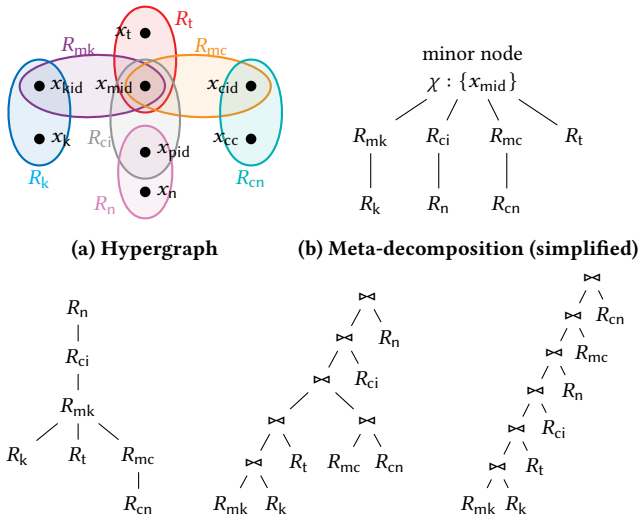
Our framework naturally incorporates projection pushdown for early cardinality reduction. When building a plan for a subquery, such as $\mathsf{plan}(T_{p\to q})$ in Algorithm 3, we can immediately project the result to only the attributes needed for subsequent operations. Specifically, after computing the joins within the subtree $T_q$, we only need to retain (1) attributes that are part of the final output, and (2) attributes required for joining with the remaining relations, which is $\kappa(q)$ by definition.

This strategy is especially effective for relation-dominated queries, where a single relation contains all output attributes. If a subtree contains no output attributes, we can project its result to only $\kappa(q)$, i.e., the join keys with the remaining relations, which can significantly reduce the size of intermediate results. However, traditional dynamic-programming-based optimizers lack such a comprehensive structural view. They therefore struggle to identify such opportunities but have to consider the entire search space of query plans and rely solely on the cost estimation.

*Example 5.3.* Take query 17f in the join order benchmark (JOB) as an example. Omitting irrelevant attributes in each relation, this query can be abstractly represented as

$$\begin{aligned} Q \leftarrow \pi_{x_\mathsf{n}}(&R_\mathsf{ci}[x_\mathsf{mid}, x_\mathsf{pid}] \bowtie R_\mathsf{cn}[x_\mathsf{cid}, x_\mathsf{cc}] \bowtie \sigma_{x_\mathsf{k}=\dots}(R_\mathsf{k}[x_\mathsf{kid}, x_\mathsf{k}]) \\ &\bowtie R_\mathsf{mc}[x_\mathsf{mid}, x_\mathsf{cid}] \bowtie R_\mathsf{mk}[x_\mathsf{mid}, x_\mathsf{kid}] \\ &\bowtie \sigma_{x_\mathsf{n}=\dots}(R_\mathsf{n}[x_\mathsf{pid}, x_\mathsf{n}]) \bowtie R_\mathsf{t}[x_\mathsf{mid}, x_\mathsf{t}]). \end{aligned}$$

This is a relation-dominated query, as it has only one output attribute $x_\mathsf{n}$, which is an attribute of $R_\mathsf{n}$. Fig. 7 shows the hypergraph, the optimal width-1 query plan (found by metaDecomp given cardinalities of the real data set), along with the join tree inducing it, and a width-2 query plan (which is the plan given by dynamic programming). In the width-1 plan in Fig. 7d, *all intermediate join results can be projected to a single attribute*, since the interface is always only one attribute, and there is no output attribute before the final join with $R_\mathsf{n}$. But, with the width-2 query plan in Fig. 7e, *the result of* $((R_\mathsf{mk} \bowtie R_\mathsf{k}) \bowtie R_\mathsf{t}) \bowtie R_\mathsf{ci}$ *has to be projected to three attributes,* $x_\mathsf{mid}$, $x_\mathsf{pid}$, *and* $x_\mathsf{n}$—the former two are join keys, and $x_\mathsf{n}$ is the output attribute. Executing these plans, we indeed observe that *the width-1 plan has a* 1.72x *speedup over the width-2 plan.*




(a) Hypergraph (b) Meta-decomposition (simplified) (c) Join tree inducing the width-1 plan in Fig. 7d (d) Optimal width-1 query plan, induced by the join tree in Fig. 7c (e) Optimal plan by dynamic programming, of width 2

**Figure 7: The hypergraph, the meta-decomposition, the optimal width-1 query plan with the inducing join tree, and the optimal query plan (of width 2) found by dynamic programming, of the query JOB 17f in Example 5.3**

As with evaluating join-projection queries, one can easily modify the cost model and use a join-tree-based query evaluation algorithm, including any Yannakakis-style algorithm, that incorporates cost-based optimization via meta-decomposition. We hope this work inspires further research in related areas and encourages the integration of these theoretically desirable query-evaluation techniques into practical systems. **P9a R2.O1** **P3 R1.O3**

# 6 Experimental Evaluation & Analysis

In this section, we present experimental results to answer the following research questions:

**(RQ1)** How fast is query optimization using meta-decompositions?
**(RQ2)** How good are width-1 plans, compared to the optimal?
**(RQ3)** To what extent is meta-decomposition-based query optimization sensitive to cardinality misestimation? **P10 R2.O3**
**(RQ4)** To what extent can the proposed join tree enumeration techniques help structure-based query evaluation?

## 6.1 Experimental Setup

*6.1.1 Experimental Environment.* Experiments are conducted on an Apple M4 Pro machine with 14 cores and 48 GB RAM, running macOS Tahoe 16.0. We use DuckDB as our test database because of its excellent performance and simple cost model. The experiments use DuckDB 1.2.2, Scala 3.6.3, GraalVM 17.0.12, and LLVM 20.1.3.

We implement metaDecomp in Scala. It first computes the meta-decomposition for a given query (Section 4) and then applies our cost-based optimization algorithm (Section 5) to select the best

width-1 query plan. In the experiments, we compare the following query optimization approaches:

(1) metaDecomp, using our implementation.
(2) DPconv [43], the state-of-the-art dynamic programming optimizer.
(3) DuckDB's native optimizer [39], which uses DPccp [36] for queries with up to 12 relations and Greedy Operator Ordering (GOO) [17] otherwise.
P9(c) R1.O8 (4) UnionDP, a heuristic approach proposed in [34, Algorithm 4] which (i) partitions the query graph into subgraphs, each with size up to a threshold (which we set to 12, consistent with DuckDB), (ii) runs dynamic programming on each subgraph, and (iii) runs dynamic programming on the entire graph, treating each subgraph as a composite node.
P9(a) R2.O1 (5) Yannakakis$^+$ [47], a state-of-the-art structure-guided query evaluation approach.
(6) LearnedRewrite [58, 59], a machine-learning-based query rewriter.
P9(b) R2.O2 (7) LLM-R$^2$ [33], a query rewriter based on machine learning and large language models (LLMs).

For Yannakakis$^+$, Learned Rewrite, and LLM-R$^2$, we use their implemented systems as black boxes and run the rewritten queries in DuckDB. For metaDecomp, DPconv, and UnionDP, the query plans are enforced in DuckDB by rewriting queries into a sequence of subqueries that create a temporary view at each step, as is standard in, e.g., [20, 47]. P3 R1.O3 R3.O4 For all these three methods, we apply the same standard approach of selection and projection pushdown. Namely, we project out all irrelevant attributes (those that are neither in the output nor in subsequent joins) at every step, and we integrate each selection (filter) condition in the join or scan operation at the lowest possible level, i.e., as soon as all relevant attributes exist.

Each query plan is executed 10 times, and we report the median optimization and execution time. To ensure accuracy, we exclude I/O time from runtime measurements. The full dataset is preloaded into the DuckDB database, and the estimated cardinalities are preloaded into memory, with all loading time excluded from timing. We enforce a 5-minute timeout on execution time. Queries that none of the optimizers can complete within this time limit are excluded from the results.

*6.1.2 Queries, Benchmarks, and Cardinalities.* We evaluate our approach on the following four benchmarks:

(1) The SPJ (select-project-join) queries in the **Decision Support Benchmark (DSB)** [16]. We only consider acyclic queries (240 out of 300) and evaluate them on a 10 GB database instance.
(2) The **Join Order Benchmark (JOB)** [32] over the IMDB dataset. All queries in this benchmark are acyclic.
(3) **Musicbrainz**, a benchmark proposed by [34] with significantly larger queries that join up to 26 relations. We take only the acyclic queries from the benchmark (243 out of 375).
(4) **JOBLarge**, a new benchmark we introduce to extend JOB to simulate practical but larger analytical scenarios, where each query is generated by selecting two queries from the JOB that share at least one common output attribute and merging them into a single query, with a join on a shared attribute, such that the query remains acyclic. We only retain generated queries that take DuckDB at least 50 ms to evaluate.

P7 R1.O5

| Benchmarks → | | DSB | JOB | Music-brainz | JOB-Large | Overall |
|---|---|---|---|---|---|---|
| Number of relations | Min | 5 | 4 | 2 | 8 | 2 |
| | Median | 6 | 8 | 10 | 18 | 8 |
| | Max | 9 | 17 | 26 | 34 | 34 |
| Fan-out | Min | 2 | 2 | 1 | 2 | 1 |
| | Median | 3 | 3 | 4 | 4 | 3 |
| | Max | 4 | 5 | 9 | 5 | 9 |
| Number of join trees | Min | 5 | 12 | 2 | 72 | 2 |
| | Median | 7 | 144 | 22500 | 40000 | 126 |
| | Max | 54 | 352512 | $> 10^8$ | $> 10^8$ | $> 10^8$ |

**Table 1: Structural properties of queries in each benchmark**

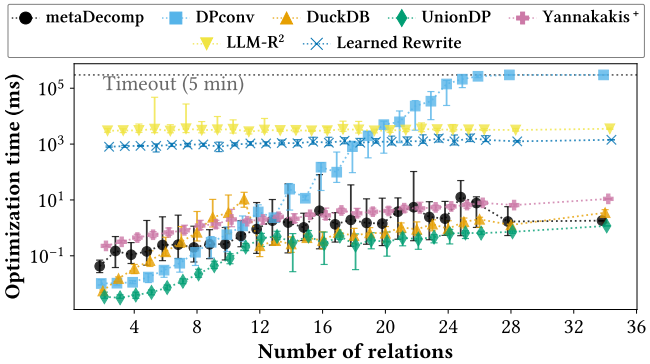



**Figure 8: Optimization time for all queries in all benchmarks. For each number of relations, the marker shows the average, and the whiskers show the minimum and maximum time.**

In Table 1, we show detailed statistics of the structural properties of the queries in each benchmark. P7 R1.O5

For metaDecomp, DPconv, and UnionDP, which all follow the same cost model, we precompute (by running appropriate `COUNT(*)` queries) true cardinalities of all possible intermediate join results for the relatively small benchmarks DSB and JOB, in the approach and format consistent with the implementation of DPconv [43]. For the larger benchmarks, Musicbrainz and JOBLarge, computing exact cardinalities for all possible intermediate joins is infeasible due to the much larger query sizes. Therefore, we use estimated cardinalities instead, via DuckDB's `EXPLAIN` command. Note, however, that these estimates can sometimes be highly inaccurate and may not reflect the true relative sizes of intermediate results. DuckDB, Yannakakis$^+$, Learned Rewrite, and LLM-R$^2$ are used as black boxes with their own cost models.

## 6.2 Results

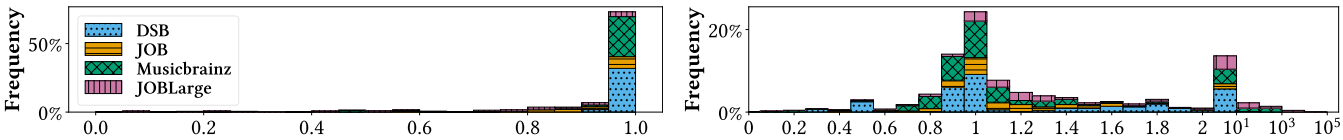
In this section, we present the results of our experiments. Due to space constraints, we include only summarized figures with all queries from all benchmarks. Detailed separate figures for individual benchmarks are included in Appendix F.

*6.2.1 (RQ1) Optimization Time.* We first compare the time taken for optimization. The overall results are shown in Fig. 8. The figure clearly shows that the optimization time of metaDecomp remains

P8–9 R1.O4 R1.O8 R2.O1 R2.O2

| Metric ↓ | Benchmark → / Method ↓ | DSB Mean[3] | DSB Median | DSB 95th | DSB 99th | JOB Mean[3] | JOB Median | JOB 95th | JOB 99th | Musicbrainz Mean[3] | Musicbrainz Median | Musicbrainz 95th | Musicbrainz 99th | JOBLarge Mean[3] | JOBLarge Median | JOBLarge 95th | JOBLarge 99th |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Optimization time | metaDecomp | 0.15 ms | 0.10 ms | 0.32 ms | 1.01 ms | 0.38 ms | 0.26 ms | 0.94 ms | 1.04 ms | 2.62 ms | 0.37 ms | 10.45 ms | 45.05 ms | 0.59 ms | 0.37 ms | 1.67 ms | 4.17 ms |
| | DPconv | 0.08 ms | 0.03 ms | 0.36 ms | 0.37 ms | 8.32 ms | 0.16 ms | 38.22 ms | 45.08 ms | 7.37 s | 0.30 ms | 14.62 s | 244.44 s | 42.17 s | 1.26 s | > 5 min | > 5 min |
| | DuckDB | 0.32 ms | 0.12 ms | 1.83 ms | 1.87 ms | 1.13 ms | 0.32 ms | 5.58 ms | 8.43 ms | 1.89 ms | 0.47 ms | 11.34 ms | 18.59 ms | 0.62 ms | 0.55 ms | 1.23 ms | 3.81 ms |
| | UnionDP | 0.01 ms | 0.007 ms | 0.04 ms | 0.04 ms | 0.10 ms | 0.03 ms | 0.38 ms | 0.45 ms | 0.25 ms | 0.11 ms | 0.64 ms | 0.77 ms | 0.29 ms | 0.24 ms | 0.77 ms | 1.18 ms |
| | Yannakakis$^+$ | 0.78 ms | 0.70 ms | 1.48 ms | 1.50 ms | 1.44 ms | 1.37 ms | 3.78 ms | 4.59 ms | 2.29 ms | 1.72 ms | 6.43 ms | 8.15 ms | 3.55 ms | 3.27 ms | 6.53 ms | 10.89 ms |
| | LearnedRewrite | 0.89 s | 0.95 s | 1.03 s | 1.06 s | 0.96 s | 0.94 s | 1.13 s | 1.30 s | 1.11 s | 1.00 s | 1.78 s | 2.14 s | 1.09 s | 1.10 s | 1.29 s | 1.42 s |
| | LLM-R$^2$ | 3.03 s | 2.92 s | 3.66 s | 4.71 s | 3.75 s | 2.99 s | 3.98 s | 24.96 s | 3.11 s | 3.03 s | 3.57 s | 6.10 s | 3.40 s | 3.11 s | 5.72 s | 6.84 s |
| Execution time | metaDecomp | 0.01 s | 0.01 s | 0.04 s | 0.04 s | 0.04 s | 0.03 s | 0.08 s | 0.11 s | 1.08 s | 0.52 s | 2.56 s | 13.75 s | 1.02 s | 0.17 s | 2.31 s | 23.84 s |
| | DPconv | 0.02 s | 0.01 s | 0.09 s | 0.10 s | 0.04 s | 0.04 s | 0.09 s | 0.11 s | 1.42 s | 0.50 s | 2.65 s | 28.27 s | 17.50 s | 0.24 s | 89.05 s | > 5 min |
| | DuckDB | 23.85 s | 0.01 s | > 5 min | > 5 min | 0.04 s | 0.04 s | 0.09 s | 0.13 s | 95.92 s | 0.52 s | > 5 min | > 5 min | 22.46 s | 0.13 s | > 5 min | > 5 min |
| | UnionDP | 0.03 s | 0.01 s | 0.10 s | 0.11 s | 0.05 s | 0.04 s | 0.10 s | 0.12 s | 58.75 s | 0.28 s | > 5 min | > 5 min | 25.04 s | 0.15 s | > 5 min | > 5 min |
| | Yannakakis$^+$ | 0.03 s | 0.02 s | 0.08 s | 0.08 s | 0.06 s | 0.06 s | 0.12 s | 0.15 s | 13.63 s | 0.41 s | 34.61 s | > 5 min | 5.11 s | 0.13 s | 1.52 s | 232.60 s |
| | LearnedRewrite | 23.95 s | 0.05 s | > 5 min | > 5 min | 26.91 s | 0.15 s | > 5 min | > 5 min | 42.47 s | 0.42 s | > 5 min | > 5 min | 42.03 s | 0.14 s | > 5 min | > 5 min |
| | LLM-R$^2$ | 23.79 s | 0.01 s | > 5 min | > 5 min | 0.04 s | 0.04 s | 0.10 s | 0.14 s | 96.32 s | 0.53 s | > 5 min | > 5 min | 29.94 s | 0.12 s | > 5 min | > 5 min |
| Overall evaluation time | metaDecomp | 0.01 s | 0.01 s | 0.04 s | 0.04 s | 0.04 s | 0.03 s | 0.08 s | 0.11 s | 1.09 s | 0.52 s | 2.65 s | 13.75 s | 1.02 s | 0.17 s | 2.31 s | 23.84 s |
| | DPconv | 0.02 s | 0.01 s | 0.09 s | 0.10 s | 0.05 s | 0.04 s | 0.10 s | 0.12 s | 8.78 s | 0.51 s | 26.36 s | 246.63 s | 58.26 s | 4.32 s | > 5 min | > 5 min |
| | DuckDB | 23.85 s | 0.01 s | > 5 min | > 5 min | 0.04 s | 0.04 s | 0.09 s | 0.13 s | 95.93 s | 0.52 s | > 5 min | > 5 min | 22.46 s | 0.13 s | > 5 min | > 5 min |
| | UnionDP | 0.03 s | 0.01 s | 0.10 s | 0.11 s | 0.05 s | 0.04 s | 0.10 s | 0.12 s | 58.75 s | 0.28 s | > 5 min | > 5 min | 25.04 s | 0.15 s | > 5 min | > 5 min |
| | Yannakakis$^+$ | 0.03 s | 0.02 s | 0.08 s | 0.08 s | 0.06 s | 0.06 s | 0.13 s | 0.15 s | 13.63 s | 0.41 s | 34.61 s | > 5 min | 5.11 s | 0.13 s | 1.52 s | 232.60 s |
| | LearnedRewrite | 24.84 s | 1.01 s | > 5 min | > 5 min | 27.87 s | 1.12 s | > 5 min | > 5 min | 54.93 s | 1.43 s | > 5 min | > 5 min | 43.12 s | 1.28 s | > 5 min | > 5 min |
| | LLM-R$^2$ | 26.82 s | 3.03 s | > 5 min | > 5 min | 3.79 s | 3.05 s | 4.01 s | 24.99 s | 99.43 s | 4.06 s | > 5 min | > 5 min | 33.34 s | 3.41 s | > 5 min | > 5 min |
| Overall speedup of metaDecomp over ... | DPconv | 1.22x | 0.99x | 2.90x | 3.29x | 1.18x | 1.08x | 2.77x | 3.11x | 1.34x | 0.99x | 15.49x | 138.59x | 27.61x | 21.23x | 2308.55x | 5014.14x |
| | DuckDB | 2.75x | 0.97x | 16938.94x | 18945.33x | 1.11x | 1.07x | 2.03x | 3.53x | 6.39x | 1.86x | 238.77x | 901.19x | 1.46x | 0.82x | 118.01x | 1872.97x |
| | UnionDP | 1.28x | 1.03x | 3.38x | 6.57x | 1.12x | 1.04x | 2.09x | 3.01x | 2.67x | 0.94x | 293.61x | 844.52x | 1.91x | 0.94x | 606.52x | 2328.21x |
| | Yannakakis$^+$ | 1.83x | 1.91x | 2.77x | 7.03x | 1.61x | 1.50x | 3.75x | 5.07x | 1.07x | 0.72x | 47.66x | 656.33x | 0.76x | 0.73x | 2.13x | 22.05x |
| | LearnedRewrite | 142.29x | 123.70x | 16993.51x | 19006.30x | 59.91x | 38.45x | 7958.89x | 11031.32x | 14.49x | 11.44x | 442.62x | 967.06x | 13.65x | 8.85x | 2444.33x | 7240.85x |
| | LLM-R$^2$ | 379.88x | 282.20x | 17095.75x | 19117.48x | 103.68x | 98.29x | 484.18x | 832.41x | 65.93x | 78.95x | 927.52x | 2488.64x | 23.98x | 21.60x | 174.23x | 3632.91x |

**Table 2: Statistics of query evaluation time**




**(a) Ratio between the costs[4]** **(b) Speedup of execution**

**Figure 9: Comparison of the optimal width-1 plan over the global optimal**

almost constant as the number of relations increases, typically below 10 milliseconds, as is the case for DuckDB, UnionDP, and Yannakakis$^+$. The two machine-learning-based approaches, LLM-R$^2$ and Learned Rewrite, exhibit stable but higher optimization times in the order of seconds, due to the overhead of the complex models and their interaction with LLMs. DPconv, however, exhibits exponential growth, and its optimization time surpasses that of metaDecomp starting at approximately 9 relations. For many queries in Musicbrainz and JOBLarge (starting from approximately 25 relations), DPconv fails to find a valid plan within the 5-minute limit, again highlighting that traditional exponential-time dynamic-programming-based optimizers do not scale to such large queries.

*6.2.2 (RQ2) Quality of width-1 query plans.* Even though metaDecomp restricts its search to width-1 plans, as shown in Fig. 9a, in most queries, it *finds query plans with cost values comparable to the optimal*. Furthermore, Fig. 9b shows the speedups achieved by executing the optimal width-1 plans over executing the globally optimal plan. Note that, although one would typically not expect speedups since the optimal plans have the minimum possible cost values, in many cases, the optimal width-1 query plans in fact show speedups in the actual execution, thanks to the small intermediate result sizes by exploiting the structural information of queries.

P2 R1.O1 R1.O2

*6.2.3 (RQ1) & (RQ2) Together: Overall Performance.* Taking both optimization and execution time into account, we observe in Table 2, Fig. 10, and Fig. 11 that metaDecomp shows a clear advantage overall. This is especially the case for queries that are more complex, as shown by the 95th and 99th percentiles in Table 2, and in the points towards the right in Fig. 11, where DPconv or DuckDB takes more than 1 second ($10^3$ ms). (Due to the space constraints, for Fig. 10 and Fig. 11, we only show the comparison with DPconv and DuckDB. Comparison with all other methods is included in Appendix F.) For DuckDB, UnionDP, Yannakakis$^+$, LearnedRewrite, and LLM-R$^2$, although their optimization is fast, they are also very likely to generate plans that are significantly worse than the optimal width-1 query plans, especially for larger queries. In addition, for Yannakakis$^+$, it also has an overhead of the reduction phase, which may be costly for certain queries. For DPconv, the disadvantage is mostly due to the exponential optimization time, which becomes the dominating factor of the overall evaluation time for large queries.

P9a R2.O1 R1.O4

[3] Arithmetic mean for time, geometric mean for speedup factors.

[4] The cost of the globally optimal plan divided by the cost of the optimal width-1 plan, analogous to speedups. A ratio of 1 indicates the two have the same cost, and is included in the bar between 0.95 and 1.0. Since width-1 plans are a restricted class, their costs are at least the global optimal and thus have ratios at most 1.

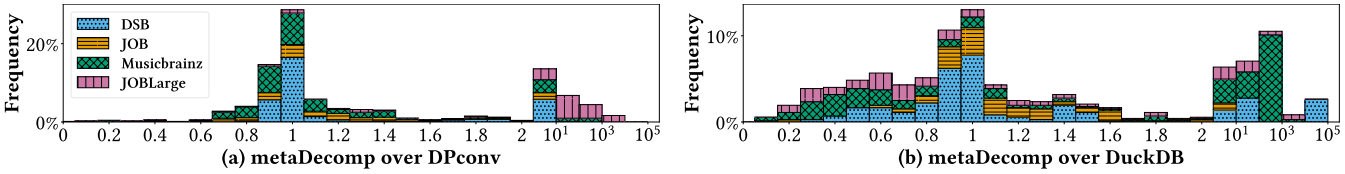



**Figure 10: Histogram of the overall speedup of using metaDecomp over using DPconv and DuckDB**

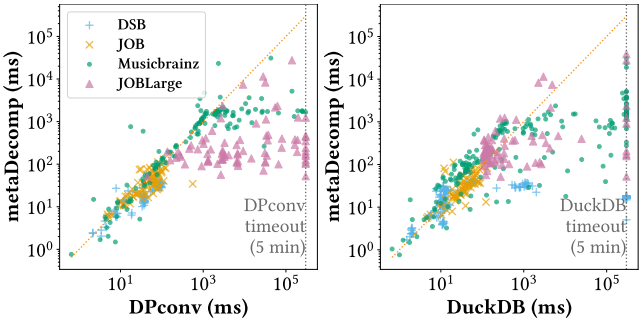



**Figure 11: Scatter plot of overall query evaluation time of using metaDecomp over using DPconv and DuckDB**

P10 R2.O3

| Method ↓ | Benchmark → / Cardinality ↓ | DSB Mean | DSB Median | DSB 95th | DSB 99th | JOB Mean | JOB Median | JOB 95th | JOB 99th |
|---|---|---|---|---|---|---|---|---|---|
| metaDecomp | Exact | 0.01 s | 0.01 s | 0.04 s | 0.04 s | 0.04 s | 0.03 s | 0.08 s | 0.11 s |
| | Misestimated | 0.02 s | 0.01 s | 0.06 s | 0.12 s | 0.05 s | 0.04 s | 0.11 s | 0.15 s |
| DPconv | Exact | 0.02 s | 0.01 s | 0.09 s | 0.10 s | 0.04 s | 0.04 s | 0.09 s | 0.11 s |
| | Misestimated | 0.03 s | 0.01 s | 0.11 s | 0.14 s | 0.46 s | 0.05 s | 0.23 s | 2.07 s |
| UnionDP | Exact | 0.03 s | 0.01 s | 0.10 s | 0.11 s | 0.05 s | 0.04 s | 0.10 s | 0.12 s |
| | Misestimated | 0.03 s | 0.01 s | 0.11 s | 0.14 s | 0.48 s | 0.05 s | 0.22 s | 1.86 s |

**Table 3: Query execution time when optimized with exact vs misestimated cardinalities**

The overall results demonstrate that finding and executing width-1 plans with metaDecomp is indeed highly efficient in practice.

P10 R2.O3

*6.2.4 (RQ3) Sensitivity to cardinality misestimation.* Recent research has shown that structure-based query optimization tends to be more robust even in the presence of errors in cardinality estimation [8, 47, 57]. This is also the case for metaDecomp. To illustrate, we compare metaDecomp with DPconv and UnionDP, which use the exact same cost model, by running them on DSB and JOB queries with (1) exact and (2) misestimated cardinalities. The misestimated cardinalities are generated from the exact cardinalities by adding a multiplicative noise to the exact cardinalities using a log-normal distribution. Specifically, for each original cardinality $C$, we generate a perturbed estimate $\hat{C} = C \cdot e^{\epsilon}$, where $\epsilon \sim \mathcal{N}(0, \sigma^2)$, i.e., following a normal (Gaussian) distribution with standard deviation $\sigma$ set to 10. From the results in Table 3, metaDecomp still exhibits excellent performance despite such cardinality estimation errors. The advantage is particularly prominent in the 99th percentile of the JOB benchmark, where DPconv and UnionDP are significantly misled by inaccurate cardinality estimations.

*6.2.5 (RQ4) Meta-decompositions applied to other structure-based optimization techniques.* Finally, we demonstrate how meta-decompositions can help other recently developed structure-based optimization techniques. A crucial step in those techniques is selecting a good join tree, as it can significantly influence the structures

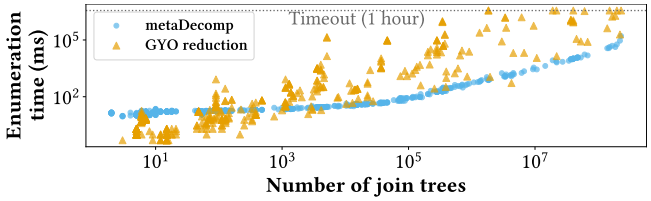



**Figure 12: Time to enumerate join trees**

and, consequently, the cost of the generated query plans. Thus, we first evaluate metaDecomp's efficiency in *enumerating join trees*, compared to the standard GYO-reduction-based approach from [14]. As shown in Fig. 12, metaDecomp can exhibit up to *four orders-of-magnitude of speedup* over the traditional approach. In particular, for many highly complex queries in the Musicbrainz and JOBLarge benchmarks with more than $10^7$ join trees, the traditional approach already takes over 1 hour, whereas metaDecomp can enumerate the entire search space in 2 seconds to 1 minute.

A more effective approach is to apply cost-based optimization directly on the meta-decomposition, as has already been done in metaDecomp, since this eliminates the need to enumerate the exponential number of join trees. Therefore, in our second experiment, we use Yannakakis$^+$ [47] to rewrite queries, using the join tree selected by metaDecomp that induces the optimal width-1 query plan. *This modification achieves an average speedup of* 1.11x *on JOB queries.* These results demonstrate that structure-based evaluation algorithms, such as Yannakakis$^+$, can benefit significantly from our efficient cost-based optimization framework.

## 7 Conclusions and Future Work

In this paper, we have introduced novel techniques for query optimization based on structural properties via meta-decompositions. They offer promising opportunities to integrate theoretically desirable structure-based optimization into practical database systems to efficiently evaluate large, complex queries. A natural next step is to extend our width notion and meta-decomposition-based framework for cyclic queries. While width-1 query plans are infeasible for cyclic queries based on Theorem 3.6, we may, e.g., identify minimal-width query plans and represent all minimal-width hypertree decompositions. In addition, to improve our current simple strategy of filter pushdown, it is possible to incorporate existing techniques for evaluating conjunctive queries with comparisons [49]. The performance of such methods could also benefit from our cost-based optimization framework. Finally, although our technique has shown its robustness, it would still benefit from more accurate cardinality estimation. Recent research [11, 55] shows promising techniques that can be adopted to develop efficient, reliable cardinality estimators suitable within our proposed framework.

P12 R3.O4

P10 R2.O3

# References


[1] Serge Abiteboul, Richard Hull, and Victor Vianu. 1996. *Foundations of Databases*. Addison-Wesley, Reading, Massachusetts.

[2] Mahmoud Abo Khamis, Hung Q. Ngo, and Atri Rudra. 2016. FAQ: Questions Asked Frequently. In *Proceedings of the 35th ACM SIGMOD-SIGACT-SIGAI Symposium on Principles of Database Systems* (San Francisco, California, USA) *(PODS '16)*. Association for Computing Machinery, New York, NY, USA, 13–28. https://doi.org/10.1145/2902251.2902280

[3] Mahmoud Abo Khamis, Hung Q. Ngo, and Dan Suciu. 2017. What Do Shannon-type Inequalities, Submodular Width, and Disjunctive Datalog Have to Do with One Another?. In *Proceedings of the 36th ACM SIGMOD-SIGACT-SIGAI Symposium on Principles of Database Systems* (Chicago, Illinois, USA) *(PODS '17)*. Association for Computing Machinery, New York, NY, USA, 429–444. https://doi.org/10.1145/3034786.3056105

[4] Guillaume Bagan, Arnaud Durand, and Etienne Grandjean. 2007. On Acyclic Conjunctive Queries and Constant Delay Enumeration. In *Computer Science Logic*, Jacques Duparc and Thomas A. Henzinger (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 208–222.

[5] Liese Bekkers, Frank Neven, Stijn Vansummeren, and Yisu Remy Wang. 2024. Instance-optimal acyclic join processing without regret: Engineering the Yannakakis algorithm in column stores. *arXiv preprint arXiv:2411.04042* (2024).

[6] Kristin Bennett, Michael C. Ferris, and Yannis E. Ioannidis. 1991. *A genetic algorithm for database query optimization*. Technical Report #1004. Center for Parallel Optimization. https://minds.wisconsin.edu/bitstream/handle/1793/59438/TR1004.pdf?sequence=1

[7] Philip A. Bernstein and Dah-Ming W. Chiu. 1981. Using Semi-Joins to Solve Relational Queries. *J. ACM* 28, 1 (Jan. 1981), 25–40. https://doi.org/10.1145/322234.322238

[8] Altan Birler, Alfons Kemper, and Thomas Neumann. 2024. Robust Join Processing with Diamond Hardened Joins. *Proc. VLDB Endow.* 17, 11 (July 2024), 3215–3228. https://doi.org/10.14778/3681954.3681995

[9] Altan Birler, Mihail Stoian, and Thomas Neumann. 2025. Optimizing Linearized Join Enumeration by Adapting to the Query Structure. *Datenbanksysteme für Business, Technologie und Web (BTW 2025)* (2025), 171–194. https://doi.org/10.18420/BTW2025-09

[10] Angela Bonifati, Wim Martens, and Thomas Timm. 2020. An analytical study of large SPARQL query logs. *VLDB J.* 29, 2-3 (2020), 655–679. https://doi.org/10.1007/S00778-019-00558-9

[11] Walter Cai, Magdalena Balazinska, and Dan Suciu. 2019. Pessimistic Cardinality Estimation: Tighter Upper Bounds for Intermediate Join Cardinalities. In *Proceedings of the 2019 International Conference on Management of Data* (Amsterdam, Netherlands) *(SIGMOD '19)*. Association for Computing Machinery, New York, NY, USA, 18–35. https://doi.org/10.1145/3299869.3319894

[12] Jin Chen, Guanyu Ye, Yan Zhao, Shuncheng Liu, Liwei Deng, Xu Chen, Rui Zhou, and Kai Zheng. 2022. Efficient Join Order Selection Learning with Graph-based Representation. In *Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. ACM, Washington DC USA, 97–107. https://doi.org/10.1145/3534678.3539303

[13] Sophie Cluet and Guido Moerkotte. 1995. On the complexity of generating optimal left-deep processing trees with cross products. In *Database Theory — ICDT '95*, Georg Gottlob and Moshe Y. Vardi (Eds.). Springer, Berlin, Heidelberg, 54–67. https://doi.org/10.1007/3-540-58907-4_6

[14] Binyang Dai, Qichen Wang, and Ke Yi. 2023. SparkSQL+: Next-generation Query Planning over Spark. In *Companion of the 2023 International Conference on Management of Data*. ACM, Seattle WA USA, 115–118. https://doi.org/10.1145/3555041.3589715

[15] Shaleen Deep, Hangdong Zhao, Austen Z. Fan, and Paraschos Koutris. 2024. Output-sensitive Conjunctive Query Evaluation. *Proc. ACM Manag. Data* 2, 5, Article 220 (Nov. 2024), 24 pages. https://doi.org/10.1145/3695838

[16] Bailu Ding, Surajit Chaudhuri, Johannes Gehrke, and Vivek Narasayya. 2021. DSB: a decision support benchmark for workload-driven and traditional database systems. *Proceedings of the VLDB Endowment* 14, 13 (Sept. 2021), 3376–3388. https://doi.org/10.14778/3484224.3484234

[17] Leonidas Fegaras. 1998. A new heuristic for optimizing large queries. In *Database and Expert Systems Applications*, Gerald Quirchmayr, Erich Schweighofer, and Trevor J.M. Bench-Capon (Eds.). Springer, Berlin, Heidelberg, 726–735. https://doi.org/10.1007/BFb0054528

[18] Georg Gottlob, Gianluigi Greco, Nicola Leone, and Francesco Scarcello. 2016. Hypertree Decompositions: Questions and Answers. In *Proceedings of the 35th ACM SIGMOD-SIGACT-SIGAI Symposium on Principles of Database Systems, PODS 2016, San Francisco, CA, USA, June 26 - July 01, 2016*, Tova Milo and Wang-Chiew Tan (Eds.). ACM, 57–74. https://doi.org/10.1145/2902251.2902309

[19] Georg Gottlob, Martin Grohe, Nysret Musliu, Marko Samer, and Francesco Scarcello. 2005. Hypertree Decompositions: Structure, Algorithms, and Applications. In *Graph-Theoretic Concepts in Computer Science, 31st International Workshop, WG 2005, Metz, France, June 23-25, 2005, Revised Selected Papers (Lecture Notes in Computer Science, Vol. 3787)*, Dieter Kratsch (Ed.). Springer, 1–15. https://doi.org/10.1007/11604686_1

[20] Georg Gottlob, Matthias Paul Lanzinger, Davide Mario Longo, Cem Okulmus, Reinhard Pichler, and Alexander Selzer. 2023. Reaching Back to Move Forward: Using Old Ideas to Achieve a New Level of Query Optimization, Vol. 3409. CEUR-WS.org. https://doi.org/10.34726/5396 Accepted: 2024-01-29T11:16:26Z.

[21] Georg Gottlob, Nicola Leone, and Francesco Scarcello. 2002. Hypertree Decompositions and Tractable Queries. *J. Comput. Syst. Sci.* 64, 3 (2002), 579–627. https://doi.org/10.1006/JCSS.2001.1809

[22] Georg Gottlob, Nicola Leone, and Francesco Scarcello. 2003. Robbers, marshals, and guards: Game theoretic and logical characterizations of hypertree width. *J. Comput. Syst. Sci.* 66, 4 (2003), 775–808. https://doi.org/10.1016/S0022-0000(03)00030-8

[23] Goetz Graefe. 1995. The Cascades Framework for Query Optimization. *IEEE Data(base) Engineering Bulletin* 18 (1995), 19–29.

[24] Goetz Graefe and William J. McKenna. 1993. The Volcano optimizer generator: Extensibility and efficient search. In *Proceedings of IEEE 9th International Conference on Data Engineering*. IEEE Comput. Soc. Press, Vienna, Austria, 209–218. https://doi.org/10.1109/ICDE.1993.344061

[25] Marc H. Graham. 1979. *On the universal relation*. Technical Report. University of Toronto.

[26] Yuxing Han, Ziniu Wu, Peizhi Wu, Rong Zhu, Jingyi Yang, Liang Wei Tan, Kai Zeng, Gao Cong, Yanzhao Qin, Andreas Pfadler, Zhengping Qian, Jingren Zhou, Jiangneng Li, and Bin Cui. 2021. Cardinality estimation in DBMS: a comprehensive benchmark evaluation. *Proceedings of the VLDB Endowment* 15, 4 (Dec. 2021), 752–765. https://doi.org/10.14778/3503585.3503586

[27] Xiao Hu. 2025. Output-Optimal Algorithms for Join-Aggregate Queries. *Proc. ACM Manag. Data* 3, 2, Article 104 (June 2025), 27 pages. https://doi.org/10.1145/3725241

[28] Xiao Hu and Qichen Wang. 2023. Computing the Difference of Conjunctive Queries Efficiently. *Proc. ACM Manag. Data* 1, 2, Article 153 (June 2023), 26 pages. https://doi.org/10.1145/3589298

[29] Manas R. Joglekar, Rohan Puttagunta, and Christopher Ré. 2016. AJAR: Aggregations and Joins over Annotated Relations. In *Proceedings of the 35th ACM SIGMOD-SIGACT-SIGAI Symposium on Principles of Database Systems* (San Francisco, California, USA) *(PODS '16)*. Association for Computing Machinery, New York, NY, USA, 91–106. https://doi.org/10.1145/2902251.2902293

[30] Donald Kossmann and Konrad Stocker. 2000. Iterative dynamic programming: a new class of query optimization algorithms. *ACM Transactions on Database Systems* 25, 1 (March 2000), 43–82. https://doi.org/10.1145/352958.352982

[31] Paraschos Koutris, Stijn Vansummeren, Qichen Wang, Yisu Remy Wang, and Xiangyao Yu. 2026. Database Theory in Action: Yannakakis' Algorithm. In *29th International Conference on Database Theory (ICDT 2026)*. 21:1–21:6. https://doi.org/10.4230/LIPIcs.ICDT.2026.21

[32] Viktor Leis, Bernhard Radke, Andrey Gubichev, Atanas Mirchev, Peter Boncz, Alfons Kemper, and Thomas Neumann. 2018. Query optimization through the looking glass, and what we found running the Join Order Benchmark. *The VLDB Journal* 27, 5 (Oct. 2018), 643–668. https://doi.org/10.1007/s00778-017-0480-7

[33] Zhaodonghui Li, Haitao Yuan, Huiming Wang, Gao Cong, and Lidong Bing. 2024. LLM-$R^2$ : A Large Language Model Enhanced Rule-Based Rewrite System for Boosting Query Efficiency. *Proceedings of the VLDB Endowment* 18, 1 (Sept. 2024), 53–65. https://doi.org/10.14778/3696435.3696440

[34] Riccardo Mancini, Srinivas Karthik, Bikash Chandra, Vasilis Mageirakos, and Anastasia Ailamaki. 2022. Efficient Massively Parallel Join Optimization for Large Queries. In *Proceedings of the 2022 International Conference on Management of Data*. ACM, Philadelphia PA USA, 122–135. https://doi.org/10.1145/3514221.3517871

[35] Ryan Marcus, Parimarjan Negi, Hongzi Mao, Nesime Tatbul, Mohammad Alizadeh, and Tim Kraska. 2021. Bao: Making Learned Query Optimization Practical. In *Proceedings of the 2021 International Conference on Management of Data (SIGMOD '21)*. Association for Computing Machinery, New York, NY, USA, 1275–1288. https://doi.org/10.1145/3448016.3452838

[36] Guido Moerkotte and Thomas Neumann. 2008. Dynamic programming strikes back. In *Proceedings of the 2008 ACM SIGMOD international conference on Management of data (SIGMOD '08)*. Association for Computing Machinery, New York, NY, USA, 539–552. https://doi.org/10.1145/1376616.1376672

[37] Thomas Neumann and Bernhard Radke. 2018. Adaptive Optimization of Very Large Join Queries. In *Proceedings of the 2018 International Conference on Management of Data (SIGMOD '18)*. Association for Computing Machinery, New York, NY, USA, 677–692. https://doi.org/10.1145/3183713.3183733

[38] Heinz Prüfer. 1918. Neuer Beweis eines Satzes über Permutationen. *Archiv der Mathematischen Physik* 27 (1918), 742–744.

[39] Mark Raasveldt and Hannes Mühleisen. 2019. DuckDB: an Embeddable Analytical Database. In *Proceedings of the 2019 International Conference on Management of Data*. ACM, Amsterdam Netherlands, 1981–1984. https://doi.org/10.1145/3299869.3320212

[40] Francesco Scarcello, Gianluigi Greco, and Nicola Leone. 2007. Weighted hypertree decompositions and optimal query plans. *J. Comput. System Sci.* 73, 3 (May 2007),

475–506. https://doi.org/10.1016/j.jcss.2006.10.010

[41] Wolfgang Scheufele and Guido Moerkotte. 1996. *Constructing Optimal Bushy Processing Trees for Join Queries is NP-hard.* Technical Report. Universität Mannheim.

[42] P. Griffiths Selinger, M. M. Astrahan, D. D. Chamberlin, R. A. Lorie, and T. G. Price. 1979. Access path selection in a relational database management system. In *Proceedings of the 1979 ACM SIGMOD international conference on Management of data (SIGMOD '79)*. Association for Computing Machinery, New York, NY, USA, 23–34. https://doi.org/10.1145/582095.582099

[43] Mihail Stoian and Andreas Kipf. 2024. DPconv: Super-Polynomially Faster Join Ordering. *Proc. ACM Manag. Data* 2, 6, Article 234 (Dec. 2024), 26 pages. https://doi.org/10.1145/3698809

[44] Transaction Processing Council (TPC). 2022. TPC Benchmark H. Available: https://tpc.org/tpc_documents_current_versions/pdf/tpc-h_v3.0.1.pdf.

[45] Bennet Vance. 1998. *Join-order optimization with Cartesian products.* Ph. D. Dissertation. Oregon Graduate Institute of Science and Technology. https://digitalcollections.ohsu.edu/record/177

[46] Bennet Vance and David Maier. 1996. Rapid bushy join-order optimization with Cartesian products. In *Proceedings of the 1996 ACM SIGMOD international conference on Management of data (SIGMOD '96)*. Association for Computing Machinery, New York, NY, USA, 35–46. https://doi.org/10.1145/233269.233317

[47] Qichen Wang, Bingnan Chen, Binyang Dai, Ke Yi, Feifei Li, and Liang Lin. 2025. Yannakakis+: Practical Acyclic Query Evaluation with Theoretical Guarantees. *Proc. ACM Manag. Data* 3, 3, Article 235 (June 2025), 28 pages. https://doi.org/10.1145/3725423

[48] Qichen Wang, Qiyao Luo, and Yilei Wang. 2024. Relational Algorithms for Top-$k$ Query Evaluation. *Proc. ACM Manag. Data* 2, 3, Article 168 (May 2024), 27 pages. https://doi.org/10.1145/3654971

[49] Qichen Wang and Ke Yi. 2022. Conjunctive Queries with Comparisons. In *Proceedings of the 2022 International Conference on Management of Data* (Philadelphia, PA, USA) *(SIGMOD '22)*. Association for Computing Machinery, New York, NY, USA, 108–121. https://doi.org/10.1145/3514221.3517830

[50] Yifei Yang, Hangdong Zhao, Xiangyao Yu, and Paraschos Koutris. 2024. Predicate Transfer: Efficient Pre-Filtering on Multi-Join Queries. (2024).

[51] Zongheng Yang, Wei-Lin Chiang, Sifei Luan, Gautam Mittal, Michael Luo, and Ion Stoica. 2022. Balsa: Learning a Query Optimizer Without Expert Demonstrations. In *Proceedings of the 2022 International Conference on Management of Data*. ACM, Philadelphia PA USA, 931–944. https://doi.org/10.1145/3514221.3517885

[52] Mihalis Yannakakis. 1981. Algorithms for Acyclic Database Schemes. In *VLDB '81: Proceedings of the seventh international conference on Very Large Data Bases*, Vol. 7. 82–94.

[53] Clement T. Yu and Meral Z. Özsoyoğlu. 1979. An algorithm for tree-query membership of a distributed query. In *The IEEE Computer Society's Third International Computer Software and Applications Conference, COMPSAC 1979, 6-8 November, 1979, Chicago, Illinois, USA*. IEEE, 306–312. https://doi.org/10.1109/CMPSAC.1979.762509

[54] Tao Yu, Rui Zhang, Kai Yang, Michihiro Yasunaga, Dongxu Wang, Zifan Li, James Ma, Irene Li, Qingning Yao, Shanelle Roman, Zilin Zhang, and Dragomir Radev. 2018. Spider: A Large-Scale Human-Labeled Dataset for Complex and Cross-Domain Semantic Parsing and Text-to-SQL Task. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, Ellen Riloff, David Chiang, Julia Hockenmaier, and Jun'ichi Tsujii (Eds.). Association for Computational Linguistics, Brussels, Belgium, 3911–3921. https://doi.org/10.18653/v1/D18-1425

[55] Haozhe Zhang, Christoph Mayer, Mahmoud Abo Khamis, Dan Olteanu, and Dan Suciu. 2025. LpBound: Pessimistic Cardinality Estimation Using $\ell_p$-Norms of Degree Sequences. *Proc. ACM Manag. Data* 3, 3, Article 184 (June 2025), 27 pages. https://doi.org/10.1145/3725321

[56] Hangdong Zhao, Austen Z. Fan, Xiating Ouyang, and Paraschos Koutris. 2024. Conjunctive Queries with Negation and Aggregation: A Linear Time Characterization. *Proc. ACM Manag. Data* 2, 2, Article 75 (May 2024), 19 pages. https://doi.org/10.1145/3651138

[57] Junyi Zhao, Kai Su, Yifei Yang, Xiangyao Yu, Paraschos Koutris, and Huanchen Zhang. 2025. Debunking the Myth of Join Ordering: Toward Robust SQL Analytics. *Proc. ACM Manag. Data* 3, 3, Article 146 (June 2025), 28 pages. https://doi.org/10.1145/3725283

[58] Xuanhe Zhou, Guoliang Li, Chengliang Chai, and Jianhua Feng. 2021. A learned query rewrite system using Monte Carlo tree search. *Proc. VLDB Endow.* 15, 1 (Sept. 2021), 46–58. https://doi.org/10.14778/3485450.3485456

[59] Xuanhe Zhou, Guoliang Li, Jianming Wu, Jiesi Liu, Zhaoyan Sun, and Xinning Zhang. 2023. A Learned Query Rewrite System. *Proceedings of the VLDB Endowment* 16, 12 (Aug. 2023), 4110–4113. https://doi.org/10.14778/3611540.3611633

# A Glossary of Frequently-Used Terms and Notations in the Paper

P1 R3.R3

Table 4 gives a glossary of frequently-used terms and notations in the paper, each with a brief intuitive explanation.

| Term / Notation | Explanation |
|---|---|
| $f(S)$ on a set $S$ of elements | Set of all $f(a)$ for all elements $a$ in $S$ |
| $\cup S$ on a set $S$ of sets | Union of all sets in $S$ |
| Conjunctive query | A query with joins, with selection and projection involving only attributes in base relations. |
| Full (conjunctive) query | Conjunctive query where no attribute is projected out. |
| Boolean (conjunctive) query | Conjunctive query whose output contains no attribute, but only a Boolean value indicating whether there exists any tuple satisfying all predicates. |
| Relation-dominated query | Conjunctive query in which all output attributes are present in some single relation |
| Hypergraph $H(Q)$ associated with query $Q$ | Hypergraph where each vertex is an attribute, and each hyperedge corresponds to a relation, containing all attributes in the relation. |
| $\lambda$-label $\lambda(p)$ | In a hypertree decomposition or meta-decomposition, maps each tree vertex $p$ to a set of hyperedges (relations). Non-empty in hypertree decompositions, possibly empty in minor nodes of meta-decompositions. |
| $\chi$-label $\chi(p)$ | In a hypertree decomposition or meta-decomposition, maps each tree vertex $p$ to a set of hypergraph vertices (attributes). Subset of vertices in the $\lambda$-label |
| $\kappa$-label $\kappa(p)$ | In a meta-decomposition, maps each tree vertex to the "interface", i.e., hypergraph vertices in the intersection between $p$ and other tree vertices not on the subtree rooted at $p$. |
| $T_p$, subtree of $T$ rooted at $p$ | Subtree of $T$ containing $p$ and its descendants |
| $T_{p \to q}$ | If $q$ is a child of $p$, same as $T_q$. If $q$ is a parent of $p$, the subtree of $T$ with all vertices except those in $T_p$ |
| Fan-out $f(p)$, $f(T)$ | For a tree vertex $p$, the number of children. For a tree $T$, the maximum number of children per vertex. |
| Induced query $Q(p)$ | For vertices $p$ on a join tree or query plan, the join of all relations on the subtree rooted at $p$, with appropriate selection and projection applied. |
| Interface $I(p)$ | For a vertex $p$ on a meta-decomposition or query plan, the intersection (of vertices) between $p$ and other vertices not in the subtree rooted at $p$. |
| Width $w(p)$, $w(\mathcal{P})$ of (vertex on) query plan | For a vertex $p$ on a query plan, the number of relations needed to cover the attributes in the interface. For an entire query plan $\mathcal{P}$, the maximum number of relations needed to cover the attributes in the interface of any intermediate step. |
| Width $w(p)$, $w(\mathcal{T})$ of (vertex on) hypertree decomposition | For a vertex $p$ on a hypertree decomposition, the number of hyperedges / relations in the $\lambda$-label. For a hypertree decomposition $\mathcal{T}$, the maximum number of hyperedges / relations in the $\lambda$-label of a single vertex. |
| Ear, reducible hyperedge/relation | In GYO reduction on an acyclic hypergraph/query, a hyperedge/relation whose intersection (hypergraph vertices / query attributes) with the rest of the hyperedges/relations is covered by some other single hyperedge/relation. |

**Table 4: Glossary of frequently-used terms and notations in the paper**

# B Missing Materials for Section 1

## B.1 Number of Acyclic Queries in Major Benchmarks

Table 5 shows the numbers and proportions of acyclic queries in major benchmarks. One can clearly see that they indeed account for a majority.

| Benchmark | # Queries | # Acyclic | % Acyclic |
|---|---|---|---|
| **TPC-H** [44] | 22 | 21 | 95.45 % |
| **JOB** [32] | 113 | 113 | 100 % |
| **STATS-CEB** [26] | 2603 | 2603 | 100 % |
| **Spider-NLP** [54] | 4712 | 4709 | 99.94 % |
| **DSB** [16] (SPJ queries only) | 300 | 240 | 80 % |
| **Musicbrainz** [34] | 375 | 243 | 64.8 % |
| **Total** | 8125 | 7929 | **97.59 %** |

**Table 5: Number of acyclic queries in major benchmarks**

# C Missing Proofs in Section 3

## C.1 Proof of Theorem 3.2

Let $p^* \in V(\mathcal{P})$ be a node in the plan where the maximum width is achieved, i.e., $w(p^*) = w(\mathcal{P})$. By the definition of $w(p^*)$, there must exist a set of relations $S \subseteq E(H_{p^*})$ such that $|S| = w(p^*)$ and $I(p^*) \subseteq \bigcup_{e \in S} e$. Therefore, the result of $Q(p^*)$ must be a subset of the join of all relations in $S$. In the worst-case scenario, where all relations in $S$ share no common attributes and their join is a Cartesian product, the size of this join is $O(N^{|S|})$, which gives an upper bound of $O(N^{w(\mathcal{P})})$.

## C.2 Proof of Theorem 3.5

When $Q(p)$ only involves one relation, $Q(p) = q$ holds for all join trees because of the coverage condition of a join tree. Let $T$ be the join tree satisfies the condition, for any $q \in V(T)$, the interface $I_q$ between the induced hypergraph $H_q$ for subtree $T_q$ rooted by $q$ and the residual hypergraph $\overline{H_q}$ satisfies $I_q \subseteq q$, because of the connectedness condition of the join tree. In addition, $Q(p) = Q(q)$ implies the induced hypergraph $H_p = H_q$, therefore, for every $p \in \mathcal{P}$, we can find a set $S = \{q\}$ that covers $I_p$, and $w(p) = 1$ holds for $p$, which completes the proof of if-direction.

For the only-if direction, given a width-1 query plan $P$, we can construct such a join tree $T$ with induction. For any join operation $p \in V(\mathcal{P})$. Assuming there exists a valid join tree for both of the two child nodes $c_1, c_2$, denoted $T_{c_1}, T_{c_2}$. Let $q_1 \in V(T_{c_1})$ be the relation that covers the interface $I_{c_1}$ and $q_2 \in V(T_{c_2})$ covers $I_{c_2}$, we make $q_1$ be the root of $T_{c_1}$ and $q_2$ be the root of $T_{c_2}$, then $I_{c_1} \cap I_{c_2} = q_1 \cap q_2$, and we can merge $T_1$ and $T_2$ to obtain $T'$ by connecting $q_1$ and $q_2$. In addition, there exists $q \in V(T_{c_1}) \cup V(T_{c_2})$ since $w(p) = 1$, such tree $T'$ is a valid join tree for node $p$. Combining with the base case where $T$ contains only one node, we then complete the proof.

## C.3 Proof of Theorem 3.8

Following Theorem 3.5, we have

Lemma C.1. *For all width-1 query plans $\mathcal{P}$ induced by a join tree $T$, for all non-leaf nodes $t \in V(T)$ with distinct children $c_1$ and $c_2$, if $p_1, p_2 \in V(\mathcal{P})$ are the nodes on $\mathcal{P}$ such that $Q(p_1) = Q(c_1)$ and $Q(p_2) = Q(c_2)$, then there exists no $q \in \mathcal{P}$ such that $Q(q) = Q(p_1) \bowtie Q(p_2) = Q(c_1) \bowtie Q(c_2)$.*

Proof. Suppose there is such a $q$, then by Definition 3.4, there would have been an edge between $c_1$ and $c_2$ on $T$, contradicting with the assumption that $c_1$ and $c_2$ are distinct children of some node $t$. □

This means that, given a width-1 query plan $\mathcal{P}$ induced by a join tree $T$, for each $t \in V(T)$, with children $c_1, \ldots, c_{f(t)}$ and the corresponding $p \in V(\mathcal{P})$ such that $Q(t) = Q(p)$, if we consider the sub-query-plans $\mathcal{P}_{c_1}, \ldots, \mathcal{P}_{c_{f(t)}}$, induced by the sub-join-trees $T_{c_1}, \ldots, T_{c_{f(t)}}$ rooted at the children of $t$, each as a single relation, then the sub-query-plan $\mathcal{P}_p$ rooted at $p$ is a *left-deep plan* consisting of $t, \mathcal{P}_{c_1}, \ldots, \mathcal{P}_{c_{f(t)}}$, and $t$ is an operand of the deepest join operation.

This can be done by a dynamic programming algorithm, as shown in Algorithm 4. In the algorithm, $\mathsf{children}(v)$ is the set of all children of $v$.

In the algorithm, the outer loop (Line 1) is executed $|V(T)| = |Q|$ times. The complexity of the loop body is dominated by enumerating all possible $T$ (Line 13) and $u \in T$ (Line 14). Noting that the number of children of $v$ is bounded by the fan-out $f(p)$, the complexity of this part is given by

**Algorithm 4:** Finding the optimal width-1 query plan induced by a join tree of an acyclic query

**input** : A join tree $T$
**output** : The optimal width-1 query plan induced by $T$

```
foreach v ∈ V(T), in bottom-up order do
    foreach child c of v do
        if λ(c) ≠ ∅ then
            P(v, {c}) ← P(c, children(c) ∪ {c})
        else
            P(v, {c}) ← P(c, children(c))
    if λ(v) ≠ ∅ then
        P(v, {v}) ← query plan with one node v
    if λ(v) = ∅ then
        S ← 2^children(v) \ ∅
    else
        S ← {T ∪ {v} | T ∈ 2^children(v) \ ∅}
    foreach T ∈ S, ordered by increasing size do
        P(v, T) ← argmin_{p ∈ {P(v, T\{u}) ⋈ P(v, {u}) | u ∈ T}} cost(p)
return P(root(T), S(root(T)))
```

$$
\begin{aligned}
\sum_{i=1}^{f(v)} \binom{f(v)}{i} \times i &= \sum_{i=1}^{f(v)} \frac{f(v)!}{i!(f(v)-i)!} \times i \\
&= \sum_{i=1}^{f(v)} \frac{f(v)!}{(i-1)!(f(v)-i)!} \\
&= \sum_{i=1}^{f(v)} f(v) \times \frac{(f(v)-1)!}{(i-1)!((f(v)-1)-(i-1))!} \\
&= \sum_{i=1}^{f(v)} f(v) \times \binom{f(v)-1}{i-1} \\
&= f(v) \sum_{i=0}^{f(v)-1} \binom{f(v)-1}{i} \\
&= f(v) 2^{f(v)-1} \\
&\leq \max_{v \in V(T)} f(v) 2^{f(v)-1} = f(T) 2^{f(T)-1}.
\end{aligned}
$$

So, overall, the complexity is $O(f(T)2^{f(T)-1}|Q|)$.

# D Missing Materials in Section 4

## D.1 Missing Algorithms that Algorithm 2 is Based on

*D.1.1 Enumeration of Non-Homomorphic Trees Based on Prüfer Sequences.* For minor nodes $p$, as is illustrated by Example 2.3, all of its neighbours $q$ for which $\kappa(q) = \chi(p)$ could be connected as an arbitrary tree with these neighbours being vertices. In order to enumerate all such possibilities without repetition, we (1) enumerate all non-homomorphic trees up to rerooting and (2) enumerate all rerootings for each tree enumerated in step (1). We start with the enumeration of non-homomorphic trees up to rerooting—namely, if two trees are simply rerootings of each other, they are treated as the same and only one of them will be enumerated. This can be done based on Prüfer sequences [38]. A Prüfer sequence is a sequence of $|V| - 2$ numbers, each of which is an integer between 1 and $|V|$, inclusive. It can be shown that each sequence bijectively encodes one tree (up to rerooting) of size $|V|$, with vertices labelled by integers ranging from 1 to $|V|$, inclusive. Therefore, we use the algorithm Algorithm 5 which simply enumerates all such numeric sequences and converts each sequence into a tree.

**Algorithm 5:** enumerateTrees($V$): Enumerate all non-homomorphic trees, up to rerooting, with vertices $V$, using Prüfer sequences

```
input  : Set of vertices V
output : A set 𝒯 of all non-isomorphic trees with vertices V
Arbitrarily order the vertices in V and let v_i be the i-th vertex in V under this ordering
𝒯 ← ∅
foreach sequence P = p_1, ..., p_{|V|-2} of |V| - 2 integers, each with value in range [1, |V|] do
    foreach i = 1, ..., |V| do
        d(i) ← 1
    S ← [1, |V|]                    // maintains the set of indices i with d(v_i) = 1, in increasing order
    foreach i = 1, ..., |V| - 2 do
        d(s_i) ← d(p_i) + 1
        S ← S \ {i}
    Initialize a tree T with set of vertices V and no edge
    foreach i = 1, ..., |V| - 2 do
        j ← remove the least element in S
        d(i) ← d(i) - 1
        d(j) ← d(j) - 1
        if d(i) = 1 then
            Add i into S
        In T, add v_j as a child of v_i
    i, j ← the remaining two elements in S
    In T, add v_j as a child of v_i          // the direction does not matter
    𝒯 ← 𝒯 ∪ {T}
return 𝒯
```

We then recursively enumerate all possible rerootings of a tree $T$ such that the new root $r'$ has $\kappa \subseteq \chi(r')$, where $\kappa$ is the set of keys as required by the original algorithm Algorithm 2. This can be done as shown in Algorithm 6.

*D.1.2 Enumeration of Rerootings.* Algorithm 6 recursively enumerates all rerootings of a tree $T$ such that the new root $r'$ has $\kappa \subseteq \chi(r')$, where $\kappa$ is the set of keys as required by Algorithm 2.

**Algorithm 6:** rerootings($T, p, \kappa$): Enumerate all valid rerootings of a tree $T$ such that the new root $r'$ has $\kappa \subseteq \chi(r')$

```
input  : A tree T, optionally the previous parent p of the root r of T before the previous rotation, the expected key κ
output : A set 𝒯 of all valid rerootings of T such that the new root r' has κ ⊆ χ(r')
𝒯 ← ∅
foreach child c of the root r of T do
    if κ ⊆ χ(c) and c ≠ p then
        T' ← T rerooted to c
        𝒯 ← 𝒯 ∪ {T'} ∪ rerootings(T', r, κ)
return 𝒯
```

## D.2 Proof of Lemma 4.7

We start with the following lemma:

Lemma D.1. *Given an acyclic hypergraph $H$, if there exist two ears $e_1, e_2 \in E(H)$ such that $o(e_1, H) = o(e_2, H)$, then $o(e_1, H) = o(e_2, H) = e_1 \cap e_2$.*

Proof. ($\subseteq$): $o(e_1, H) = e_1 \cap (\cup(E(H) \setminus \{e_1\})) \subseteq e_1$, and, similarly, $o(e_2, H) \subseteq e_2$. So it follows that $o(e_1, H) = o(e_2, H) \subseteq e_1 \cap e_2$.
($\supseteq$): Since $e_1 \in E(H) \setminus \{e_2\}$, we have $o(e_1, H) = o(e_2, H) = e_2 \cap (\cup(E(H) \setminus \{e_2\})) \supseteq e_1 \cap e_2$. □

A hypergraph is acyclic if and only if it has a join tree, satisfying conditions ($H_1$)–($H_3$). Since $e_1$ and $e_2$ are ears in $H$ such that $o(e_1, H) = o(e_2, H) \subseteq e_2$, in the GYO algorithm for $H$, $e_1$ can already be removed, with $e_2$ being its witness. Therefore, there exists a join tree $T$ of $H$ where there is a leaf node $p_1$ with $\lambda(p_1) = \{e_1\}$, and its parent $p_2$ has $\lambda(p_2) = \{e_2\}$. We construct a tree $T'$ that is exactly the same as $T$ except that $p_1$ and $p_2$ are merged into a single node $m$ with $\lambda(m) = \{o(e_1, H)\}$ and $\chi(m) = o(e_1, H)$, and we show that $T'$ is a valid join tree of $H'$. ($H_1$) and ($H_3$) are satisfied by construction. It remains to show that $T'$ satisfies ($H_2$). Suppose by way of contradiction that this is not the case. By ($H_2$) on the join tree $T$, this means, on $T'$, either (1) there exist $s, t \in V(T')$ such that $m$ is on the path between $s$ and $t$, and $\chi(s) \cap \chi(t) \nsubseteq \chi(m)$, or (2) there exists $s \in V(T')$ and $t$ on the path between $s$ and $m$, such that $\chi(s) \cap \chi(m) \nsubseteq \chi(t)$.

For case (1), since, by Lemma D.1, $\chi(m) = o(e_1, H) = e_1 \cap e_2 = \chi(p_1) \cap \chi(p_2)$, if $\chi(s) \cap \chi(t) \nsubseteq \chi(m)$, this means either $\chi(s) \cap \chi(t) \nsubseteq \chi(p_1)$ or $\chi(p) \cap \chi(q) \nsubseteq \chi(p_2)$. On $T$, since $p_1$ is a leaf node, $p_2$ should be on the path between $s$ and $t$. By ($H_2$) for $T$, we have $\chi(s) \cap \chi(t) \subseteq \chi(p_2)$. So it can only be that $\chi(s) \cap \chi(t) \nsubseteq \chi(p_1)$, i.e., there exists some $v \in \chi(s) \cap \chi(t) \subseteq \chi(p_2)$ such that $v \notin \chi(p_1)$. But then, $v \in o(e_2, H)$ but $v \notin o(e_1, H)$, contradicting $o(e_1, H) = o(e_2, H)$.

For case (2), we first note that $\chi(s) \cap o(e_1, H) = \chi(s) \cap e_1$. This is because ($\subseteq$) $o(e_1, H) = e_1 \cap (\cup(E(H) \setminus \{e_1\})) \subseteq e_1$, and ($\supseteq$) $\chi(s) \cap e_1 \subseteq (\cup(E(H) \setminus \{e_1\})) \cap e_1 = o(e_1, H)$. Then, $\chi(s) \cap \chi(m) = \chi(s) \cap o(e_1, H) = \chi(s) \cap e_1 = \chi(s) \cap \chi(p_1)$. So, $\chi(s) \cap \chi(p_1) = \chi(s) \cap \chi(m) \nsubseteq \chi(p_2)$. Since $p_1$ is a leaf and $p_2$ is the parent of $p_1$ on $T$, $p_2$ must be on the path between $s$ and $p_1$, so this contradicts ($H_2$) for $T$.

## D.3 Proof of Theorem 4.8

We start by considering the while-loop (Line 3). At the start of each iteration, an ear must exist because the hypergraph $H'$ must be acyclic, as guaranteed by Lemma 4.7. Furthermore, $E(H')$ always decreases in size by at least 1 in each iteration, because we only remove ears from $E(H')$, except when we have $e_1, e_2 \in S$ such that $o(e_1, H') = o(e_2, H')$, in which case we remove all (at least 2) those ears sharing the same intersection but add only one special hyperedge $e_m$ to $E(H')$. Therefore, the body of the while loop is executed at most $|E(H)|$ times.

For each iteration of the while loop, all sets $S$ (Line 4) can be enumerated in $O(|E(H')|)$, if we maintain two hash maps: (1) $T_1$, which maps each $o \subseteq V(H')$ to all hyperedges $e \in E(H')$ such that $o(e, H') = o$, and (2) $T_2$, which maps each $v \in V(H')$ to the number of hyperedges $e \in E(H')$ such that $v \in e$. Each time a hyperedge $e$ is added or removed from $E(H')$, we update $T_2$ for each $v \in e$, and then update $T_1$ if some $v \in e$ is/was covered by exactly one hyperedge. These updates can be done in $O(|E(H')|)$, which is $O(|E(H)|)$.

With these observations, the runtime $O(|E(H)|^3)$ can be obtained by following the control flow.

## D.4 Proof of Theorem 4.9

We show that $M$ output by the algorithm satisfies ($H_1$)–($H_5$).

*($H_1$).* For each $e \in E(H)$, the algorithm has to remove $e$ from $E(H')$ on Line 10, Line 15, or Line 21 before it terminates. In any of these cases, the algorithm will create a node $p$ in $M$ with $\lambda(p) = \{e\}$, on Line 9, Line 14, or Line 20.

Before proceeding with ($H_2$), we have the following lemmas:

Lemma D.2. *For each edge $(q, p) \in E(M)$, where $\chi(p) = \{e_p\}$ and $\chi(q) = \{e_q\}$, we have either*

*(1) $e_p$ is removed as ear by the algorithm before $e_q$ is,* or
*(2) $\lambda(q) = \emptyset$ and $\kappa(q) = \chi(q)$*

*Additionally, $\kappa(p) = \chi(p) \cap \chi(q)$.*

Proof. We first have $\kappa(p) \subseteq \chi(p)$ by definition on Lines 9, 14, 18, and 20. Since the algorithm adds the edge $(q, p)$, by Lines 24 and 27, we have $\kappa(p) \subseteq \chi(q)$. Therefore, $\kappa(p) \subseteq \chi(p) \cap \chi(q)$.

We then consider the relative order in which $e_p$ and $e_q$ are removed from $E(H')$ in the algorithm.

(1) If $e_p$ is removed before $e_q$ is, then at the time $e_p$ is removed, $e_q \in E(H')$, and therefore $\chi(p) \cap \chi(q) = e_p \cap e_q \subseteq e_p \cap (\cup(E(H') \setminus \{e_p\})) = \kappa(p)$. Since also $\kappa(p) \subseteq \chi(p) \cap \chi(q)$, we have $\kappa(p) = \chi(p) \cap \chi(q)$.

(2) If $e_p$ is removed after $e_q$ is, then, symmetric to case (1), we have $\chi(p) \cap \chi(q) \subseteq \kappa(q)$. Then, since we have shown that $\kappa(p) \subseteq \chi(p) \cap \chi(q)$, we have $\kappa(p) \subseteq \chi(p) \cap \chi(q) \subseteq \kappa(q)$. By Line 24 and Line 27, this is only possible if $\lambda(q) = \emptyset$ and $\kappa(p) = \chi(q)$. Since $\kappa(p) \subseteq \chi(p)$ and $\kappa(p) = \chi(q)$, it is true that $\kappa(p) = \chi(p) \cap \chi(q)$. □

It follows that

Corollary D.3. *$p$ is a descendant of $q$ on $M$ if and only if $e_p$ is removed as ear by the algorithm before $e_q$ is.*

*($H_2$).* The algorithm iteratively adds edges $(q, p)$ to attach a connected subtree $M_p$ rooted at $p$ to a connected subtree $M^q$ that contains $q$. Suppose by way of contradiction that ($H_2$) is violated at the end, and suppose it is when processing $p \in V(M)$, that the addition of edge $(q, p)$ violates this condition for the first time. Namely, $M_p$ and $M^q$ each satisfies ($H_2$), but the resulting $M$ violates ($H_2$), i.e., for some $s, t \in V(M)$, there exists $u$ on the path betwween $s$ and $t$ such that $\chi(s) \cap \chi(t) \nsubseteq \chi(u)$. Without loss of generality, we take the shortest possible path from $s$ to $t$ where the condition is violated, where $u$ is the node immediately before $t$ on the path. Then, $\kappa(t) = \chi(t) \cap \chi(u)$.

Since $M_p$ and $M^q$ each satisfies ($H_2$), the only way it can be violated is if $s \in V(M_p)$, i.e., $s$ is a descendent of $p$, $t \in M^q$, and thus $p$ and $q$ are on the path between $s$ and $t$. For the purpose of the proof, we think of the subtree $M_p$ as being added to the subtree $M^q$ one node at a time, in a top-down order, starting from the root $p$. We then show that each time a node is added, the connectedness condition is preserved. Let $\lambda(p) = \{e_p\}$ and $\lambda(t) = \{e_t\}$.

As the base case, when $p$ is added as a child of $q$, we consider two cases on the relative order in which $e_p$ and $e_t$ are removed from $E(H')$ by the algorithm.

(1) If $e_p$ is removed as the ear by the algorithm before $e_t$ is, then $\chi(p) \cap \chi(t) = e_p \cap e_t \subseteq \kappa(p) \subseteq \chi(q)$. And, for all $u$ between $q$ and $t$ on the path between $p$ and $t$, we have $\chi(p) \cap \chi(t) = \chi(p) \cap \chi(t) \cap \chi(t) \subseteq \chi(q) \cap \chi(t) \subseteq \chi(u)$, by the connectedness condition on $M^q$.

(2) If $e_t$ is removed as the ear by the algorithm before $e_p$ is, then $\chi(p) \cap \chi(t) = e_p \cap e_t \subseteq \kappa(t)$. By Corollary D.3, $t$ is a descendant of $q$, and so the parent $u$ of $t$ is on the path between $p$ and $t$. Then, $\chi(p) \cap \chi(t) \subseteq \kappa(t) = \chi(t) \cap \chi(u) \subseteq \chi(u)$

As the induction step, assuming that the condition holds for the connected subtree with nodes in $M^q$ and all $p'$ that are added before $s$. Following a similar argument as for the base case, substituting $q$ with the parent of $s$, we can also derive that $\chi(s) \cap \chi(t) \subseteq \chi(u)$ for all $u$.

*($H_3'$).* This is guaranteed by construction on Lines 9, 14, 18, and 20.

*($H_4$)(a).* By Lemma D.2, for each $p \in V(M)$ and its parent $q$, we have $\kappa(p) = \chi(p) \cap \chi(q)$. ($\subseteq$): $\kappa(p) = \chi(p) \cap \chi(q) \subseteq \chi(q)$. ($\supseteq$): For all $s \in V(M) \setminus V(M_p)$, since $q$ is the parent of $p$, $q$ has to be on the path between $p$ and $s$. Then, by the connectedness condition ($H_2$) on $M$, we have $\chi(p) \cap \chi(s) \subseteq \chi(q)$. Therefore, $\kappa(p) = \chi(p) \cap \chi(q) \supseteq \chi(p) \cap \left(\bigcup_{s \in V(M) \setminus V(M_p)} \chi(s)\right) = \chi(p) \cap \chi(V(M) \setminus V(M_p))$.

*($H_4$)(b).* This is guaranteed by Lines 24 and 27.

*($H_5$).* Lines 4–11 and Lines 17–18 create such minor nodes when necessary. Lines 6–7 and the condition on Line 17 before creating a minor node on Line 18 ensures there are no duplicate minor nodes.

## D.5 Proof of Theorem 4.10

Vertices $V(M)$ in $M$ are either physical nodes or minor nodes. Let $V^P(M)$ be the set of physical nodes and $V^M(M)$ be the set of minor nodes, such that $V(M) = V^P(M) \cup V^M(M)$. Each time we create a physical node $p$ with $\lambda(p) = \{e\}$ (Lines 9, 14, and 20), we remove $e$ from $E(H')$ (Lines 10, 15, and 21). So there are at most $|E(H)|$ physical nodes, $|V^P(M)| \leq |E(H)|$. Now let us consider the number of minor nodes, $|V^M(M)|$. Note that each minor node $m \in V^M(M)$ must have at least two children (see Line 4 and Line 17), i.e., the fanout $f(m) \geq 2$. Then, the total number of vertices

$$\begin{aligned} |V(M)| &= |V^P(M)| + |V^M(M)| \\ &= |E(M)| + 1 = 1 + \sum_{p \in V(M)} f(p) \\ &> 1 + \sum_{m \in V^M(M)} f(m) \geq 1 + \sum_{m \in V^M(M)} 2 = 2|V^M(M)| + 1. \end{aligned}$$

Then, $|V^P(M)| + |V^M(M)| > 2|V^M(M)| + 1$, so $|V^M(M)| < |V^P(M)| - 1 = |E(H)| - 1$. So, $|V(M)| = |V^P(M)| + |V^M(M)| < |E(H)| + |E(H)| - 1 = 2|E(H)| - 1$. And $|E(M)| = |V(M)| - 1 \leq 2|E(H)| - 2$.

## D.6 Proof of Theorem 4.12

We prove by showing that all trees $T$ output by Algorithm 2 satisfy ($H_1$), ($H_2$), and ($H_3$).

For ($H_1$), note that this condition must already hold for the meta-decomposition $M$. In Algorithm 2, all physical nodes, for which $\lambda(r) \neq \emptyset$, will be kept in $\mathcal{T}$, on Line 23. Minor nodes, for which $\lambda(r) = \emptyset$, will no longer exist in the output. If $\kappa(r) \neq \chi(r)$, trees in $\mathcal{T}_p$, and thus $\mathcal{T}$, will not contain minor nodes (Line 10) but only nodes in the subtrees rooted at the children of $r$ (Lines 11–20). If $\kappa(r) = \chi(r)$, the minor node $r$ on Line 8 will eventually be removed and replaced by $v$ on Line 33. Therefore, the trees output by Algorithm 2 contain only physical nodes. ($H_3$) then follows, by ($H_3'$) for $M$.

It now remains to show ($H_2$), the connectedness condition. We prove by structural induction on vertices $v \in V(M)$, i.e., for each $v \in V(M)$, all trees $T$ returned by the function enumRec satisfy ($H_2$). In the base case, for all leaf nodes $v$, $v$ has to be physical, and the function simply returns a tree with one single node, so these conditions hold. For non-leaf nodes $v$, we assume as induction hypothesis that for all children $c$ of $v$, all trees returned by the function call enumRec($c$) satisfy ($H_2$). And we now show that it remains satisfied throughout the operations in enumRec($v$).

We start with the trees in $\mathcal{T}$ constructed on Lines 3–23. If $\lambda(v) \neq \emptyset$, this vacuously holds as there is only one tree with one vertex $r$ in $\mathcal{T}$ (Line 23). If $\lambda(v) = \emptyset$, we will show that it still holds for all $T'_c$ (constructed on Lines 11–20), assuming that it holds for $T_c \in \mathcal{T}_c$ from the previous iteration. It suffices to show that, for each $(T_P, c, T_c, d, T_d, u)$ processed in the loop, for all $p \in T_c$ and $q \in T_d$, all $s$ on the path between $p$ and $q$ has $\chi(p) \cap \chi(q) \subseteq \chi(s)$. Let the root of $T_c$ be $r_c$ and the root of $T_d$ be $r_d$. The path from $p$ to $q$ needs to go through $u$ then $r_d$. For $s$ on the path between $p$ and $u$, we have $\chi(p) \cap \chi(q) \subseteq \chi(v) = \kappa(d) \subseteq \chi(u)$. Since $T_c$ satisfies ($H_2$), we have $\chi(p) \cap \chi(q) \subseteq (\chi(p) \cap \chi(q)) \cap \chi(q) \subseteq \chi(u) \cap \chi(q) \subseteq \chi(s)$. For $s$ on the path between $r_d$ and $q$, we have $\chi(p) \cap \chi(q) \subseteq \chi(v) = \kappa(d) \subseteq \chi(r_d)$. Since $T_d$ satisfies ($H_2$), we have $\chi(p) \cap \chi(q) \subseteq (\chi(p) \cap \chi(q)) \cap \chi(q) \subseteq \chi(r_d) \cap \chi(q) \subseteq \chi(s)$.

We now continue to show that lines Line 25–Line 40 preserve the condition ($H_2$) for all $T \in \mathcal{T}$, by showing that the condition is preserved for all $T' \in \mathcal{T}'$ after each child $c \in X$ is processed, assuming that all the previous $T \in \mathcal{T}$ satisfy ($H_2$).

Let the root of $T_c$ be $r_c$. By the connectedness condition on $M$, we have $\chi(p) \cap \chi(q) \subseteq \chi(v)$, and $\chi(p) \cap \chi(q) \subseteq \chi(c)$. Since $c$ is a child of $v$, $\chi(p) \cap \chi(q) \subseteq \chi(v) \cap \chi(c) = \kappa(c)$.

(1) If $\lambda(c) \neq \emptyset$ or $\kappa(c) \neq \chi(c)$, $T_c$ is added directly as a child of $u$ on $T'$ (Line 38). For all $p \in V(T')$ and $q \in V(T_c)$, the path from $p$ to $q$ needs to go through $u$ then $r_c$. For $s$ on the path between $p$ and $u$, we have $\chi(p) \cap \chi(q) \subseteq \kappa(c) \subseteq \chi(u)$. Since $T'$ satisfies ($H_2$), we have $\chi(p) \cap \chi(q) \subseteq (\chi(p) \cap \chi(q)) \cap \chi(q) \subseteq \chi(u) \cap \chi(q) \subseteq \chi(s)$. For $s$ on the path between $r_c$ and $q$, we have $\chi(p) \cap \chi(q) \subseteq \kappa(c) \subseteq \chi(r_c)$. Since $T_c$ satisfies ($H_2$), we have $\chi(p) \cap \chi(q) \subseteq (\chi(p) \cap \chi(q)) \cap \chi(q) \subseteq \chi(r_c) \cap \chi(q) \subseteq \chi(s)$.

(2) If $\lambda(c) = \emptyset$ and $\kappa(c) \neq \chi(c)$, for each child $d$ of the root of $T_c$, we attach $T_d$ as a child of $u$. Let the root of $T_d$ be $r_d$. Then, $\kappa(d) \subseteq \chi(r_d)$ and $\kappa(d) \subseteq \chi(u)$. For all $p \in V(T')$ and $q \in V(T_d)$, the path from $p$ to $q$ needs to go through $u$ then $r_d$. For $s$ on the path between $p$ and $u$, we have $\chi(p) \cap \chi(q) \subseteq \kappa(c) = \kappa(d) \subseteq \chi(u)$. Since $T'$ satisfies ($H_2$), we have $\chi(p) \cap \chi(q) \subseteq (\chi(p) \cap \chi(q)) \cap \chi(q) \subseteq \chi(u) \cap \chi(q) \subseteq \chi(s)$. For $s$ on the path between $r_c$ and $q$, we have $\chi(p) \cap \chi(q) \subseteq \kappa(c) = \kappa(d) \subseteq \chi(r_d)$. Since $T_c$ satisfies ($H_2$), we have $\chi(p) \cap \chi(q) \subseteq (\chi(p) \cap \chi(q)) \cap \chi(q) \subseteq \chi(r_d) \cap \chi(q) \subseteq \chi(s)$.

## D.7 Proof of Theorem 4.13

We start from a few lemmas:

Lemma D.4. *For any non-root node $p$ on a meta-decomposition $M$ with parent $q$, $\kappa(p) = \chi(p) \cap \chi(q)$.*

Proof. **($\supseteq$)**: Since $q \in V(M) \setminus V(M_p)$, $\chi(p) \cap \chi(q) \subseteq \chi(p) \cap \chi(V(M) \setminus V(M_p)) = \kappa(p)$. **($\subseteq$)**: For any $v \in \kappa(p) = \chi(p) \cap \chi(V(M) \setminus V(M_p))$, i.e., any $v \in \chi(p) \cap \chi(s)$ for some $s \in V(M) \setminus V(M_p)$, the parent $q$ of $p$ lies on the path on $M$ between $p$ and $s$. Then, by the connectedness condition ($H_2$) for $M$, $v \in \chi(q)$. □

Lemma D.5. *For all $p \in V(M)$ with $\lambda(p) = \emptyset$ and $\kappa(p) = \chi(p)$, all children $c$ of $p$ satisfy $\kappa(c) = \chi(p)$.*

Proof. Suppose by way of contradiction that there exists such a $p \in V(T)$ with child $c$ such that $\kappa(c) \neq \chi(p)$. By Lemma D.4, we have $\kappa(c) = \chi(c) \cap \chi(p) \subseteq \chi(p)$. Since $\kappa(p) = \chi(p) \neq \emptyset$, $p$ is not the root of $M$ and thus has a parent $q$. Again by Lemma D.4, we have $\kappa(p) = \chi(p) \cap \chi(q) \subseteq \chi(q)$. But then, $\kappa(c) \subseteq \chi(p) = \kappa(p) \subseteq \chi(q)$, contradicting ($H_4$)(b) since $\kappa(c) \neq \chi(p)$ but $q \in V(T) \setminus V(T_p)$. □

Lemma D.6. *Given a meta-decomposition $M$, if some non-root node $p \in V(M)$ has $\lambda(p) = \emptyset$ and $\kappa(p) = \chi(p)$, then its parent $q$ has either $\lambda(q) \neq \emptyset$ or $\kappa(q) \neq \chi(q)$.*

Proof. Suppose by way of contradiction that, on a meta-decomposition $M$, there is $p \in V(M)$ such that $\lambda(p) = \emptyset$ and $\kappa(p) = \chi(p)$, and its parent $q$ has $\lambda(q) = \emptyset$ and $\kappa(q) = \chi(q)$. Then, $\kappa(p) \subseteq \chi(q) = \kappa(q)$. Since $\kappa(q) = \chi(q) \neq \emptyset$, $q$ has a parent as well. Let the parent of $q$ be $s$. Then, $\kappa(q) = \chi(q) \cap \chi(s) \subseteq \chi(s)$. So, $\kappa(p) \subseteq \kappa(q) \subseteq \chi(s)$. By ($H_5$), $\kappa(p) \neq \kappa(q) = \chi(q)$, contradicting ($H_4$)(b). □

Lemma D.7. *Given a meta-decomposition $M$ of an acyclic hypergraph $H$, for all $v \in V(M)$,*

*(1) if $\lambda(v) = \emptyset$ and $\kappa(v) = \chi(v)$, then, for each join tree $T$ of $H$, there exists some $S = \{p \in V(M) \mid \lambda(p) \neq \emptyset, \chi(v) \subseteq \chi(p)\}$ such that the set $V_v^P = \{p \in V(M_v) \mid \lambda(p) \neq \emptyset\} \cup S$ induces a connected subtree $T_v$ of $T$;*

*(2) if $\lambda(v) \neq \emptyset$ or $\kappa(v) \neq \chi(v)$, then, for each join tree $T$ of $H$, the set $V_v^P = \{p \in V(M_v) \mid \lambda(p) \neq \emptyset\}$, i.e., physical nodes on the subtree $M_v$ rooted at $v$, induces a connected subtree $T_v$ of $T$.*

Proof. We prove by structural induction on $M$. In the base case, for leaf nodes $v \in V(M)$, there is only one node on the subtree $M_v$, so the statement is vacuously true. For non-leaf nodes $v \in V(M)$, we assume as induction hypothesis that the statement is true for all children $c$ of $v$. Let $c$ and $d$ be the children of $v$ such that $p \in V(M_c)$ and $q \in V(M_d)$. We now consider two cases of $v$.

(1) If $\lambda(v) = \emptyset$ and $\kappa(v) = \chi(v)$, then, for any $s$ on the path between $p$ and $q$ on $T$ such that $s \notin M_v$, we have $\chi(p) \cap \chi(q) \subseteq \chi(s)$ by the connectedness condition on $T$. By (the contrapositive of) Lemma D.6, $\lambda(c) \neq \emptyset$ or $\kappa(c) \neq \chi(c)$, so by the inductive hypothesis, $V_c^P$ induces a connected subtree $T_c$ of $T$. Similarly, $V_d^P$ induces a connected subtree $T_d$ of $T$. If $c = d$, then $p$ and $q$ are in the same subtree rooted at the same child of $v$, and the statement would hold. If $c \neq d$, on the path from $p$ to $q$ on $T$, let the last node on $T_c$ be $x$ and the first node on $T_d$ be $y$. $s$ is between $x$ and $y$. Then, the path from $c$ and $d$ has to go through $x$ and $y$, and hence also $s$. Therefore, $\chi(c) \cap \chi(d) \subseteq \chi(s)$. By Lemma D.5, in this case, $\kappa(v) = \kappa(c) = \kappa(d) = \chi(c) \cap \chi(d) \subseteq \chi(s)$.

(2) If $\lambda(v) \neq \emptyset$ or $\kappa(v) \neq \chi(v)$, we would like to show that for any $s$ on the path between $p$ and $q$ on $T$, $s \in V(M_v)$. Suppose by way of contradiction that this is not the case, i.e., there exists some $s$ on the path between $p$ and $q$ on $T$, but $s \in V(M) \setminus V(M_v)$. By

the connectedness condition on $T$, we have $\chi(p) \cap \chi(q) \subseteq \chi(s)$. Furthermore, on $M$, $\chi(p) \cap \chi(s) \subseteq \chi(v)$ and $\chi(q) \cap \chi(s) \subseteq \chi(v)$, so $\chi(p) \cap \chi(q) = \chi(p) \cap \chi(q) \cap \chi(s) \subseteq \chi(v)$. Let the parent of $p$ on $M$ be $t$. Then, $\chi(t) \cap \chi(s) \subseteq \chi(v)$. Then, $\kappa(p) = \kappa(p) \cap \chi(s) = \chi(p) \cap \chi(t) \cap \chi(s) = (\chi(p) \cap \chi(s)) \cap (\chi(t) \cap \chi(s)) \subseteq \chi(v) \cap \chi(v) = \chi(v)$. But, by ($H_4$), this means either $t = v$ or $t$ is a minor node with $\kappa(t) = \chi(t)$. In either case, we have $\chi(t) \cap \chi(q) \subseteq \chi(v)$, because (i) if $t = v$, $\chi(t) \cap \chi(q) = \chi(v) \cap \chi(q) \subseteq \chi(v)$, and (ii) if $t$ is a minor node with $\kappa(t) = \chi(t)$, then $\chi(t) \cap \chi(q) = \kappa(t) \cap \chi(q) = \kappa(p) \cap \chi(q) \subseteq \chi(p) \cap \chi(q) \subseteq \chi(v)$. Also, we have $\kappa(t) = \chi(p) \cap \chi(t) \subseteq \chi(q)$, because either $t$ is on the path between $p$ and $q$, or $p$ is on the path between $t$ and $q$. Let the parent of $v$ be $u$. Then, we have $\kappa(t) = \kappa(t) \cap \chi(q) = \chi(t) \cap \chi(q) = (\chi(t) \cap \chi(q)) \cap (\kappa(p) \cap \chi(q)) \subseteq (\chi(t) \cap \chi(q)) \cap (\chi(p) \cap \chi(q)) \subseteq \chi(v) \cap \chi(s) \subseteq \chi(u)$. But, since it cannot be that $\kappa(v) = \chi(v)$ in this case, this violates ($H_4$), contradiction. □

Lemma D.8. *Given a meta-decomposition $M$ of hypergraph $H$, for all $v \in V(M)$, let $V_v^P = \{p \in V(M_v) \mid \lambda(p) \neq \emptyset\}$. Then,*

*(1) if $\lambda(v) = \emptyset$ and $\kappa(v) = \chi(v)$, then Algorithm 2 enumerates all join trees of the hypergraph $H_v$ with $E(H_v) = \lambda(V_v^P) \cup \{\chi(v)\}$ and $V(H_v) = \cup E(H_v)$;*

*(2) if $\lambda(v) \neq \emptyset$ or $\kappa(v) \neq \chi(v)$, then Algorithm 2 enumerates all join trees of the hypergrpah $H_v$ with $E(H_v) = \lambda(V_v^P)$ and $V(H_v) = \cup E(H_v)$.*

Proof. We will show that each join tree $T$ of $H$ can be enumerated by the algorithm based on the meta-decomposition $M$ of $H$. We prove by structural induction on vertices $v \in V(M)$. If $v$ is a leaf node, there is only one node in the subtree $M_v$, so this statement is vacuously true. If $v$ is a non-leaf node, consider the following cases.

(1) If $\lambda(v) = \emptyset$ and $\kappa(v) = \chi(v)$, by Lemma D.6, all children $c$ of $v$ have either $\lambda(c) \neq \emptyset$ or $\kappa(c) \neq \chi(c)$, so, by the induction hypothesis, Algorithm 2 enumerates all join trees of hypergraph $H_c$ with $E(H_c) = \bigcup_{p \in V_c^P} \lambda(p)$ and $V(H_c) = \bigcup_{e \in E(H_c)} e$. Also, note that it is possible to obtain a meta-decomposition $M'_v$ of the hypergraph $H_v$ from $M_v$ by simply replacing $v$ by a physical node $v'$ with $\lambda(v') = \{\chi(v)\}$, $\chi(v') = \chi(v)$, and $\kappa(v') = \emptyset$. By Lemma D.7, for each child $c$ of $v$, on any join tree $T_c$ of hypergraph $H_c$, $V_c^P$ induces a connected subtree $T_c$ of $T$. Furthermore, $T_c$ is a join tree of the meta-decomposition $M_c$, and can hence be enumerated on Line 6. So it remains to show that the algorithm enumerates all possible ways to combine these subtrees along with $v$. For each possible join tree $T_v$ of $H_v$, without loss of generality, assume that $T_v$ is rooted at $v'$. $T_v$ can be constructed from the algorithm as follows. We first construct a tree $T_P$, by a top-down traversal of $T_v$, where, for each node $p$, if (i) $\chi(v) \subseteq \chi(p)$, (ii) $p \in M_c$, i.e., in the subtree $M_c$ rooted at some child $c$ of $v$, and (iii) its parent $q$ is in a different subtree $M_d$ rooted at some child $d \neq c$ of $v$, then we add an edge from $d$ to $c$. If $p = v$, and its parent $q \in V(M_d)$ for some child $d$ of $v$, we add an edge from $d$ to $v$. If $p \in V(M_c)$ for some child $c$ of $v$, and its parent $q = v$, we add an edge from $v$ to $c$. Note that, for all pairs of $c$ and $d$ where we add edges, $\kappa(c) = \kappa(d) = \chi(v)$. Now, this tree $T_P$ is a spanning tree of the clique with vertices $C \cup \{v\}$, where $C$ is the set of all children of $v$, so it can be enumerated by the Prüfer sequence on Line 8. For each child $c$ of $v$, by Lemma D.7, $V_c^P$ induces a connected subtree $T_c$ of $T_v$. We traverse the vertices $c \in V(T_P)$ in bottom-up order. For each $c$ and each child $d$ of $c$ on $T_P$, let $r_d$ be the root of the connected subtrees $T_d$ of $T_v$ induced by $V_d^P$, and $u$ be the parent of $r_d$ on $T_v$. The algorithm will be able to reroot $T_d$ to $r_d$, find $u$ (since $\kappa(d) = \kappa(v) = \chi(v) = \chi(c) \subseteq \chi(u)$), and attach $T_d$ as a child of $u$. Finally, by Lemma D.5, in this case, $S = C$ and $X = \emptyset$, so the second part will not be executed at all.

(2) If $\lambda(v) = \emptyset$ and $\kappa(v) \neq \chi(v)$, starting from Lines 11–20, note that it is not possible to have a child $c \in C$ such that $\kappa(c) = \chi(c)$, because if that is the case, $\kappa(c) = \chi(c) = \chi(v)$, violating the uniqueness condition ($H_5$). Therefore, for all $c \in C$, we have $\lambda(c) \neq \emptyset$ or $\lambda(c) \neq \chi(c)$. Furthermore, we claim that the vertices $V_C^P = \bigcup_{c \in C} V_c^P$ induces a connected subtree $T_C$ on any join tree $T_v$ of $M_v$. This is because, if this is not the case, i.e., if there is some $s$ on the path between some $p \in V_c^P$ and $q \in V_d^P$, where $c$ and $d$ are children of $v$ with $\kappa(c) = \kappa(d) = \chi(v)$, such that $\kappa(s) \subsetneq \chi(v)$, then $\chi(p) \cap \chi(q) = \chi(v) \not\subseteq \kappa(s)$, violating the connectedness condition ($H_2$). Therefore, we can follow the same method as in case (1) to construct $T_P$ with vertices $C$ and show that, at the end of this part, it is possible to obtain a partial join tree $T$ with vertices in $V_C^P$.

Now consider all children $c \in X = \{c \in C \mid \kappa(c) \subsetneq \chi(v)\}$.

(i) If $\lambda(c) = \emptyset$ and $\kappa(c) = \chi(c)$, by Lemma D.6, all children $d$ of $c$ has either $\lambda(d) \neq \emptyset$ or $\kappa(d) \neq \chi(d)$, so, by Lemma D.7, $V_d^P$ induces a connected subtree in any join tree. We build a tree $T_c$ by (a) collecting all maximal connected subtrees of the join tree with nodes $p \in V_c^P$ and (b) adding a common parent $c$ as the root node. This is a valid join tree of the hypergraph $H_c$ with $E(H_c) = \left(\bigcup_{p \in V_c^P} \lambda(p)\right) \cup \{\chi(c)\}$ and $V(H_c) = \bigcup_{e \in E(H_c)} e$, and, by our inductive hypothesis, can be enumerated. For each child $d$ of $c$ on $T_c$, let the parent of $d$ on $T$ be $u$. The algorithm will be able to find such $u$ to attach $T_d$, because $\kappa(d) = \chi(d) \cap \chi(c) \subseteq \chi(u)$, by the connectedness condition on $T$.

(ii) If $\lambda(c) \neq \emptyset$ or $\kappa(c) \neq \chi(c)$, similarly to case (i), by Lemma D.7, $V_c^P$ induces a connected subtree on $T$, which, by our inductive hypothesis, can be enumerated. Then, the algorithm will be able to find such $u$ to attach $T_c$, because $\kappa(c) = \chi(c) \cap \chi(v) \subseteq \chi(u)$, by the connectedness condition on $T$.

(3) If $\lambda(v) \neq \emptyset$, we can follow a similar argument for the children $c \in X$ in case (2). □

Finally, based on Lemma D.8, taking $v = r$, the root of $M$ (for which $\kappa(r) = \emptyset \neq \chi(r)$), we finish the proof of completeness for the algorithm.

## D.8 Polynomial-Delay Algorithm to Enumerate All Join Trees Based on a Meta-Decomposition

Algorithm 7 shows an alternative implementation of Algorithm 2, which does not require all possible join trees induced by each children to have been enumerated before being able to proceed to construct join trees induced by the current node. Instead, Algorithm 7 uses callback functions (higher-order functions) as continuation after each join tree induced by a child is enumerated.

**Algorithm 7:** Enumerating all join trees based on a meta-decomposition, in polynomial delay

```
input  : A meta-decomposition M = (V(M), r, E(M), λ, χ, κ)
output : All possible non-homomorphic join trees up to rerooting
Function enumRec(v: a node on M, callback(T) : a callback function once enumRec obtains a tree T):
    C ← set of children c of v on M, sorted by partial order ⊇ on κ(c)
    if λ(v) = ∅ then
        S ← {c ∈ C | κ(c) = χ(v)}  // they can be found at the head of the sorted C
        X ← C \ S
        if κ(v) = χ(v) then
            T_v ← tree with one vertex v
            enumSpanningTrees(S ∪ {v}, X, callback)
        else
            enumSpanningTrees(S, X, callback)
    else
        T_v ← tree with one vertex v
        attachChildren(T_v, C, callback)
Function enumSpanningTrees(S: set of nodes, C: set of children, callback):
    foreach tree T_p with vertices V given by a Prüfer sequence do
        enumMinorOriginSubtrees(T_p, S, X, callback)
Function enumMinorOriginSubtrees(T: partial tree, S: remaining origins, X: remaining children, callback):
    if S = ∅ then
        attachChildren(T, X, callback)
    else
        c ← some node in S
        S' ← S \ {c}
        enumRec(c, T_c ⇒ enumOrigins(T with c replaced by T_c, S', X, callback))
Function attachChildren(T: partial tree, X: remaining children, callback):
    if X = ∅ then
        callback(T)  // We finish constructing a valid join tree. Now we call the callback function.
    else
        c ← some node in S
        S' ← S \ {c}
        if λ(c) = ∅ and κ(c) = χ(c) then
            enumRec(c, T_c ⇒ { foreach u ∈ V(T) such that κ(c) ⊆ χ(u) do attachChildren(T with u replaced by T_c, S', callback)}
        else
            foreach u ∈ V(T) such that κ(c) ⊆ χ(u) do
                enumRec(c, T_c ⇒ { foreach T' ∈ rerootings(T, κ(c), ∅) do attachChildren(T with c replaced by T', S', callback}
enumRec(r, T ⇒ { foreach T' ∈ rerootings(T, ∅, ∅), emit T'}
```

# E Missing Materials for Section 5

## E.1 Supporting Re-branching for Algorithm 3

For each node $p$, if there exists a subtree $T_q$ that can be attached as a child of $p$, we simply use an additional bit in the DP table (plan) to indicate whether $T_q$ is included. And we call optimizeLocal with or without the relations in $T_q$ accordingly. We note again that this is a very rare possibility in practical queries.

## E.2 Greedy Algorithm for Local Join Ordering

The implementation of local join ordering optimization using a greedy algorithm is as shown in Algorithm 8.

**Algorithm 8:** Greedy implementation of optimizeLocal

**input** : Node $q$ on meta representation, a set of query plans $\mathcal{P}$
**output** : A query plan that joins the relation in $q$ with all query plans in $\mathcal{P}$

**if** *$q$ is a minor node* **then** // $\lambda(q) = \emptyset$, there is no relation in $q$
  $P \leftarrow$ remove $P_1$ from $\mathcal{P}$ with the minimum result cardinality
**else**
  $P \leftarrow$ the relation in $\lambda(q)$
**while** $\mathcal{P} \neq \emptyset$ **do**
  $P_i \leftarrow$ remove $P_i$ from $\mathcal{P}$ that minimizes the cost to join $P$ with $P_i$
  $P \leftarrow$ the plan $P_i \bowtie P$
**return** $P$

# F Missing Materials for Section 6

## F.1 Optimization Time of metaDecomp and Other Methods for Different Query Sizes on Each Benchmark

Fig. 13 shows the detailed breakdown of optimization time measured for queries in each of the four benchmarks used in the paper. The trends that can be observed in each of the four benchmarks and the overall figure (Fig. 8) are mostly consistent, with the running time of DPconv starting to overtake that of metaDecomp starting from approximately 8 relations, and DPconv starting to timeout (taking more than 5 minutes) from approximately 25 relations.

P9
R1.O8
R2.O1
R2.O2

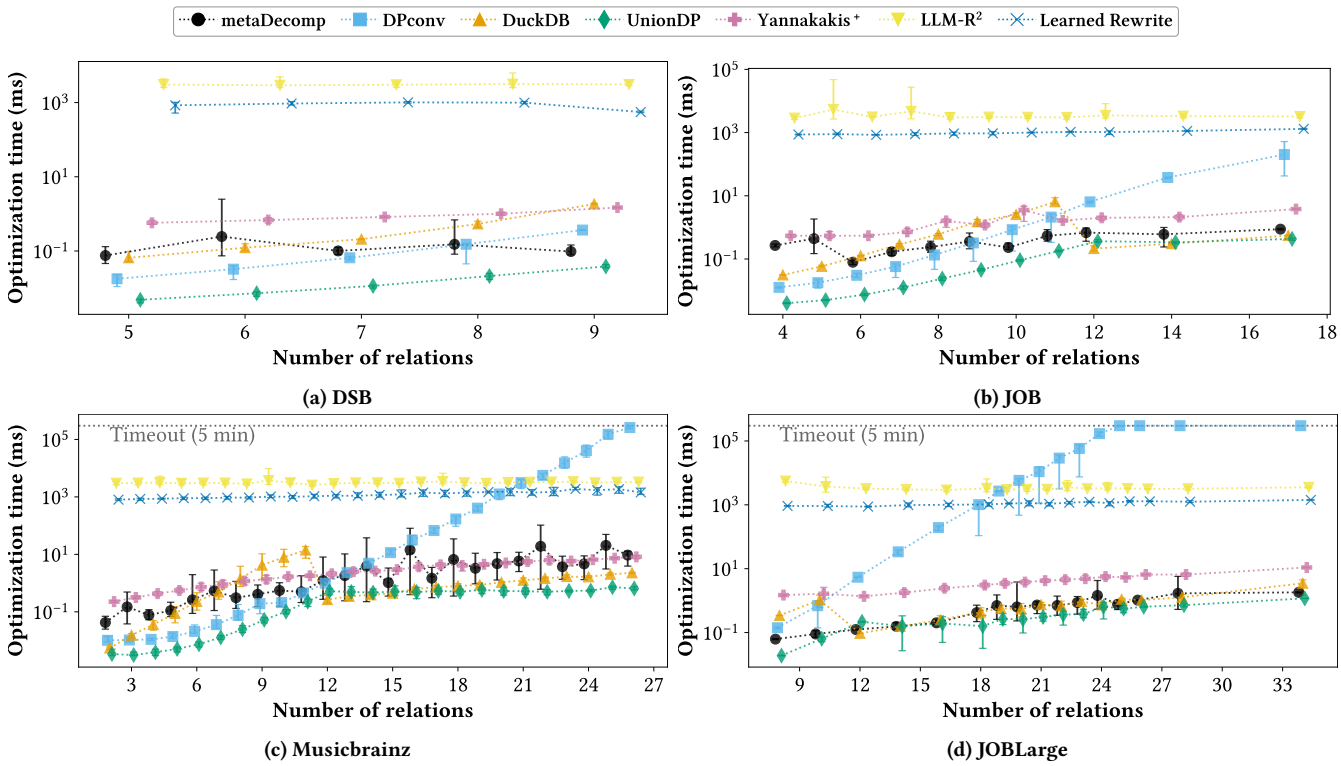
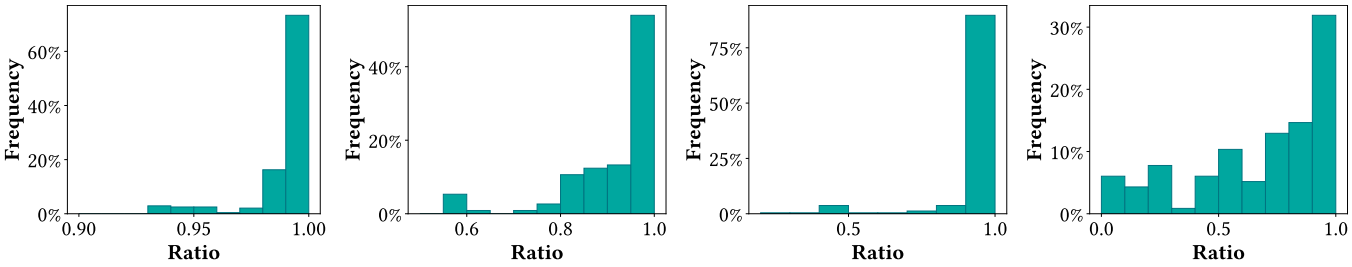



**Figure 13: Optimization time for queries in each benchmark. For each number of relations, the dot shows the average, and the whiskers show the minimum and maximum optimization time.**

## F.2 Comparison of the Globally Optimal Query Plans and the Optimal Width-1 Plans

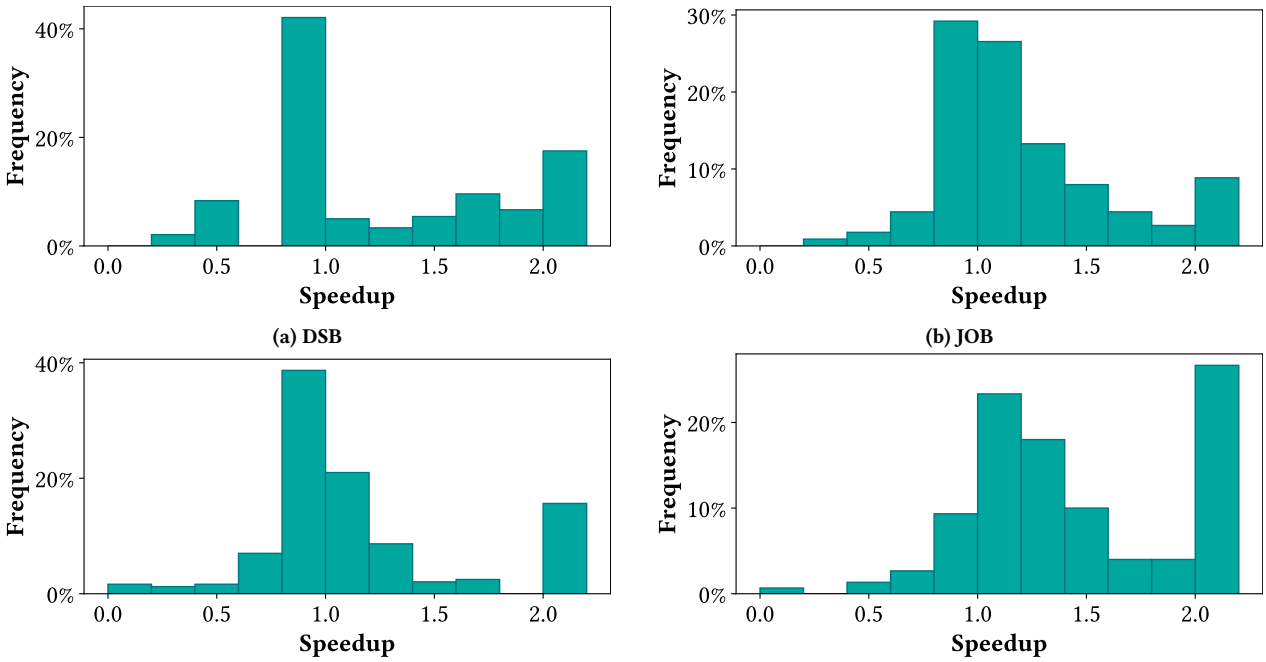
*F.2.1 Cost Values.* Fig. 14 shows histograms of the ratios between the costs of the globally optimal query plans over the costs of the optimal width-1 plans for queries in each benchmark. This measure is analogous to speedups. A ratio of 1.0 indicates that the optimal width-1 plan has the same cost as the globally optimal plan, and such cases are included in the bar to the left of 1. This measure is analogous to speedups, although the ratio should be at most 1 as the width-1 query plans are a restricted class. However, we can still observe that the optimal width-1 query plans tend to have costs very close to the global optimal, especially in the DSB and Musicbrainz benchmarks. We note also that the costs of query plans for the Musicbrainz and JOBLarge are derived using estimated cardinalities (instead of the exact cardinalities as for DSB and JOB). Therefore, for these two benchmarks, the ratios may not accurately reflect the ratios of the true costs.




(a) DSB (b) JOB (c) Musicbrainz (d) JOBLarge

**Figure 14: Ratios between the costs of the globally optimal query plans over the costs of the optimal width-1 plans for queries in each benchmark.**

*F.2.2 Execution Time.* Fig. 15 shows histograms of the speedups of the actual execution of the optimal width-1 query plans over that of the globally optimal plans. In most cases, we observe speedups close to 1, which indicates that the actual performance between the two indeed tends to be similar. Furthermore, although one should not expect speedups as the width-1 plans are being compared with the global optimal, we actually observe significant speedups, even many over 2, especially in the JOBLarge benchmark. This is thanks to the possibilities of early projection of irrelevant attributes made possible by enforcing the connectedness conditions in width-1 query plans.




(a) DSB (b) JOB (c) Musicbrainz (d) JOBLarge

**Figure 15: Speedups of execution time of the optimal width-1 plans over the globally optimal query plans for queries in each benchmark. Speedups over 2 are all included in the bar to the right of ratio 2.0.**

## F.3 Total Query Evaluation Time Using metaDecomp versus Using Other Methods on Each Benchmark

### *F.3.1 metaDecomp versus DPconv.*

*Histograms of speedup.* Fig. 16 shows histograms of the speedups of the overall query execution time using metaDecomp over using DPconv for queries in each of the four benchmarks. In DSB, JOB, and Musicbrainz, most of the queries show a speedup close to 1. In JOBLarge, however, we observe a large number of significant speedups of multiple orders of magnitudes, mostly due to the high optimization time of DPconv for the large query sizes in this benchmark.

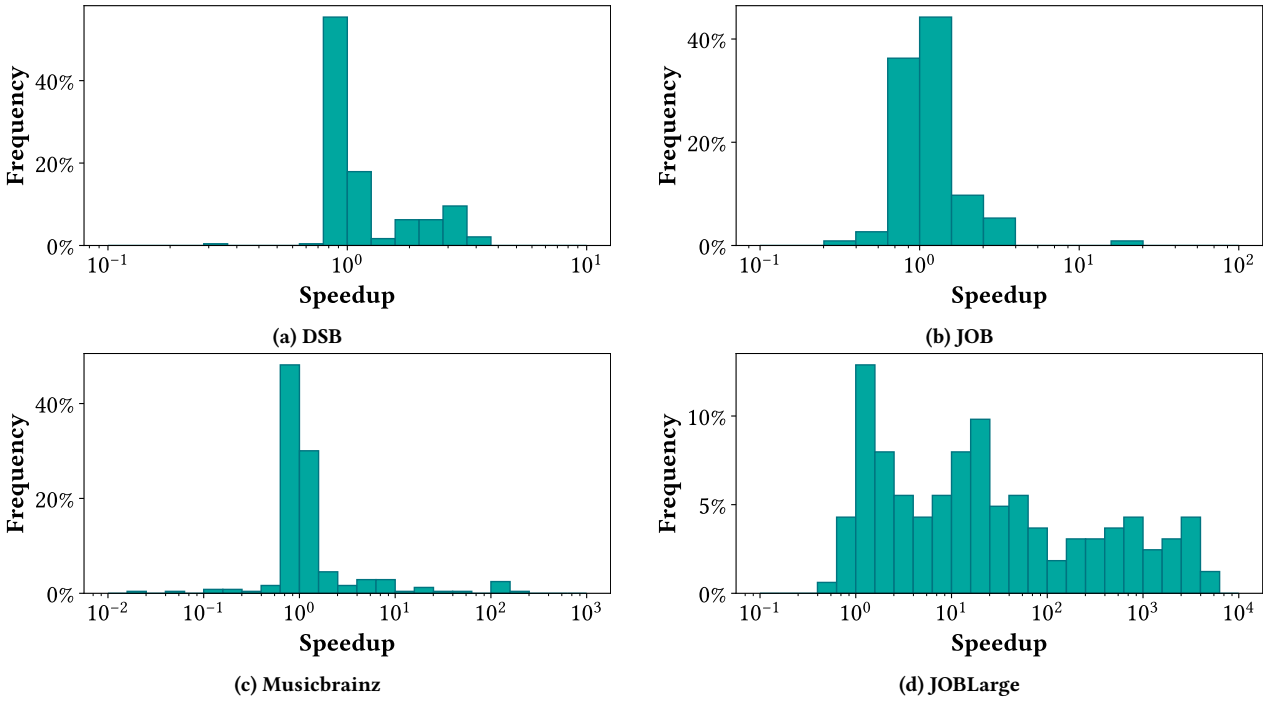
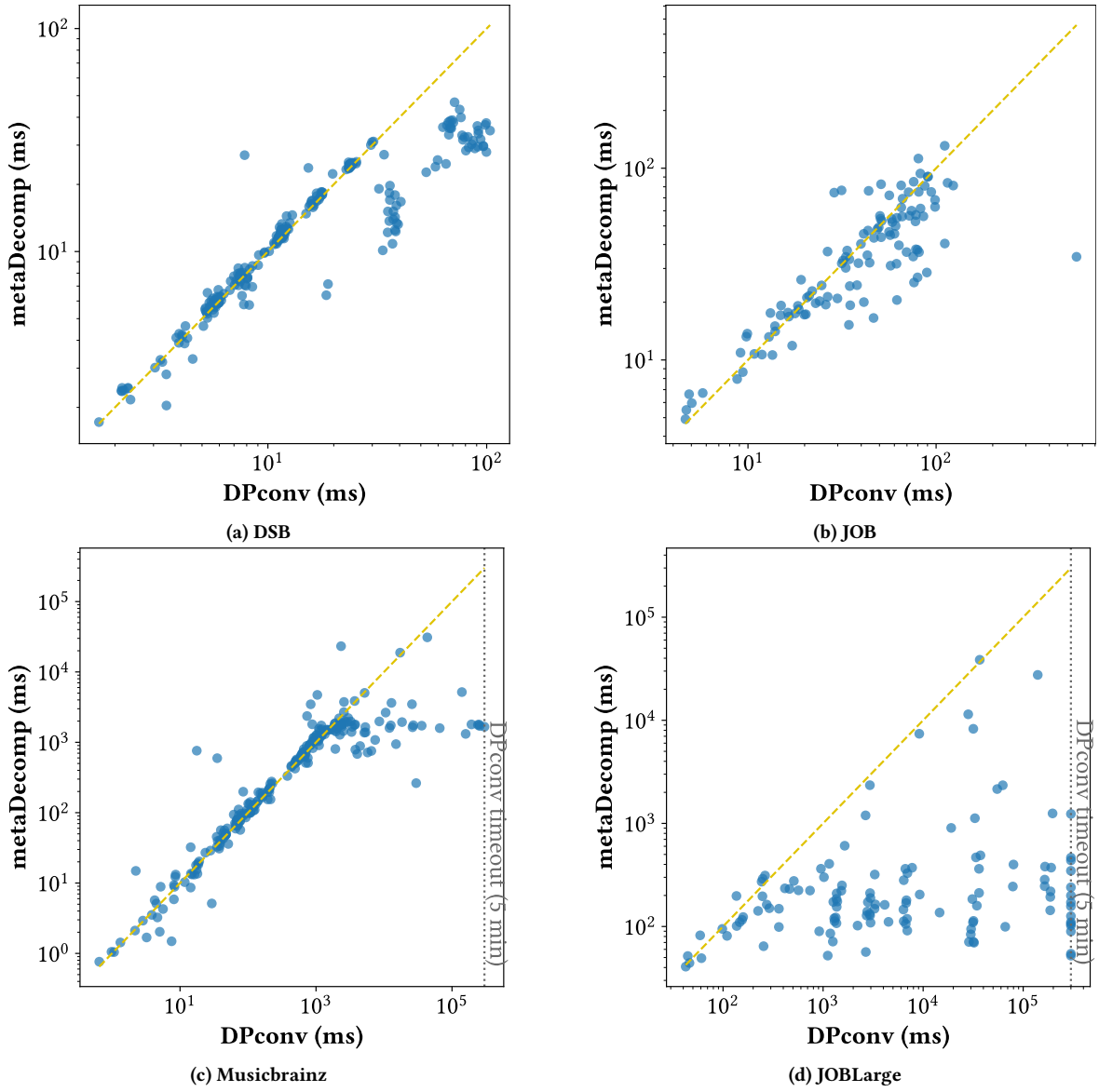



**Figure 16: Histograms of the speedups of overall query evaluation time of metaDecomp over DPconv on each benchmark**

*Scatter plots.* Fig. 17 shows scatter plots of the overall query evaluation time using metaDecomp and using DPconv measured for queries in each of the four benchmarks. The trends that can be observed in each of the four benchmarks and the overall figure (Section 6.2.3) are mostly consistent. The two approach have similar performance for queries that takes DPconv less than 1 second ($10^3$ ms). For those taking DPconv more than 1 second, metaDecomp starts to show significant speedups. Notably, for complex queries in Musicbrainz (Fig. 17c) and JOBLarge (Fig. 17d) for which DPconv times out (taking more than 5 minutes), metaDecomp can evaluate them within merely 1 second.




(a) DSB (b) JOB (c) Musicbrainz (d) JOBLarge

**Figure 17: Scatter plot of overall query evaluation time of using metaDecomp versus using DPconv on each benchmark.**

### F.3.2 metaDecomp versus DuckDB.

*Histograms of speedup.* Fig. 18 shows histograms of the speedups of the overall query execution time using metaDecomp over using DuckDB for queries in each of the four benchmarks. Most of the queries show a speedup close to 1, with some notable speedups especially for larger queries in the Musicbrainz benchmark.

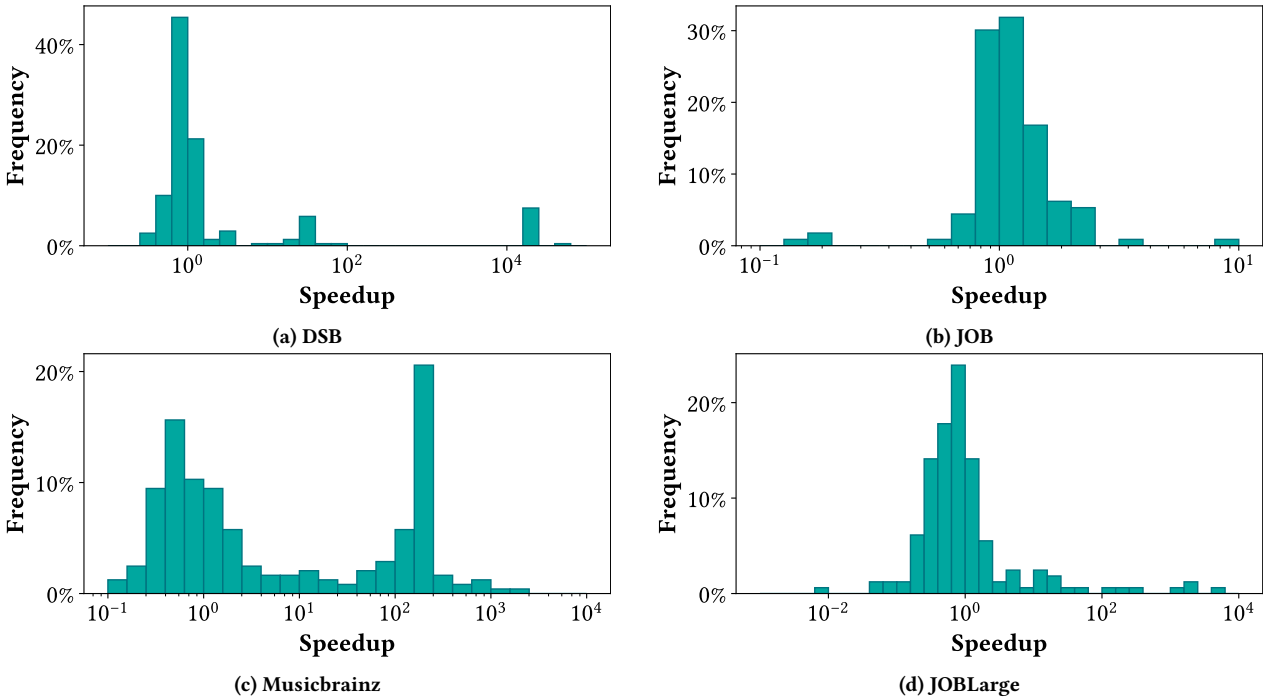



**Figure 18: Histograms of the speedups of overall query evaluation time of metaDecomp over DuckDB on each benchmark**

*Scatter plots.* Fig. 19 shows the detailed breakdown of overall query evaluation time using metaDecomp and using DuckDB measured for queries in each of the four benchmarks used in the paper. With the exception of JOBLarge, all approaches have similar performance for queries that takes DuckDB less than 100 ms. For those taking DuckDB more than 100 ms, metaDecomp starts to show significant speedups, mostly due to the greedy strategy of DuckDB used for large queries, which does not have optimality guarantees but may lead to highly suboptimal plans.

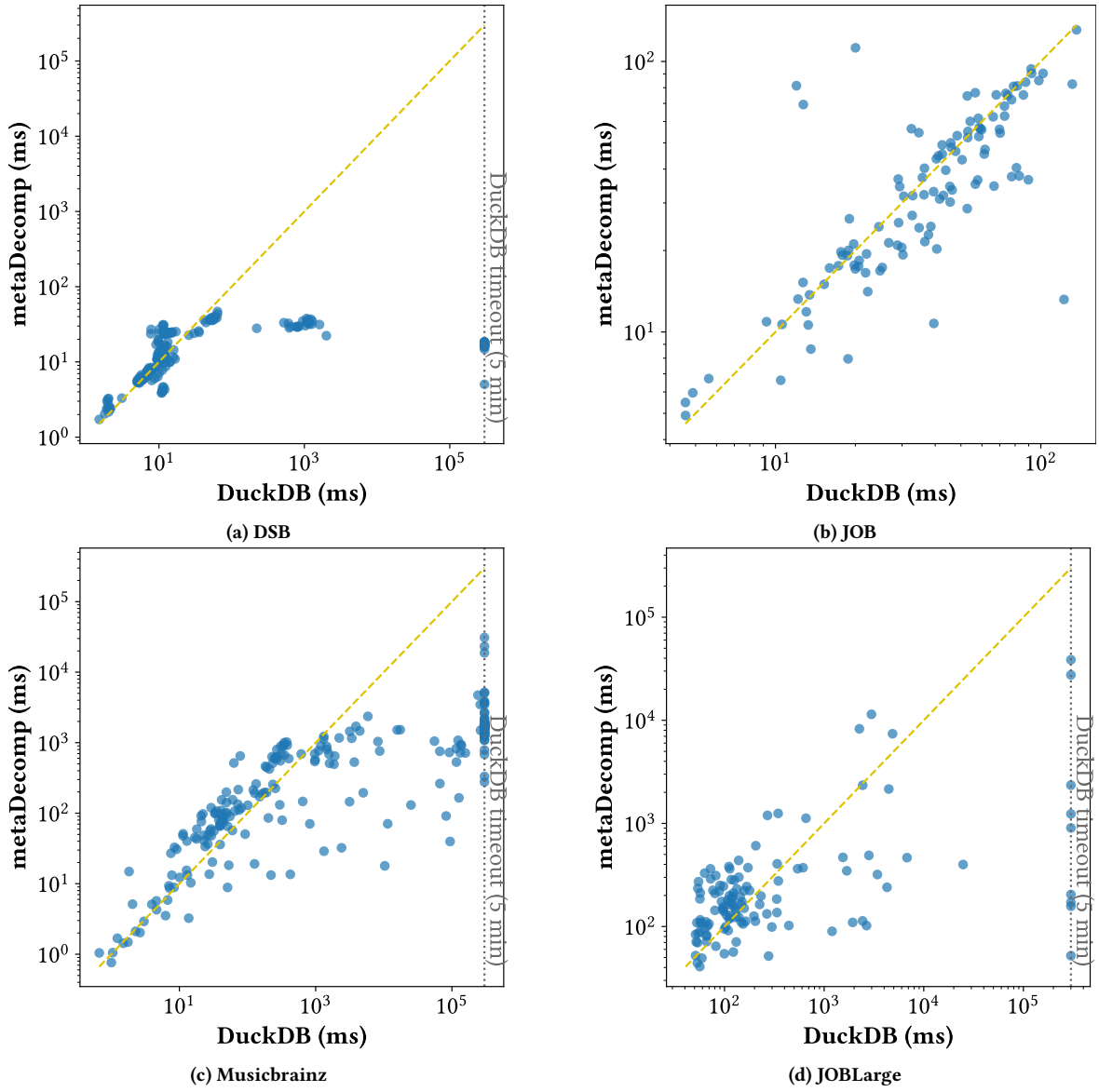



**Figure 19: Scatter plot of overall query evaluation time of using metaDecomp/DPconv versus using DuckDB on each benchmark.**

P9c
R1.O8

### *F.3.3 metaDecomp versus UnionDP.*

*Histograms of speedup.* Fig. 20 shows histograms of the speedups of the overall query execution time using metaDecomp over using UnionDP for queries in each of the four benchmarks. In the relatively smaller benchmarks DSB and JOB, the performance is similar. In Musicbrainz and JOBLarge, however, we observe a large number of significant speedups of multiple orders of magnitudes, mostly due to the fact that UnionDP starts to use the heuristic partitioning which may lead to highly suboptimal plans.

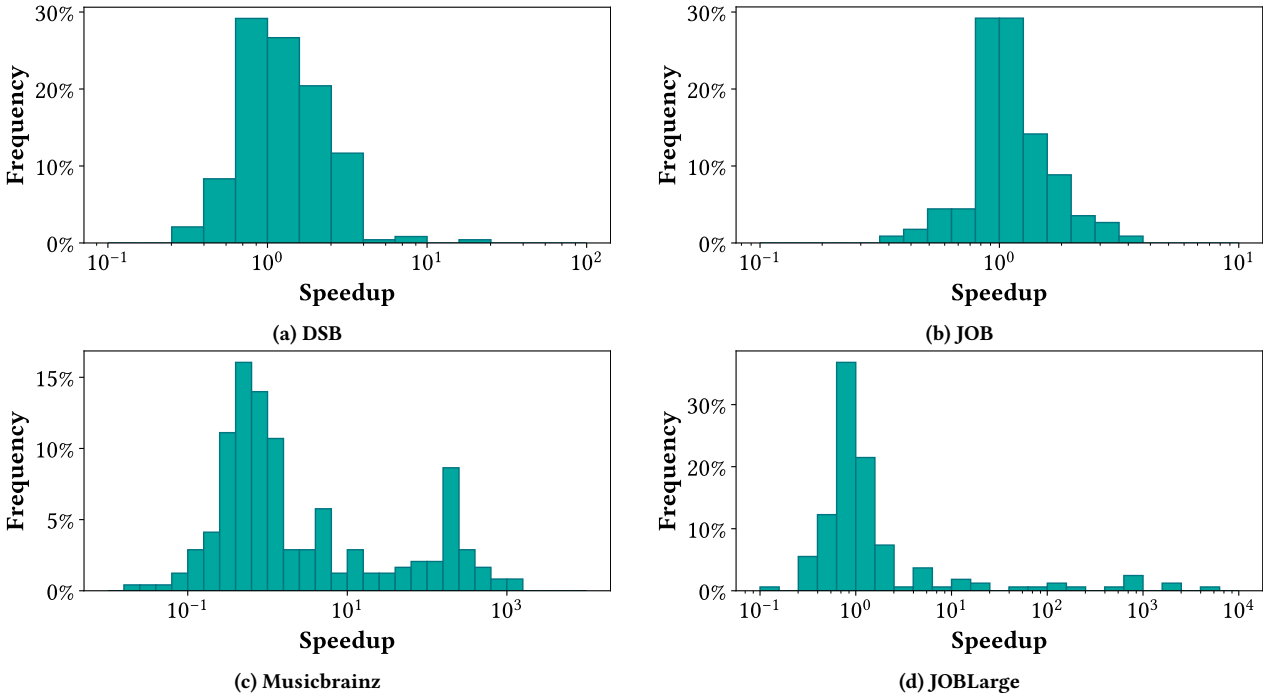
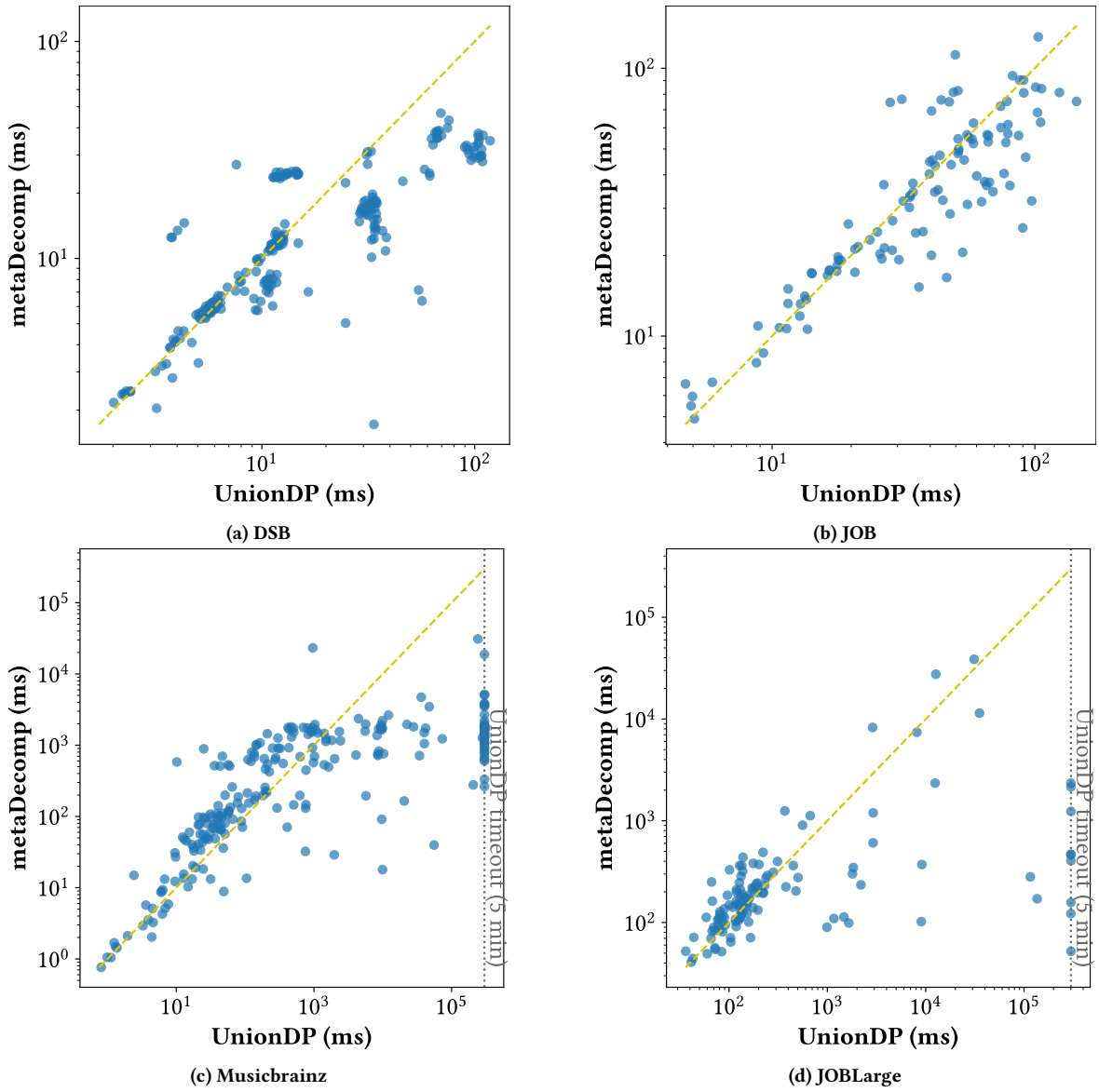



**Figure 20:** Histograms of the speedups of overall query evaluation time of metaDecomp over UnionDP on each benchmark

*Scatter plots.* Fig. 21 shows scatter plots of the overall query evaluation time using metaDecomp and using UnionDP measured for queries in each of the four benchmarks. The two approach have similar performance for queries that takes UnionDP less than 1 second ($10^3$ ms). For those taking UnionDP more than 1 second, metaDecomp starts to show significant speedups. This is especially the case in the Musicbrainz and JOBLarge benchmarks.




(a) DSB (b) JOB (c) Musicbrainz (d) JOBLarge

**Figure 21:** Scatter plot of overall query evaluation time of using metaDecomp versus using UnionDP on each benchmark.

P9a
R2.O1

### *F.3.4 metaDecomp versus Yannakakis$^+$.*

*Histograms of speedup.* Fig. 22 shows histograms of the speedups of the overall query execution time using metaDecomp over using Yannakakis$^+$ for queries in each of the four benchmarks. The major advantage of metaDecomp over Yannakakis$^+$ is the elimination of the overhead of full reduction, and this is very clearly reflected in the speedups in the smaller benchmarks DSB and JOB. On the larger benchmarks Musicbrainz and JOBLarge, metaDecomp also shows comparable performance, with significant speedups on a few queries in which the full reduction takes a very long time.

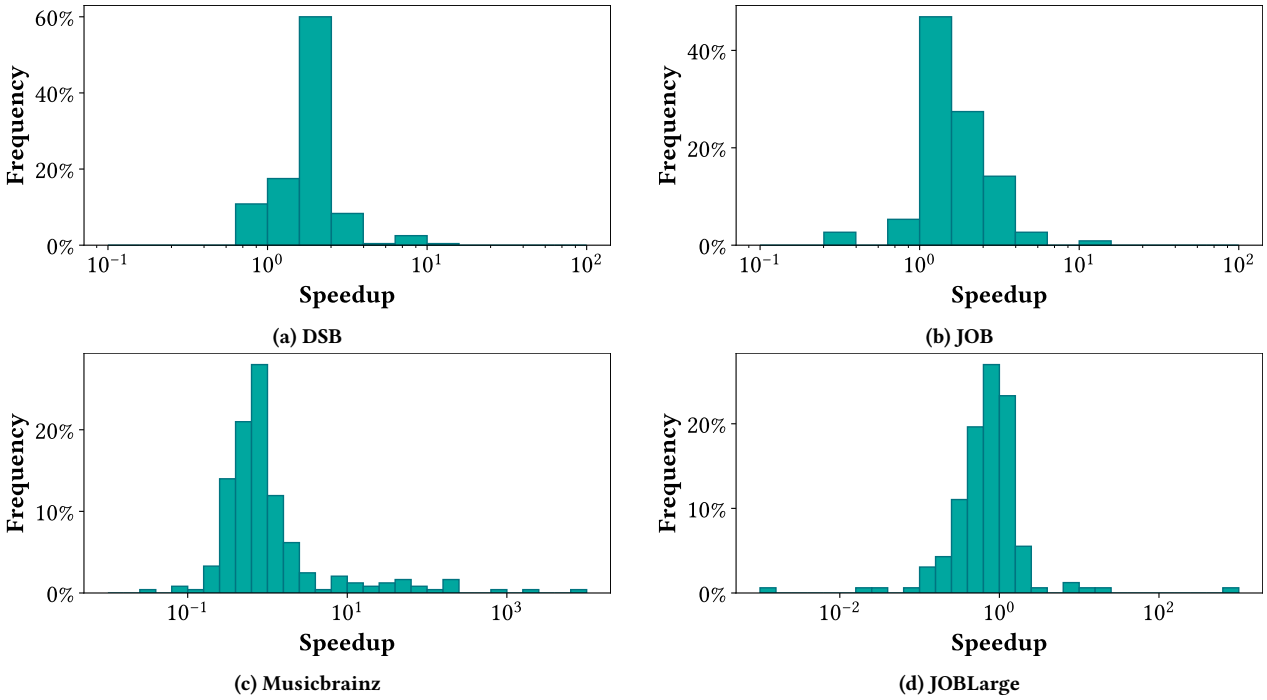
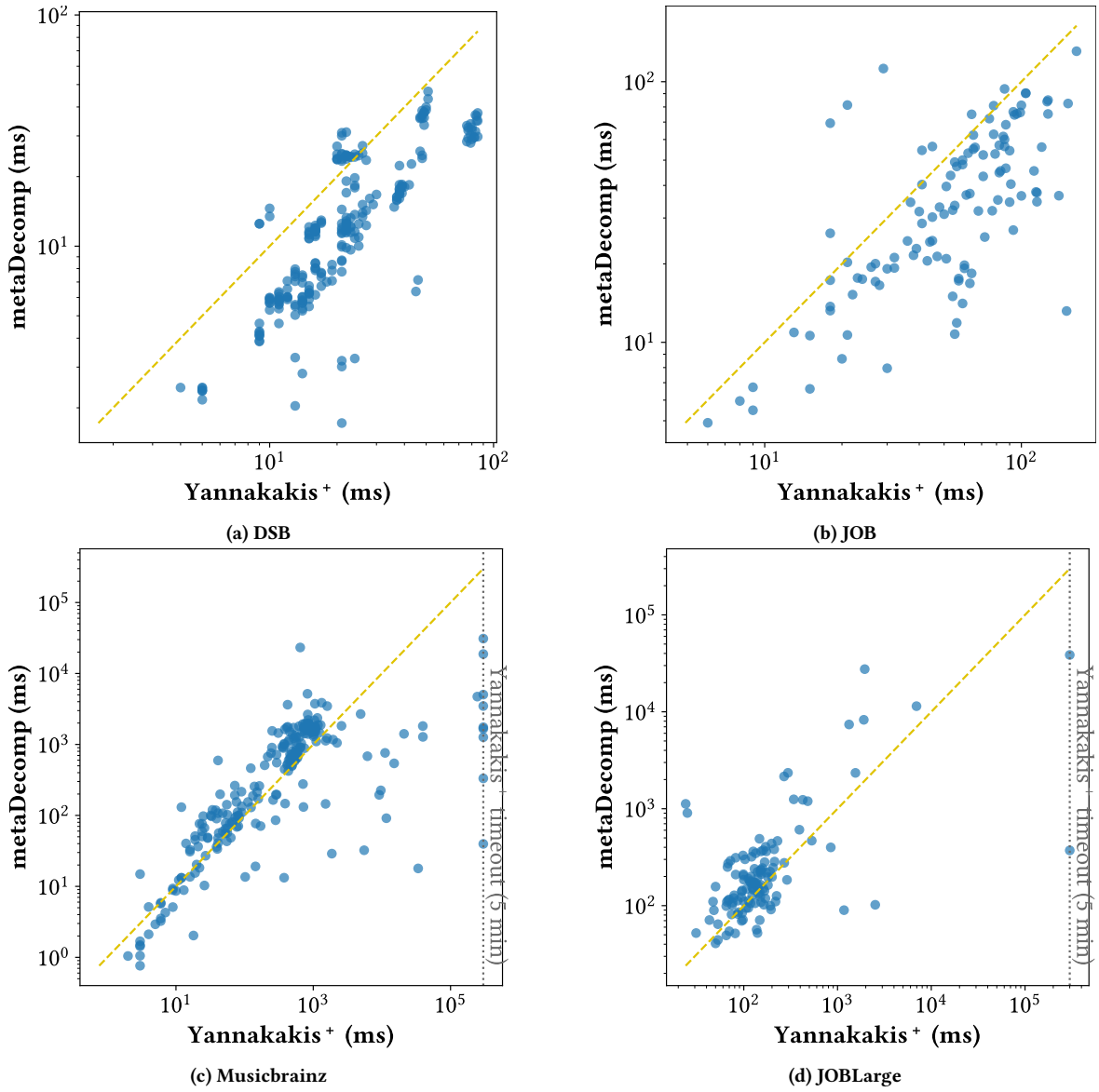



**Figure 22:** **Histograms of the speedups of overall query evaluation time of metaDecomp over Yannakakis$^+$ on each benchmark**

*Scatter plots.* Fig. 23 shows scatter plots of the overall query evaluation time using metaDecomp and using Yannakakis$^+$ measured for queries in each of the four benchmarks. Again, the elimination of the overhead of full reduction allows for speedups in many queries in all four benchmarks, especially in queries in metaDecomp also shows comparable performance, with significant speedups on a few queries in Musicbrainz and JOBLarge for which the full reduction takes a very long time.




(a) DSB (b) JOB (c) Musicbrainz (d) JOBLarge

**Figure 23:** Scatter plot of overall query evaluation time of using metaDecomp versus using Yannakakis$^+$ on each benchmark.

P9b R2.O2

### F.3.5 metaDecomp versus LearnedRewrite.

*Histograms of speedup.* Fig. 24 shows histograms of the speedups of the overall query execution time using metaDecomp over using Learned Rewrite for queries in each of the four benchmarks. We can observe a clear overall advantage of metaDecomp over Learned Rewrite, due to (1) the large optimization overhead and (2) the limitation of the cost-based optimization and rewrite rules of Learned Rewrite.

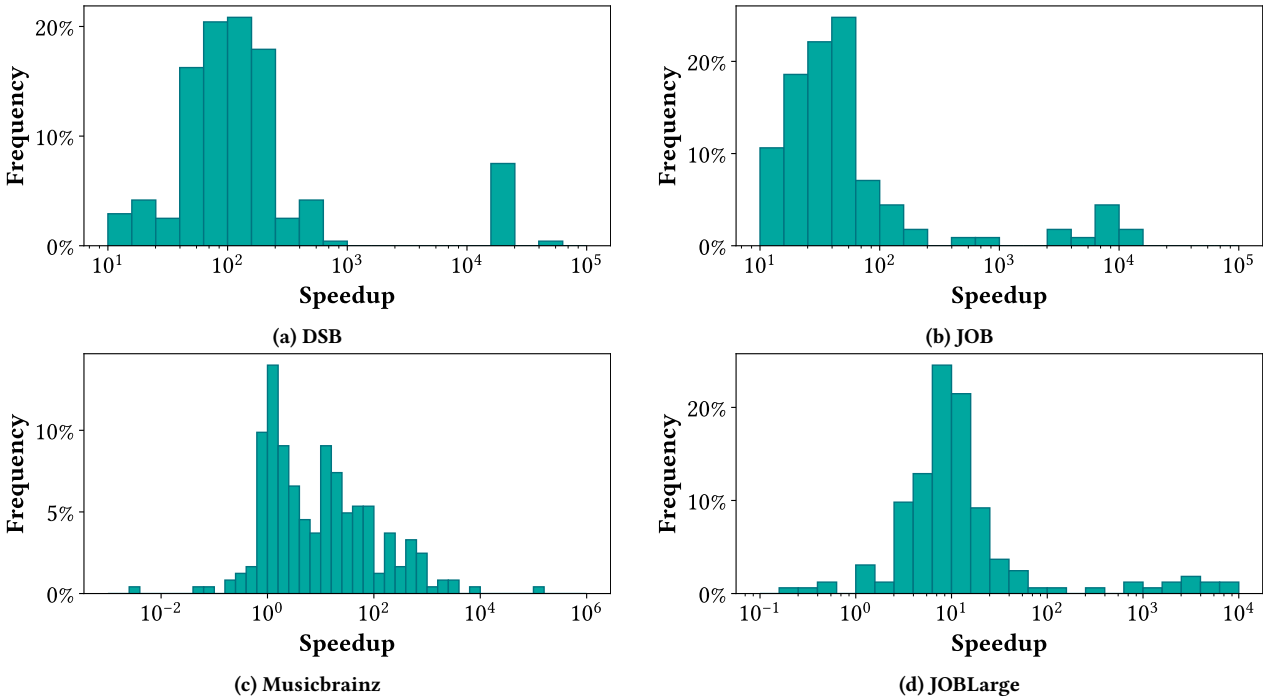



**Figure 24:** Histograms of the speedups of overall query evaluation time of metaDecomp over Learned Rewrite on each benchmark

*Scatter plots.* Fig. 25 shows scatter plots of the overall query evaluation time using metaDecomp and using Learned Rewrite, measured for queries in each of the four benchmarks. Again, we observe a clear overall advantage of metaDecomp over Learned Rewrite in all four benchmarks. In particular, it takes Learned Rewrite an overhead of at least 1 second to optimize each query. For many small queries, this is already more than the overall evaluation time for metaDecomp as well as other traditional query optimization approaches.

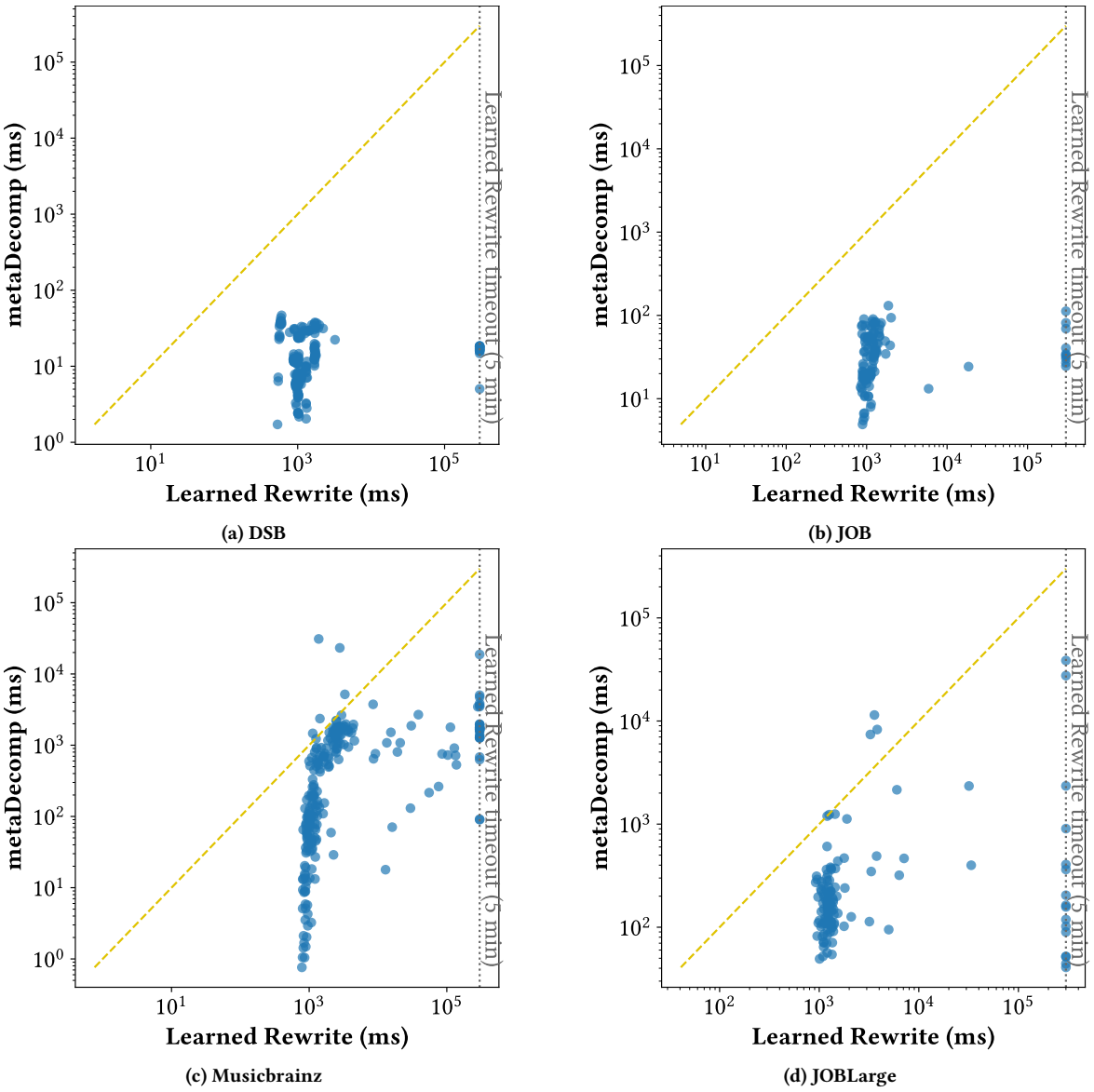



**Figure 25:** **Scatter plot of overall query evaluation time of using metaDecomp versus using Learned Rewrite on each benchmark.**

P9b
R2.O2

### F.3.6 metaDecomp versus LLM-R$^2$.

*Histograms of speedup.* Fig. 26 shows histograms of the speedups of the overall query execution time using metaDecomp over using LLM-R$^2$ for queries in each of the four benchmarks. Similar to the case for Learned Rewrite, we can observe a clear overall advantage of metaDecomp over LLM-R$^2$, due to (1) the large optimization overhead and (2) the limitation of the cost-based optimization and rewrite rules of LLM-R$^2$.

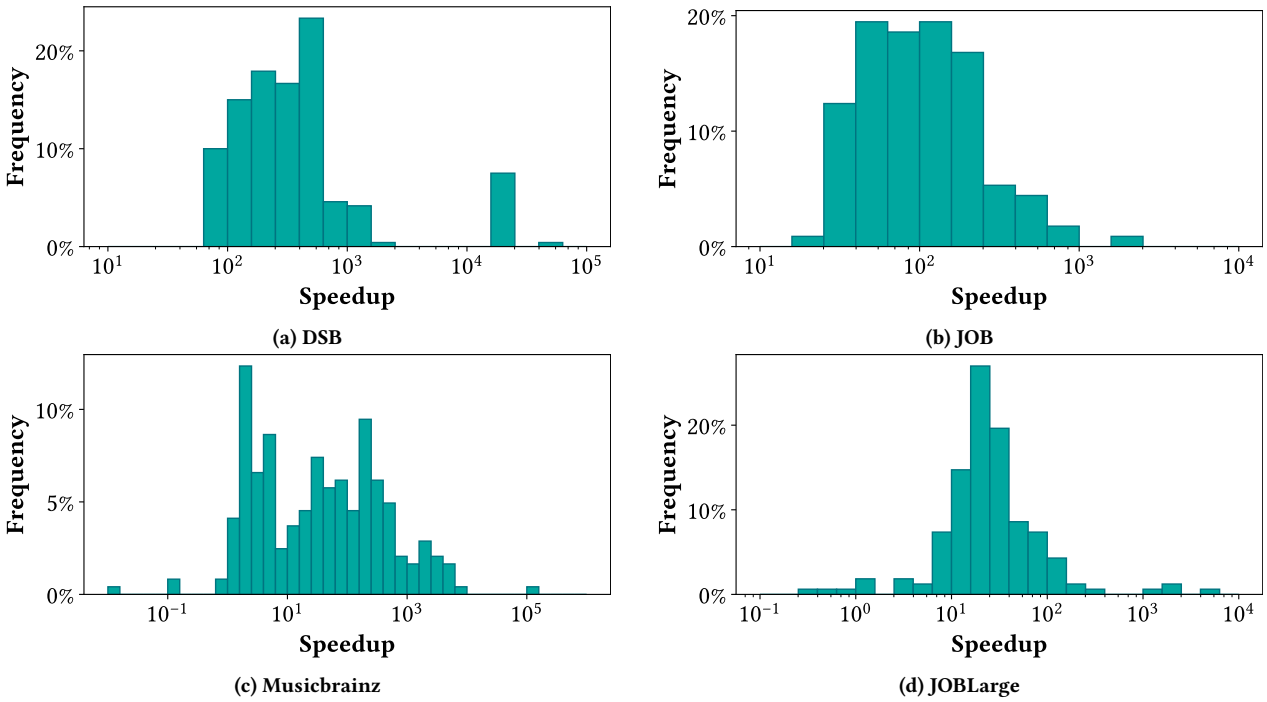



**Figure 26:** Histograms of the speedups of overall query evaluation time of metaDecomp over LLM-R$^2$ on each benchmark

*Scatter plots.* Fig. 27 shows scatter plots of the overall query evaluation time using metaDecomp and using LLM-$R^2$, measured for queries in each of the four benchmarks. Again, we observe a clear overall advantage of metaDecomp over LLM-$R^2$ in all four benchmarks. Similar to the case with Learned Rewrite, it takes LLM-$^2$ an even higher overhead of at least 3 seconds to optimize each query. For many small queries, this is already more than the overall evaluation time for metaDecomp as well as other traditional query optimization approaches.

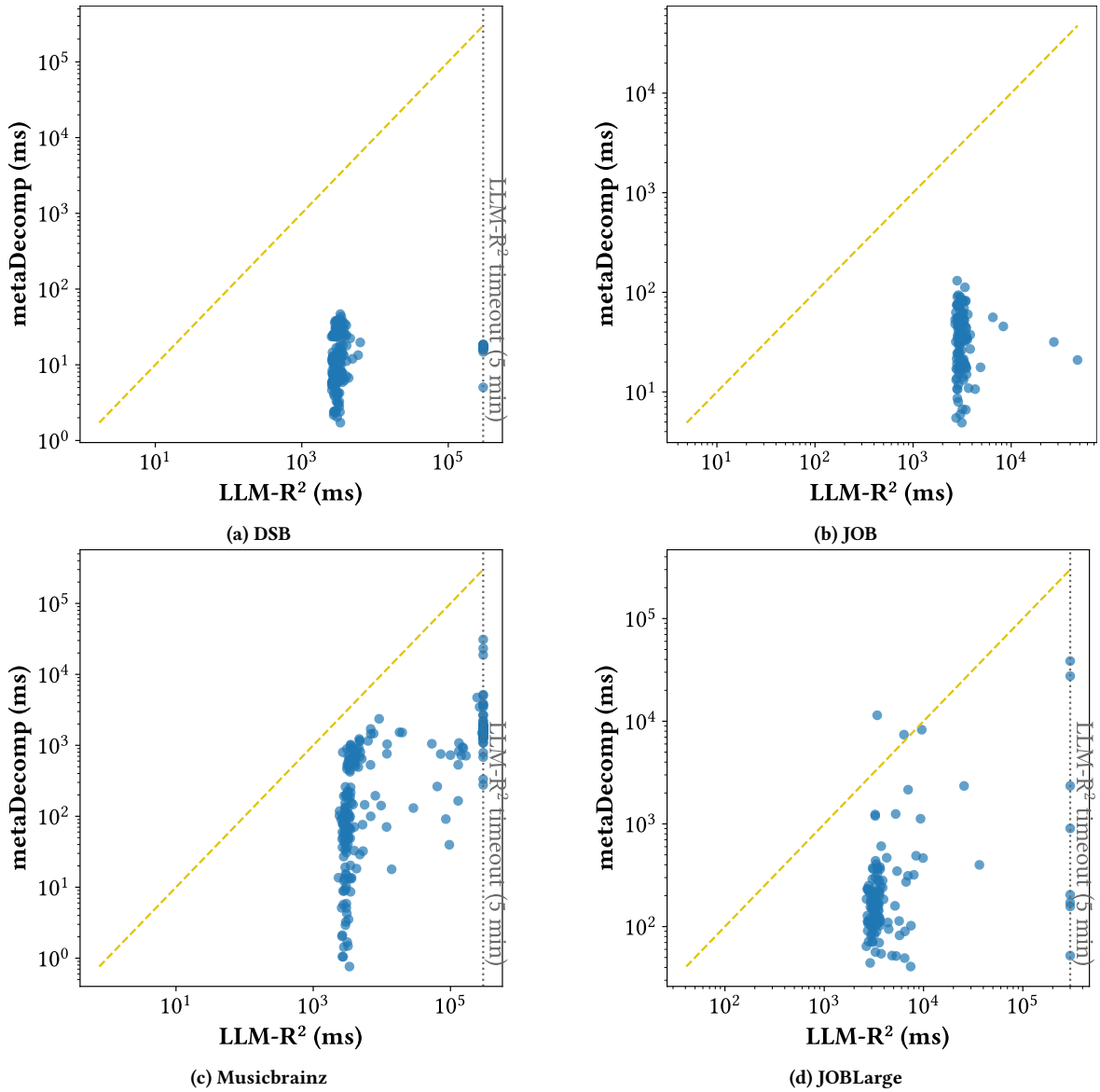
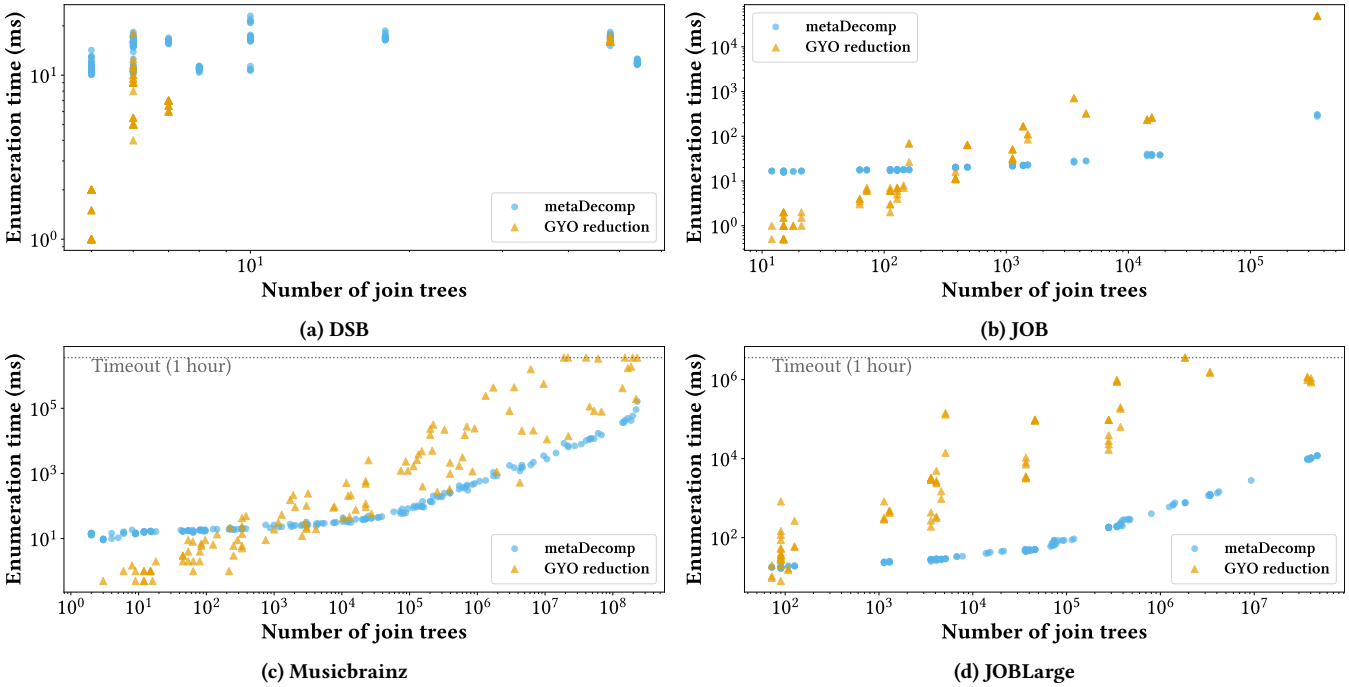



**Figure 27:** **Scatter plot of overall query evaluation time of using metaDecomp versus using LLM-$R^2$ on each benchmark.**

## F.4 Time to Enumerate Join Trees Using metaDecomp versus Using GYO Reduction on Each Benchmark

R1.O6

Fig. 28 shows the time it takes for metaDecomp and traditional GYO reduction to enumerate all join trees of queries with different numbers of join trees. The trends are similar to the one shown by Fig. 12 in the main body of the paper. Despite having an overhead for simpler queries, when the number of join trees reaches approximately 1000, metaDecomp starts to show a clear advantage because of its polynomial amortized enumeration delay, and the difference could be over 4 orders of magnitude. In particular, for many highly complicated queries in Musicbrainz with with more than $10^7$ join trees, the traditional approach already takes over 1 hour, whereas metaDecomp can enumerate all join trees in merely between 2 seconds and 1 minute.




(a) DSB (b) JOB (c) Musicbrainz (d) JOBLarge

**Figure 28:** Scatter plot of join tree enumeration time using metaDecomp versus using GYO reduction on each benchmark.